

AUTOR(A) DA DISSERTAÇÃO

Título da Dissertação



UNIVERSIDADE FEDERAL DE UBERLÂNDIA
FACULDADE DE MATEMÁTICA

2019

AUTOR(A) DA DISSERTAÇÃO

Título da Dissertação

Dissertação apresentada ao Programa de Pós-
Graduação...

UBERLÂNDIA - MG
2010

Dedicatória.

Agradecimientos

Agradecimientos.

Resumo

Algumas coisas que posso colocar no resumo.

A pecuária bovina é um dos pilares fundamentais da economia brasileira. O Brasil possui um dos maiores rebanhos bovinos comerciais do mundo e é o principal exportador de carne bovina, o que torna a avaliação precisa das características da carcaça dos animais uma demanda estratégica para o setor pecuário. A ultrassonografia permite mensurar, *in vivo*, indicadores como a Área de Olho de Lombo (AOL), a Espessura de Gordura no Lombo (EGL) e a Espessura de Gordura na Garupa (EGG), relacionados à musculosidade, precocidade de terminação e qualidade da carne. Entretanto, a delimitação manual das regiões de interesse em imagens ultrassonográficas é um processo subjetivo, demorado e dependente da experiência do especialista. Neste trabalho, foram desenvolvidos e comparados dois modelos de segmentação automática dessas regiões em imagens de ultrassom de bovinos das raças Nelore e Caracu: uma rede neural convolucional U-Net com abordagem *Multitask* e uma U-Net *Multitask* integrada ao Sistema de Inferência Neuro-Fuzzy Adaptativo (ANFIS). As imagens foram obtidas de animais vivos no Centro Avançado de Pesquisa Tecnológica do Agronegócio (CAPTA) em Sertãozinho-SP, totalizando 135 imagens (108 Nelore e 27 Caracu), ampliadas por técnicas de expansão do conjunto de dados. As máscaras de segmentação foram criadas manualmente com o auxílio de uma especialista, servindo como referência para o treinamento e avaliação dos modelos. O desempenho dos métodos foi avaliado por meio do coeficiente de Dice, que quantifica a similaridade entre a segmentação produzida pelo modelo e a referência do especialista. Os resultados indicam que ... [completar com os resultados comparativos entre U-Net e U-Net+ANFIS]. O trabalho contribui para a automatização da mensuração de características da carcaça bovina, podendo auxiliar nos processos de seleção e manejo dos rebanhos.

Palavras-chave: Segmentação de imagens. Ultrassonografia bovina. U-Net. ANFIS. Conjuntos fuzzy. Área de Olho de Lombo. Redes neurais convolucionais.

foi quando a vida me deu um doce

Lista de Figuras

1	Ilustração da localização das regiões de interesse.	14
2	Imagens de ultrassom das regiões de interesse.	15
3	Exemplo de segmentação de uma imagem de ultrassom.	16
4	Fluxo de treinamento e comparação dos modelos propostos para a obtenção das medidas de AOL, EGL e EGG.	18
1.1	Estrutura de um neurônio típico.	20
1.2	Modelo de uma unidade de McCulloch e Pitts.	21
1.3	Gráficos das Funções de Ativação.	23
1.4	Representação da taxa de aprendizagem.	25
1.5	Organização da Seção 1.2.	26
1.6	Estrutura visual básica de uma unidade e uma rede neural com duas camadas ocultas.	27
1.7	Arquiteturas de Redes Neurais.	29
1.8	Ilustração do processo de <i>backpropagation</i> em uma rede neural.	30
1.9	Comparação entre o método de descida do gradiente tradicional e o Gradiente Descendente Estocástico.	32
1.10	Ilustração do <i>Overfitting</i>	33
1.11	Ilustração do <i>Dropout</i> removendo 60% das unidades aleatoriamente durante uma época de treinamento.	35
1.12	Ilustração do <i>Early Stopping</i>	35
1.13	Gráfico da derivada da função sigmoid logística.	37
1.14	Exemplos de dígitos manuscritos do conjunto de dados MNIST.	38
1.15	Arquitetura da rede neural para classificação de dígitos manuscritos. Inicialmente, é enviado o número de pixels da imagem (784) para a camada de entrada (unidade $x_{784} = 0,4$ intensidade do pixel), que é processada por uma camada oculta com 30 unidades, e a camada de saída tem 9 unidades, correspondendo às 10 classes de dígitos (0 a 9).	39
1.16	Fluxo de dados durante o processo de treinamento e avaliação da rede neural.	40
1.17	Paraboloide hiperbólico.	41
1.18	Gráfico da superfície e aproximação da superfície obtida pela MLP e a projeção do E no plano xy.	41

1.19	Exemplo de uma imagem/matriz 10×10 em escala de cinza, em que cada pixel tem um valor de intensidade entre 0 (preto) e 255 (branco).	43
1.20	Ilustração da operação de convolução discreta. O kernel w é deslizado sobre o vetor x , e em cada posição, a soma ponderada dos valores de x cobertos pelo kernel é calculada para gerar o valor correspondente na saída.	46
1.21	Exemplo de convolução bidimensional. A matriz de entrada X (matriz 7×7) é convoluída com o <i>kernel</i> W (matriz 2×2) para produzir a saída S (matriz 6×6). O <i>kernel</i> é “deslizado” sobre a matriz de entrada, formando sub-matrizes, e em cada posição, a soma ponderada dos valores dos pixels cobertos pelo <i>kernel</i> é calculada e o resultado é aplicado em uma função de ativação (σ) para gerar o valor correspondente ($\hat{y}_{11}$) na matriz de saída. b sendo o viés adicionado à soma ponderada.	48
1.22	Aplicações de diferentes <i>kernels</i> em uma imagem. A imagem original é representada em (a), juntamente com o <i>kernel</i> identidade, que não faz modificações. Em (b) a imagem passou por um <i>kernel</i> que aumenta a nitidez reforçando o pixel central e aumentando a diferença dele em relação à sua vizinhança. Em (c) foi usado um <i>kernel</i> de desfoque, que substitui um pixel pela média dele com seus vizinhos. A imagem em (d) é o resultado de aplicar na imagem em escala de cinza um <i>kernel</i> que reforça suas linhas verticais, enquanto (e) reforça suas linhas horizontais. O <i>kernel</i> em (f) reforça os contornos (bordas) da imagem. . .	49
1.23	Aplicação de <i>padding</i> em uma imagem. Em (a) a imagem original passou por zero <i>padding</i> . Em (b) a imagem original passou por <i>reflection padding</i> . A imagem em (c) passou por um <i>padding</i> constante, semelhante em (a), mas com valor diferente de zero.	51
1.24	Exemplo de Max <i>pooling</i> com tamanho de 2×2 . A imagem original é reduzida para a imagem menor após a aplicação do Max <i>pooling</i> . Cada bloco de 2×2 pixels na imagem original é substituído pelo valor máximo dentro desse bloco na imagem reduzida.	52
1.25	Arquitetura típica de uma Rede Neural Convolucional com uma camada de convolução e uma camada de <i>pooling</i> , em que é enviado para rede a imagem de um cachorro e é feita a classificação do objeto na imagem.	53
1.26	Exemplo da operação de convolução transposta.	55
2.1	Operações entre conjuntos fuzzy.	58
2.2	Arquitetura de um Sistema Baseado em Regras Fuzzy (SBRF).	62
2.3	Exemplo de inferência fuzzy utilizando o método de Mamdani.	63
2.4	Exemplo de inferência fuzzy utilizando o método de Takagi-Sugeno.	64
3.1	Imagem de entrada X e resposta desejada Y	68

3.2	Aplicação de um <i>kernel</i> de convolução na imagem de entrada X . O resultado da convolução é um mapa de características que destaca as regiões de interesse na imagem.	68
3.3	Aplicação de padding na imagem de entrada X para preservar suas dimensões durante a operação de convolução.	69
3.4	Aplicação da função de ativação ReLU no mapa de características resultante da convolução.	69
3.5	Aplicação de uma camada de <i>Max pooling</i> no mapa de características resultante.	70
3.6	Aplicação da operação de convolução transposta no mapa de <i>pooling</i> para expandir suas dimensões espaciais e voltar às dimensões originais da imagem de entrada.	70
3.7	Estrutura em forma de U da arquitetura U-Net. O <i>encoder</i> é responsável por extrair características da imagem de entrada, enquanto o <i>decoder</i> reconstrói a imagem, utilizando as características extraídas pelo <i>encoder</i>	71
3.8	Exemplo de conexões de atalho (<i>skip connections</i>) na arquitetura U-Net.	71
3.9	Exemplo de conexões de atalho (<i>skip connections</i>) na arquitetura U-Net. As conexões de atalho permitem que as informações de baixo nível, como bordas e texturas, sejam diretamente transferidas do <i>encoder</i> para o <i>decoder</i> , facilitando a reconstrução precisa da imagem segmentada.	72
3.10	Aplicação de uma última camada de convolução para gerar a imagem de saída Y . O <i>kernel</i> utilizado tem dimensões $(1 \times 1 \times 2)$, um canal para cada imagem resultante da concatenação.	73
3.11	Aplicação de um limiar (<i>threshold</i>) para gerar a imagem binária final. A imagem segmentada resultante da aplicação da função de ativação sigmoid tem valores entre 0 e 1, representando a probabilidade de cada pixel pertencer à classe segmentada. O limiar é aplicado para converter esses valores em uma imagem binária, onde pixels com valores acima do limiar são classificados como pertencentes à classe segmentada (1), enquanto pixels com valores abaixo do limiar são classificados como pertencentes ao plano de fundo (0).	73
3.12	Arquitetura completa da U-Net.	75
3.13	Função de pertinência gaussiana com centro em $c = 0$ e largura $\sigma = 1$	78
3.14	Arquitetura da U-Net com a adição de uma camada <i>Multitask</i> e uma camada de inferência fuzzy.	79
4.1	Região do olho de lombo (em amarelo), localizada entre a 12 ^a e 13 ^a costela e da garupa (em azul).	81
4.2	Aquisição da imagem de ultrassom da região do olho de lombo.	82
4.3	Demarcação manual da Área de Olho de Lombo (AOL) utilizando o aplicativo <i>ImageJ</i>	83
4.4	Demarcação manual da Espessura de Gordura no Lombo (EGL) utilizando o aplicativo <i>ImageJ</i>	84

4.5	Demarcação manual da Espessura de Gordura na Garupa (EGG) utilizando o aplicativo <i>ImageJ</i>	85
4.6	Exemplo do processo de pré-processamento das imagens de ultrassom.	87
4.7	Exemplo das máscaras de segmentação criadas para as imagens de ultrassom. . .	89
4.8	Arquitetura da U-Net utilizada e as dimensões das imagens ao longo da rede. . .	91

Lista de Tabelas

1.1	Banco de dados com as avaliações dos colegas e a decisão final de compra.	23
-----	---	----

Sumário

Agradecimentos	4
Resumo	5
Lista de Figuras	10
Lista de Tabelas	11
Introdução	18
1 Redes Neurais	19
1.1 Neurônio Biológico e Artificial	19
1.1.1 Neurônio Biológico	19
1.1.2 Unidade	20
1.1.3 Tipos de Função de Ativação	21
1.1.4 Perceptron e Algoritmo de Aprendizagem	22
1.2 Redes Neurais	26
1.2.1 Arquiteturas de Redes Neurais	27
1.2.2 <i>Backpropagation</i>	29
1.2.3 Gradiente Descendente Estocástico	31
1.2.4 <i>Overfitting</i>	32
1.2.5 Hiperparâmetros	33
1.2.6 Regularização	34
1.2.7 <i>Momentum</i> e Adam	35
1.2.8 <i>Vanishing Gradient</i>	36
1.2.9 Função de Perda <i>Binary Cross-Entropy</i>	37
1.2.10 Exemplos	38
1.3 Rede Neural para Análise de Imagens	41
1.3.1 Córtex Visual Humano e Visão Computacional	41
1.3.2 Redes Neurais Convolucionais	42
1.3.3 Convolução	43
1.3.4 Correlação Cruzada	50
1.3.5 <i>Kernel</i> Multicamadas	50

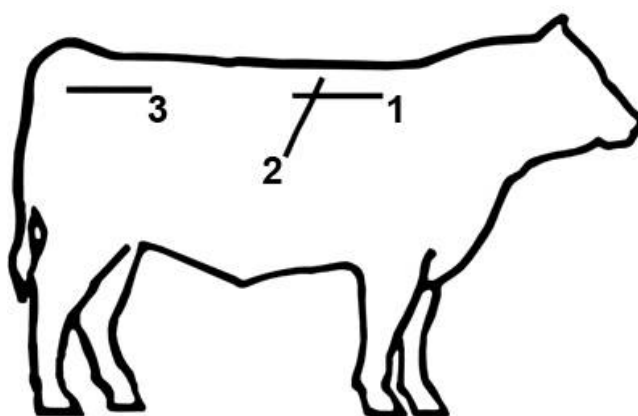
1.3.6	<i>Padding</i>	50
1.3.7	<i>Stride</i>	51
1.3.8	<i>Pooling</i>	52
1.3.9	Convolução Transposta	54
2	Teoria dos Conjuntos Fuzzy	56
2.1	Teoria dos Conjuntos Fuzzy	56
2.1.1	Conjuntos Fuzzy	56
2.1.2	Operações entre Conjuntos Fuzzy	57
2.1.3	Normas Triangulares	58
2.1.4	Níveis de um Conjunto Fuzzy	59
2.1.5	Números Fuzzy	60
2.2	Sistema Baseado em Regras Fuzzy	60
2.2.1	Regras e Inferência Fuzzy	61
2.2.2	Sistema Baseado em Regras Fuzzy	61
3	U-Net e U-Net com Sistema Adaptativo de Inferência Neuro-Fuzzy	67
3.1	U-net: Rede Convolutacional para Segmentação de Imagens	67
3.1.1	U-Net	74
3.1.2	Dice	75
3.1.3	U-Net <i>Multitask</i> com Inferência Fuzzy	76
4	Segmentação de Imagens de Ultrassom Bovino: Metodologia e Resultados	80
4.1	Descrição do Problema	80
4.1.1	Raças Nelore e Caracu	80
4.1.2	Área de Olho de Lombo (AOL), Espessura de Gordura no Lombo (EGL) e Espessura de Gordura na Garupa (EGG)	83
4.1.3	Tensorflow/Keras	85
4.1.4	Aquisição e Pré-processamento das Imagens	86
4.1.5	Máscaras	88
4.2	Treinamento e Métricas de Avaliação	89
4.2.1	<i>Hardware e Software</i>	93
4.2.2	Resultados e Análise	94
4.2.3	AOL	94
4.2.4	EGL	94
4.2.5	EGG	94

Introdução

Ao longo da história do país, o desenvolvimento econômico e social esteve profundamente enraizado em atividades agrícolas e pecuárias (GARCAO, 2015). De acordo com a Associação Brasileira das Indústrias Exportadoras de Carnes (ABIEC) (ABIEC, 2021), o complexo agroindustrial voltado à produção de carne bovina representa cerca de 8,4% do Produto Interno Bruto (PIB) nacional. Além disso, o Brasil possui o maior rebanho bovino comercial do mundo, correspondendo a 11,6% do total mundial, e ocupa a primeira posição no comércio exterior de carne bovina, sendo responsável por aproximadamente 21% da carne negociada internacionalmente (GUARALDO, 2021).

Nesse cenário, a avaliação precisa das características da carcaça dos animais desempenha um papel crucial para a cadeia produtiva. Entre os métodos disponíveis, a ultrassonografia se destaca por possibilitar o exame *in vivo*, de forma rápida, com boa precisão e custos relativamente baixos (SUGISAWA; MATOS; SUGISAWA, 2013). Por meio de imagens de ultrassom, é possível mensurar características como a Área de Olho de Lombo (AOL), a Espessura de Gordura no Lombo (EGL) e a Espessura de Gordura na Garupa (EGG), indicadores diretamente relacionados à musculosidade, precocidade de terminação e qualidade da carne (WILSON, 1998), localizadas nas regiões de Olho do Lombo (contra-filé), entre a 12^a e a 13^a costela (AOL e EGL) e Garupa (EGG), medida no começo do músculo *Biceps Femoris* (ponta da picanha). A Figura 1 mostra uma ilustração da localização das medidas no animal. A linha

Figura 1: Ilustração da localização das regiões de interesse.

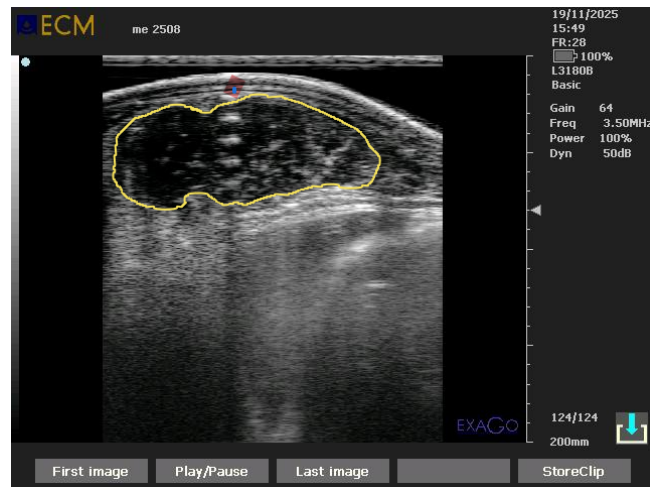


Fonte: Adaptado de (Grupo Agro Paraná, 2024).

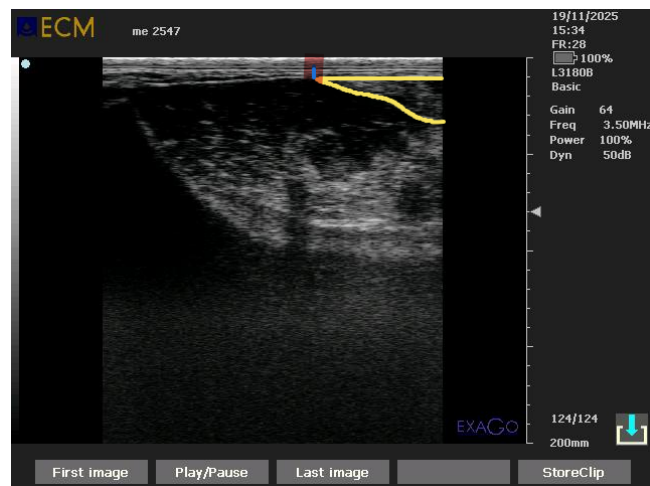
horizontal (1) indica a localização da Gordura no lombo onde é possível medir a EGL, a linha vertical (2) indica a localização do Olho do Lombo onde é possível medir a AOL e a linha

horizontal (3) indica a localização da Garupa onde é possível medir a EGG. Nestas regiões, as imagens de ultrassom obtidas são mostradas na Figura 2.

Figura 2: Imagens de ultrassom das regiões de interesse.



(a) AOL e EGL.



(b) EGG.

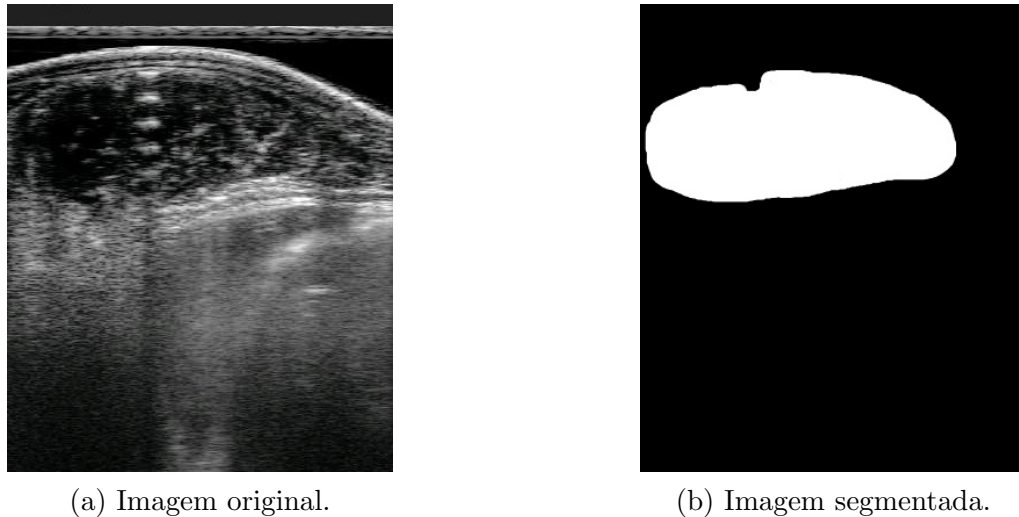
Fonte: Elaboração do Autor.

A região dentro da delimitação amarela na Figura 2 (a) corresponde à AOL, a linha azul dentro da delimitação vermelha, logo acima, indica a localização da EGL. A região dentro da delimitação vermelha, linha azul, na Figura 2 (b) corresponde à EGG, sendo o *biceps femoris* a demarcação amarela. A variabilidade nessas características influencia a comercialização e os resultados econômicos dos produtores (FELICIO, 2010), tornando sua mensuração precisa uma demanda constante do setor.

Tradicionalmente, a análise dessas imagens de ultrassom e a demarcação das regiões de interesse são realizadas manualmente por especialistas, um processo sujeito à subjetividade e que demanda tempo considerável. Diante disso, a aplicação de técnicas de visão computacional e inteligência artificial (SZELISKI, 2010) apresenta-se como uma alternativa promissora para automatizar e padronizar esse processo de mensuração. Em particular, arquiteturas de redes neurais convolucionais (*Convolutional Neural Networks* - CNNs) (O'SHEA; NASH, 2015), uma

subárea da inteligência artificial dedicada à análise e interpretação de imagens, têm demonstrado resultados expressivos em diversas aplicações biomédicas (GOODFELLOW; BENGIO; COURVILLE, 2016). Uma das principais técnicas empregadas nessas redes é a segmentação de imagens, que consiste em classificar cada pixel em uma categoria específica, possibilitando a identificação precisa das regiões de interesse. A Figura 3 ilustra o processo de segmentação.

Figura 3: Exemplo de segmentação de uma imagem de ultrassom.



Fonte: Elaboração do Autor.

A imagem original, Figura 3 (a), a região de interesse não está demarcada, podendo ser visível apenas por um especialista, enquanto na imagem segmentada, Figura 3 (b), a região de interesse foi identificada e destacada, facilitando a extração das medidas de AOL (área branca).

Entre as arquiteturas de segmentação, a U-Net¹ (RONNEBERGER; FISCHER; BROX, 2015) se destaca como principal referência. Trata-se de uma arquitetura *encoder-decoder*: o *encoder* e a etapa que extrai características progressivamente abstratas da imagem, enquanto o *decoder* as utiliza para reconstruir a segmentação. Originalmente proposta para segmentação de imagens biomédicas, a U-Net é reconhecida pela capacidade de operar com conjuntos de dados reduzidos e pela eficiência na captura de informações contextuais e de detalhes finos das imagens. Quando é necessário que a arquitetura faça algo além da segmentação, como por exemplo, segmentação e classificação, utiliza-se uma abordagem denominada *Multitask* (Multitarefa). Essa abordagem permite que a rede aprenda a realizar múltiplas tarefas simultaneamente, compartilhando representações e melhorando o desempenho geral do modelo. Paralelamente, a teoria dos conjuntos fuzzy (ZADEH, 1965) oferece ferramentas para o tratamento de incertezas e imprecisões inerentes a imagens de ultrassom, onde as fronteiras entre as estruturas anatômicas nem sempre são claramente definidas. A integração de redes neurais com a teoria dos conjuntos fuzzy (PURWONO *et al.*, 2025), por meio do Sistema de Inferência Neuro-Fuzzy Adaptativo (ANFIS), possibilita combinar a capacidade de aprendizado das redes neurais com a flexibilidade de um sistema baseado em regras fuzzy (SBRF) no tratamento de informações imprecisas.

¹O nome U-Net vem do formato da arquitetura da rede, que se assemelha à letra U.

Alguns estudos têm explorado a aplicação da U-Net em tarefas de segmentação de imagens ultrassom da região de olho de lombo (MELO *et al.*, 2022) e sua junção com a teoria dos conjuntos fuzzy (KORSHUNOVA, 2018).

Neste contexto, este trabalho tem como objetivo desenvolver e comparar dois modelos de segmentação de imagens de ultrassom de bovinos das raças Nelore e Caracu:

- Uma rede U-Net com abordagem *Multitask*;
- Uma U-Net *Multitask* com a integração do ANFIS.

Foram utilizadas 135 imagens de ultrassom obtidas de animais vivos, localizados no Centro Avançado de Pesquisa Tecnológica do Agronegócio (CAPTA) em Sertãozinho-SP, por meio do aparelho de ultrassom Aloka SSD-500. O intuito é avaliar a precisão e a eficiência dos modelos propostos na segmentação automática das regiões do olho de lombo e da garupa, viabilizando a obtenção automatizada das medidas de AOL, EGL e EGG. Por conta do número limitado de imagens disponíveis, foi necessário realizar um processo de aumento de dados (*data augmentation*) para ampliar o conjunto de treinamento e melhorar a generalização dos modelos. O processo de aumento de dados incluiu técnicas como rotação e adição de ruído às imagens originais, simulando variações que podem ocorrer durante a aquisição das imagens de ultrassom. Para essa avaliação, utilizam-se quatro métricas:

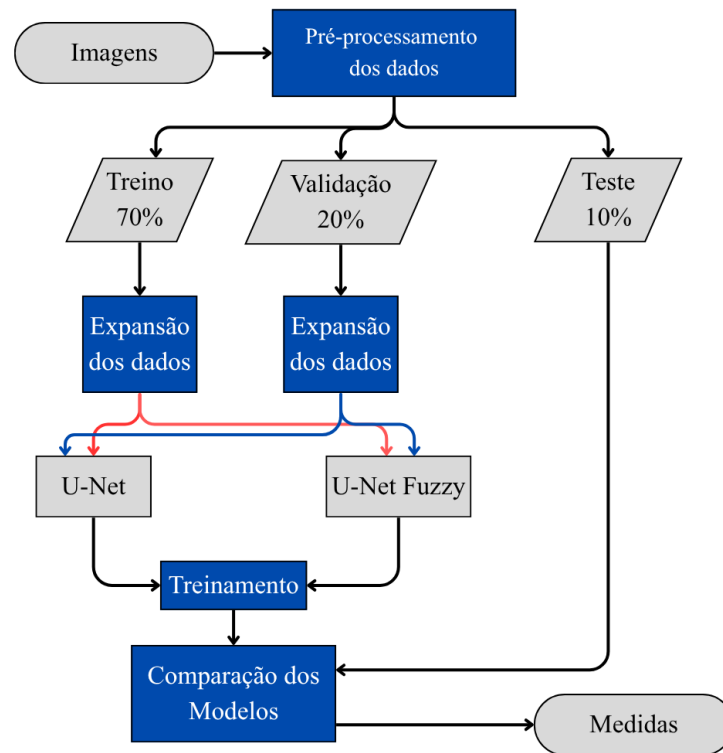
- O coeficiente de Dice (HARISHA, 2023), que quantifica a similaridade entre a imagem segmentada pelo modelo e a imagem de referência demarcada pelo especialista.
- Erro Médio Absoluto (*Mean Absolute Error* – MAE), que compara as medidas extraídas da imagem segmentada com as medidas obtidas pelo especialista.
- Acurácia, que avalia a capacidade do modelo em classificar corretamente as regiões de interesse.
- R^2 , que mede a proporção da variância explicada pelo modelo em relação à variância total.

Foram treinadas duas redes neurais para cada medida, uma com fuzzy e outra sem e foi feito uma correlação cruzada, que consiste em treinar cada modelo uma quantidade de vezes, para avaliar a consistência dos resultados. Neste caso, cada modelo foi treinado quatro vezes. A figura 4 apresenta o fluxo de treinamento e comparação dos modelos propostos, assim como a divisão do banco de dados em conjuntos de treinamento, validação e teste.

Com isso, pretende-se não apenas automatizar a segmentação das imagens de ultrassom, mas também desenvolver um aplicativo que calcule as medidas de AOL, EGL e EGG a partir de novas imagens de ultrassom, tornando a tecnologia acessível para produtores e profissionais do setor agropecuário.

Para alcançar o objetivo, o presente trabalho está organizado da seguinte forma. No Capítulo 1, são apresentados os conceitos fundamentais de redes neurais, desde a inspiração

Figura 4: Fluxo de treinamento e comparação dos modelos propostos para a obtenção das medidas de AOL, EGL e EGG.



Fonte: Elaboração do Autor.

biológica do neurônio artificial até as arquiteturas de redes neurais convolucionais. O Capítulo 2 introduz a teoria dos conjuntos fuzzy e os sistemas baseados em regras fuzzy, que fornecem a base teórica para a construção do ANFIS. O Capítulo 3 descreve detalhadamente a implementação dos modelos propostos, incluindo a arquitetura da U-Net, a integração do ANFIS e o processo de treinamento. O Capítulo 4 apresenta a organização do conjunto de dados, o treinamento e os resultados obtidos, com uma análise comparativa entre os modelos. Por fim, o Capítulo 5 conclui o trabalho, explicando o desenvolvimento do aplicativo, destacando as contribuições e sugerindo direções para pesquisas futuras.

Capítulo 1

Redes Neurais

Neste capítulo, serão apresentados os conceitos fundamentais relacionados às redes neurais, que são essenciais para o desenvolvimento do trabalho. A Seção 1.1 aborda a inspiração biológica por trás dos neurônios artificiais, enquanto a Seção 1.2 explora as arquiteturas de redes neurais, algoritmos de aprendizagem e desafios associados ao treinamento dessas redes. Esses conceitos formam a base para a construção da rede neural proposta na Seção 3.1. Todos os códigos e algoritmos presentes neste trabalho foram implementados utilizando a linguagem de programação Python, com o auxílio de bibliotecas como *NumPy*, *Matplotlib*, *openCV*, *Pillow* e *Pandas*.

1.1 Neurônio Biológico e Artificial

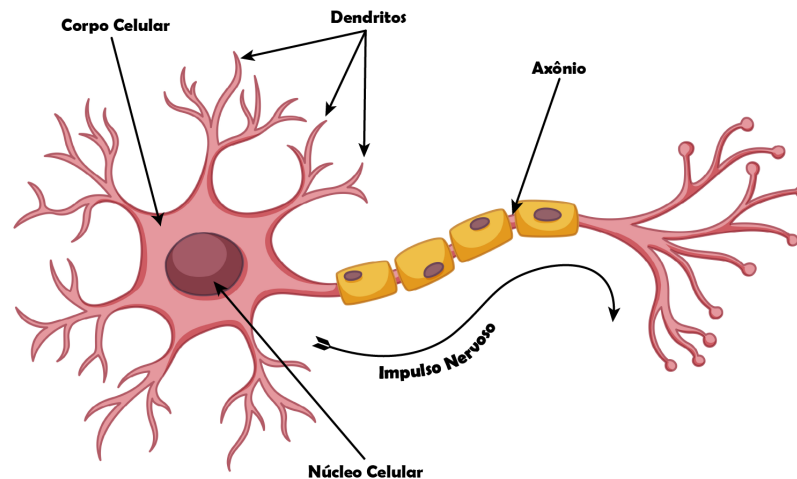
Na área de Inteligência Artificial, comumente utiliza-se o termo *neurônio artificial* para se referir a uma unidade de processamento em um modelo de rede neural. O neurônio artificial é inspirado no funcionamento do neurônio biológico, que é a unidade básica do sistema nervoso. Assim como os neurônios biológicos, os neurônios artificiais recebem entradas, processam essas entradas e produzem uma saída. A principal função de um neurônio artificial é realizar uma combinação linear das entradas, seguida por uma função de ativação não linear, que determina a saída do neurônio. A semelhança entre esses dois objetos (Neurônio artificial/Neurônio Biológico) só está na inspiração, sendo bem diferentes em seu funcionamento, com isso, neste trabalho utiliza-se o termo *unidade* (O'SHEA; NASH, 2015) para se referir ao neurônio artificial. Os conceitos apresentados a seguir são fundamentais para o entendimento do funcionamento das redes neurais, que serão abordados na seção 1.2.

1.1.1 Neurônio Biológico

Para entender o funcionamento da unidade matemática é necessário entender o funcionamento do neurônio biológico. Neurônios são “células excitáveis” que se comunicam ao transmitir impulsos elétricos. Células *excitáveis* são definidas como células capazes de produzir sinais elétricos amplos e rápidos denominados *potências de ação* (veja (STANFIELD, 2014)). A maioria dos neurônios contém três componentes principais: um corpo celular, o(s) dendrito(s) e um axônio. O corpo celular, chamado de *soma* contém o núcleo celular, local onde parte da informação

recebida de outras partes do corpo é manipulada. Os *dendritos* se ramificam a partir do corpo celular, são responsáveis por receber e integrar uma imensa quantidade de sinais provenientes de outros neurônios. Outra ramificação advinda do corpo celular é o *axônio*. Diferente dos dendritos, cuja função é *receber* informação, um axônio *envia* informações. Grande parte dos neurônios tem apenas um axônio, mas os axônios conseguem se ramificar, enviando, assim, sinais a mais de um destino. Esses três componentes, de forma resumida, formam o neurônio biológico, como mostrado na Figura 1.1.

Figura 1.1: Estrutura de um neurônio típico.



Fonte: Adaptado de (STANFIELD, 2014).

Outro fator importante em um neurônio biológico são os potenciais de ação, gerados nos dendritos, são avaliados e ponderados por meio da *força sináptica* (BEAR; CONNORS; PARADISO, 2017), que nos diz o quão importante a informação. Após esse processo, são enviados para o núcleo celular. Entre o corpo celular e o axônio será decidido se o potencial é excitatório ou inibitório. A decisão é feita caso o potencial de ação supere ou não um *limiar*. Caso seja excitatório, a informação é passada a diante, caso seja inibitório, a informação é suprimida.

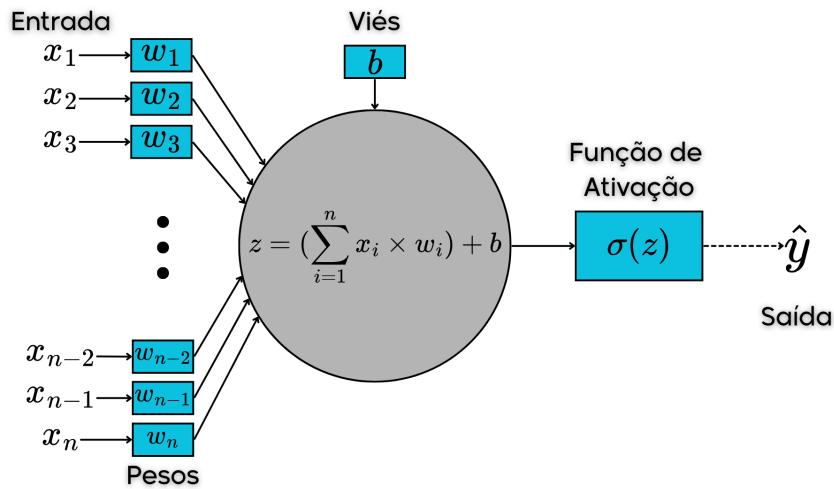
1.1.2 Unidade

A partir das informações obtidas com o estudo do neurônio biológico, pesquisadores tentaram simular seu funcionamento em computador. A implementação mais bem aceita de simulação foi proposta por dois pesquisadores, o neurobiologista Warren McCulloch e o matematico Walter Pitts em 1943 em (MCCULLOCH; PITTS, 1943) que utilizaram-se da lógica proposicional para descrever o funcionamento do sistema nervoso central, Figura 1.2. McCulloch acreditava que o neurônio biológico tinha uma natureza binária, ou seja, um neurônio influencia outro ou não, entretanto, posteriormente essa crença mostrou-se errônea com os avanços de estudos na área.

As variáveis $x_1, x_2, x_3, \dots, x_n$ são denominadas *dados de entrada*, em analogia às sinapses de um neurônio biológico. Os valores $w_1, w_2, w_3, \dots, w_n$ são os *pesos* associados a cada dado

de entrada, os pesos nos dizem o quão importante é a entrada para a *soma*, parte central na Figura 1.2. O viés tem a mesma funcionalidade do limiar no neurônio biológico, determinando

Figura 1.2: Modelo de uma unidade de McCulloch e Pitts.



Fonte: Elaboração do Autor.

se o neurônio será ativado ou não. Para resumir a notação, defini-se o viés de b (do inglês, *bias*). Na soma, a variável z é obtida fazendo a combinação linear das variáveis:

$$x_1 \times w_1 + x_2 \times w_2 + x_3 \times w_3 + \cdots + x_n \times w_n + b = z = \sum_{i=1}^n x_i \times w_i + b.$$

Em seguida, o valor z é aplicado a uma *função de ativação* $\sigma : \mathbb{R} \rightarrow \mathbb{R}$, resultando na saída $\hat{y}$, ou seja:

$$\hat{y} = \sigma \left(\sum_{i=1}^n w_i \times x_i + b \right). \quad (1.1)$$

1.1.3 Tipos de Função de Ativação

A função de ativação, representada por σ , define a saída de uma unidade, pode-se definir diferentes tipos de funções de ativação, como a função de limiar, função linear por partes e a função sigmoidal.

- **Função de Limiar:** Para este tipo de função de ativação, descrito na equação (1.2), a unidade é ativada (ou seja, produz uma saída de 1) se a soma ponderada das entradas mais o viés for maior ou igual a zero. Caso contrário, a saída é 0.

$$\sigma(z) = \begin{cases} 1, & \text{se } z \geq 0 \\ 0, & \text{se } z < 0 \end{cases}. \quad (1.2)$$

- **Função Sigmoidal:** A função sigmoidal, cujo o gráfico tem a forma de um “S”, é de longe a forma mais comum de função de ativação utilizada na construção de unidades. Esta é

definida como uma função estritamente crescente que exibe um balanceamento adequado entre comportamento linear e não linear. Um exemplo de função sigmoidal é a *sigmoid logística*¹, definida por:

$$\sigma(z) = \frac{1}{1 + \exp(-\alpha z)}, \quad (1.3)$$

em que α é um parâmetro que controla a inclinação da curva. A função sigmoid logística tem um conjunto de propriedades que se mostrarão muito úteis nos cálculos relacionados à aprendizagem dos pesos:

1. Não linear;
 2. Contínua e diferenciável em todo o domínio de $\mathbb{R}$;
 3. A derivada tem forma simples e é expressa pela própria função: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$;
 4. É estritamente monótona, ou seja, $z_1 \leq z_2 \iff \sigma(z_1) \leq \sigma(z_2)$.
- Função *Rectified Linear Unit* (ReLU): A função ReLU (do inglês, Unidade Linear Retificada) é definida como $\sigma(z) = \max(0, z)$. Esta é amplamente utilizada em redes neurais profundas (estudada na Seção 1.2) devido à sua simplicidade computacional. A função ReLU ativa a unidade apenas quando a soma ponderada das entradas mais o viés é positiva, caso contrário, a saída é zero, ou seja:

$$\sigma(z) = \begin{cases} z, & \text{se } z > 0 \\ 0, & \text{se } z \leq 0 \end{cases}. \quad (1.4)$$

Em geral, o objetivo da função de ativação é limitar a saída de uma unidade para um intervalo de valores, na Figura 1.3(a) é mostrado o gráfico da função sigmoid logística, enquanto na Figura 1.3(b) é mostrado o gráfico da função *Rectified Linear Unit*.

A principal desvantagem na unidade de McCulloch e Pitts é a ausência de um algoritmo de aprendizagem, ou seja, uma maneira de ajustar os pesos e o viés da unidade para que este seja capaz de resolver um problema específico. Essa limitação foi superada em 1958 por Frank Rosenblatt, que propôs o *Perceptron* (ROSENBLATT, 1958), um modelo de unidade com um algoritmo de aprendizagem capaz de ajustar os pesos e o viés da unidade.

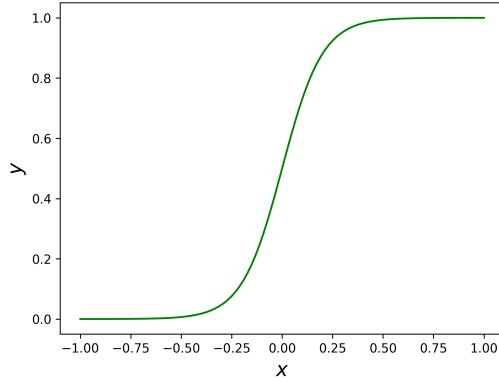
1.1.4 Perceptron e Algoritmo de Aprendizagem

O Perceptron é um modelo de unidade que utiliza um algoritmo de aprendizagem supervisionada para ajustar seus pesos e viés com base em um conjunto de dados “rotulados”². O objetivo

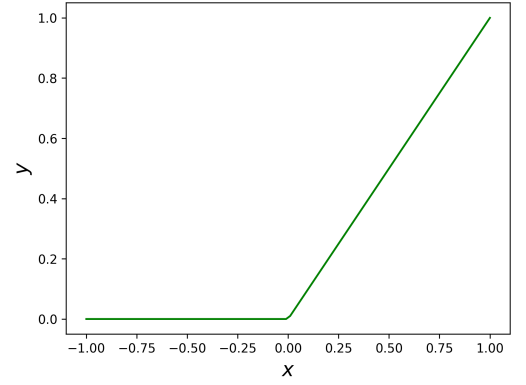
¹A função sigmoid logística foi amplamente usada no início das pesquisas em redes neurais pois a resposta do neurônio biológico é aproximadamente similar. Isso ficou claro em estudos feitos nos neurônios dos olhos de animais (LETTVIN *et al.*, 1959), neste caso, sapos. O principal motivo para o seu desuso foi a dificuldade de treinamento em redes neurais profundas devido ao problema do *vanish gradient*, subseção 1.2.8.

²Dados rotulados são conjuntos de dados de entrada associados a saídas desejadas, ou seja, para cada conjunto de entradas, há uma saída correta conhecida.

Figura 1.3: Gráficos das Funções de Ativação.



(a) Função Sigmoid.



(b) Função ReLU.

Fonte: Elaboração do Autor.

do Perceptron é encontrar uma função que mapeie corretamente as entradas para as saídas desejadas, minimizando o erro entre as previsões do modelo e os valores reais.

Para esclarecer o funcionamento do Perceptron e do algoritmo de aprendizagem, é proposto o Exemplo 1.1.

Exemplo 1.1. *Um investidor busca decidir qual ação adquirir com base nas variáveis: o valor é caro?, é confiável? e é volátil?. Para isso, dispõe das avaliações de quatro colegas que também analisam essas mesmas características antes de realizar seus investimentos.*

Tabela 1.1: Banco de dados com as avaliações dos colegas e a decisão final de compra.

	valor	confiança	volatilidade	y
x_1	0	1	0	1
x_2	1	0	1	0
x_3	1	1	1	0
x_4	0	0	0	1

Na Tabela 1.1 são apresentadas as avaliações dos colegas e a decisão final de compra em que 0 representa “não”, 1 representa “sim”, $x_1, \dots, x_4$ representam as respostas de cada colegas e y a decisão final de cada um. Com isso, o Perceptron terá três variáveis de entrada e como saída, a decisão sobre a compra da ação. Os pesos iniciais são definidos aleatoriamente como: $w_1 = w_2 = w_3 = 0$ e o viés como $b = 0$. A função de ativação utilizada é a função ReLU (1.4). Testando o primeiro dado de entrada $x_1 = (0, 1, 0)$ e $y = 1$, obtém-se:

$$z = x_1 \cdot w_1 + b = 0 \Rightarrow \hat{y} = \sigma(0) = 0,$$

o símbolo $\cdot$ representa o produto escalar entre os vetores. Como $y = 1 \neq \hat{y} = 0$, em que $\hat{y}$ é a resposta da rede definido como na equação (1.1). O Perceptron comete um erro e é necessário aplicar o algoritmo de aprendizagem (a unidade de McCulloch e Pitts pararia aqui). Para isso, define-se uma função de custo, responsável por quantificar o erro cometido durante

o treinamento. Neste exemplo, utiliza-se o Erro Quadrático Médio, dado por:

$$C(w_i, b_i) = \frac{\sum_{i=1}^k (y_i - \hat{y}_i)^2}{n}, \quad (1.5)$$

em que y_i é a resposta do dado de entrada x_i e n representa a quantidade de dados presentes no conjunto de treinamento. $C(w_i, b_i)$ não depende de w_i e b_i explicitamente, mas sim por meio de $\hat{y}_i$, ou seja, $C(w_i, b_i) = C(\hat{y}_i(w_i, b_i))$, já que $\hat{y}_i = \sigma(z_i)$ e $z_i = x_i \cdot w_i + b_i$. No caso do Exemplo 1.1, x_i e w_i são vetores.

O processo de treinamento ajusta iterativamente os pesos e o viés por meio do método da Descida do Gradiente (SG - *Gradient Descent*), cujo objetivo é minimizar a função de custo. Esse método utiliza as derivadas parciais da função de custo em relação aos pesos e ao viés, o que indica a direção de maior crescimento da função. Para reduzir o erro, atualizam-se os parâmetros na direção oposta ao gradiente. As derivadas são:

$$\frac{\partial C(w_i, b_i)}{\partial w_i} = -2 \cdot \frac{\sum_{i=1}^k (y_i - \hat{y}_i) \cdot \sigma'(z_i) \cdot x_i}{n}; \quad (1.6)$$

$$; \frac{\partial C(w_i, b_i)}{\partial b_i} = -2 \cdot \frac{\sum_{i=1}^k (y_i - \hat{y}_i) \cdot \sigma'(z_i)}{n}. \quad (1.7)$$

No o Exemplo 1.1 assume-se $\sigma'(z) = 1$ para simplificação. Como a derivada aponta para a região de maior valor da função, multiplica-se por -1 para obter a direção da região de minimização. Com isso, é obtido o Algoritmo 1.

Entrada: Taxa de aprendizado η , função de custo $C(w, b)$, pesos iniciais

$w_1, w_2, \dots, w_n$ e viés b

Saída: Pesos e viés ajustados

Inicializar os pesos w_i e o viés b com valores pequenos aleatórios;

Repita

 Calcular a saída do modelo;

$\hat{y}_i \leftarrow \sigma(\sum_{i=1}^n w_i \cdot x_i + b)$;

 Calcular o erro;

$\Delta_w \leftarrow \frac{\sum_{i=1}^n (y_i - \hat{y}_i) \cdot \sigma'(z_i) \cdot x_i}{n}$;

$\Delta_b \leftarrow \frac{\sum_{i=1}^n (y_i - \hat{y}_i) \cdot \sigma'(z_i)}{n}$;

 Atualizar os pesos e o viés;

$w_i \leftarrow w_i + \eta \Delta_w$, para todo $i = 1, 2, \dots, n$;

$b \leftarrow b + \eta \Delta_b$;

até convergência ou número máximo de iterações;

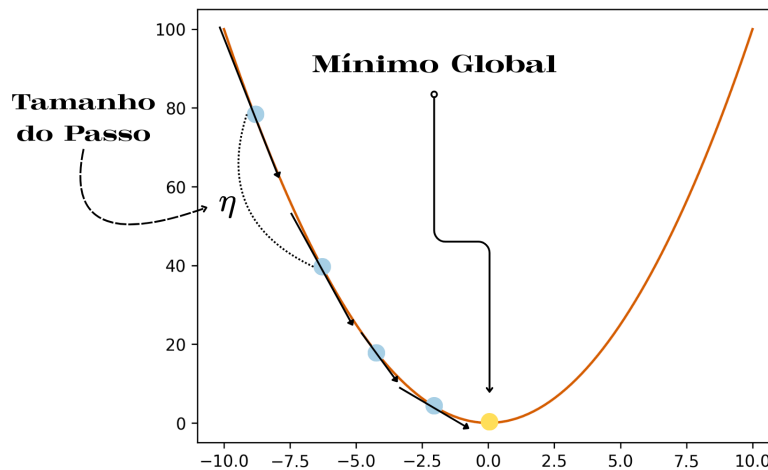
Retorne w_i, b ;

Algoritmo 1: Método da Descida do Gradiente.

A taxa de aprendizagem (η) presente no Algoritmo 1 pode ser interpretada como o tamanho

do passo dado em direção ao mínimo da função de custo, como mostra a Figura 1.4. Quando o valor dessa taxa é muito pequeno, o processo de convergência torna-se mais lento, exigindo um número maior de iterações para atingir o mínimo. Por outro lado, se o passo for muito grande, o algoritmo pode ultrapassar o ponto de mínimo, dificultando ou até impedindo a convergência adequada, para o Exemplo 1.1 é utilizada uma taxa de aprendizado de 0,1.

Figura 1.4: Representação da taxa de aprendizagem.



Fonte: Elaboração do Autor.

Aplicando o Algoritmo 1 no Exemplo 1.1, os seguintes ajustes são feitos nos pesos e no viés após uma passada pelos 4 dados presentes no banco de dados:

$$w = (-0, 1, 0, 1, -0, 1), \quad b = 0, 1.$$

Verificando as previsões da rede para os parâmetros atualizados, é obtido:

$$x_1: (0, 1, 0) \Rightarrow z = 0 \cdot (-0, 1) + 1 \cdot 0, 1 + 0 \cdot (-0, 1) + 0, 1 = 0, 2 \Rightarrow \hat{y} = 1;$$

$$x_2: (1, 0, 1) \Rightarrow z = -0, 1 + 0 + -0, 1 + 0, 1 = -0, 1 \Rightarrow \hat{y} = 0;$$

$$x_3: (1, 1, 1) \Rightarrow z = -0, 1 + 0, 1 - 0, 1 + 0, 1 = 0 \Rightarrow \hat{y} = 0;$$

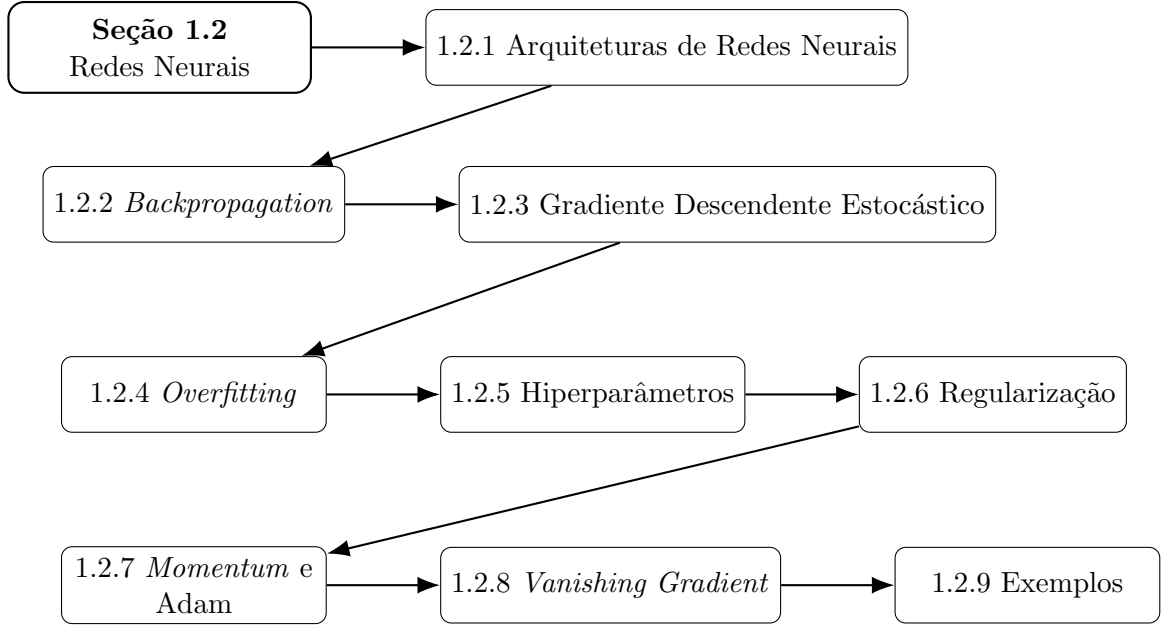
$$x_4: (0, 0, 0) \Rightarrow z = 0 + 0 + 0 + 0, 1 = 0, 1 \Rightarrow \hat{y} = 1.$$

Basta observar que $\hat{y} = \sigma(z)$ e $\sigma(z)$ é a função de ativação utilizada. Assim, após uma única época com $\eta = 0, 1$ os pesos e o viés encontrados classificam corretamente todos os exemplos do conjunto de treinamento.

Comumente o banco de dados é dividido em dois conjuntos: o conjunto de treinamento e o conjunto de teste. O conjunto de treinamento é utilizado para ajustar os pesos e o viés do Perceptron, enquanto o conjunto de teste é utilizado para avaliar o desempenho do modelo em dados não vistos durante o treinamento. Essa divisão é essencial para garantir que o modelo aprenda a generalizar bem para novos dados, evitando o problema do *overfitting* (na Seção 1.2.4), que ocorre quando o modelo se ajusta excessivamente aos dados de treinamento, perdendo a capacidade de generalização.

1.2 Redes Neurais

Figura 1.5: Organização da Seção 1.2.



Fonte: Elaboração do Autor.

Nesta seção serão apresentados os conceitos relacionados às redes neurais, sendo muito importantes para o andamento do trabalho, todos os conceitos apresentados nesta seção serão utilizados na construção da rede neural proposta na Seção 3.1.

As redes neurais são um “encadeamento” de múltiplas unidades, onde a saída de uma unidade é a entrada de outra. Redes neurais podem ser organizadas em camadas: uma *camada de entrada*, uma ou mais *camadas ocultas* e uma *camada de saída*. Cada camada é composta por várias unidades e cada unidade em uma camada está conectada a todas as unidades na camada seguinte. A Figura 1.6(a) ilustra a estrutura básica de uma unidade vista na Seção 1.1.2. Já a Figura 1.6(b) mostra uma rede neural com duas camadas ocultas, em que a camada de entrada tem três unidades, a primeira camada oculta tem cinco unidades, a segunda camada oculta tem quatro unidades e a camada de saída tem duas unidades. Matematicamente, é possível representar uma rede neural com duas camadas ocultas e uma saída da seguinte maneira:

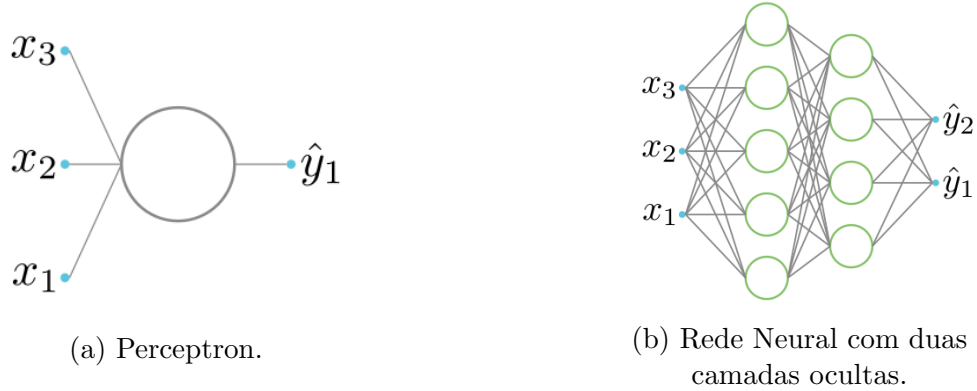
$$\hat{y} = \sigma_s \left[\sum_{i=1}^4 \sigma_i^2 \left(\sum_{j=1}^5 \sigma_j^1 \left(\sum_{k=1}^3 w_{jk}^1 x_k + b_j^1 \right) w_{ij}^2 + b_i^2 \right) w_i^s + b^s \right], \quad (1.8)$$

em que σ_j^1 é a função de ativação da j -ésima unidade da primeira camada oculta, σ_i^2 é a função de ativação da i -ésima unidade da segunda camada oculta e σ_s é a função de ativação da s -ésima unidade da camada de saída. Os pesos e os vieses são representados por w_{jk}^1 , b_j^1 , w_{ij}^2 , b_i^2 , w_i^s e b^s . Generalizando a notação, para uma rede neural com L camadas ocultas, a saída da rede

pode ser expressa como:

$$\hat{y} = \sigma_s \left[\sum_{i_L=1}^{n_L} \sigma_{i_L}^L \left(\sum_{i_{L-1}=1}^{n_{L-1}} \sigma_{i_{L-1}}^{L-1} \left(\cdots \sum_{k=1}^{n_1} \sigma_k^1 \left(\sum_{m=1}^{n_0} w_{km}^1 x_m + b_k^1 \right) w_{jk}^2 + b_j^2 \right) w_{ij}^L + b_i^L \right) w_i^s + b^s \right] \quad (1.9)$$

Figura 1.6: Estrutura visual básica de uma unidade e uma rede neural com duas camadas ocultas.



Fonte: Elaboração do Autor.

Outra forma de representar uma rede neural é utilizando a notação matricial, em que as entradas, os pesos e os vieses são representados por matrizes e vetores. Para uma rede neural com n entradas, uma camada oculta com m neurônios e uma saída, a saída da rede pode ser expressa como:

$$\underbrace{\begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix}}_{m \times n} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}}_{n \times 1} + \underbrace{\begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}}_{m \times 1}$$

Essa forma representa a saída de uma camada caso a rede tenha apenas uma camada oculta, para redes neurais com mais camadas ocultas, a notação matricial torna-se mais complexa, mas o princípio de multiplicação de matrizes e adição de vetores permanece o mesmo.

1.2.1 Arquiteturas de Redes Neurais

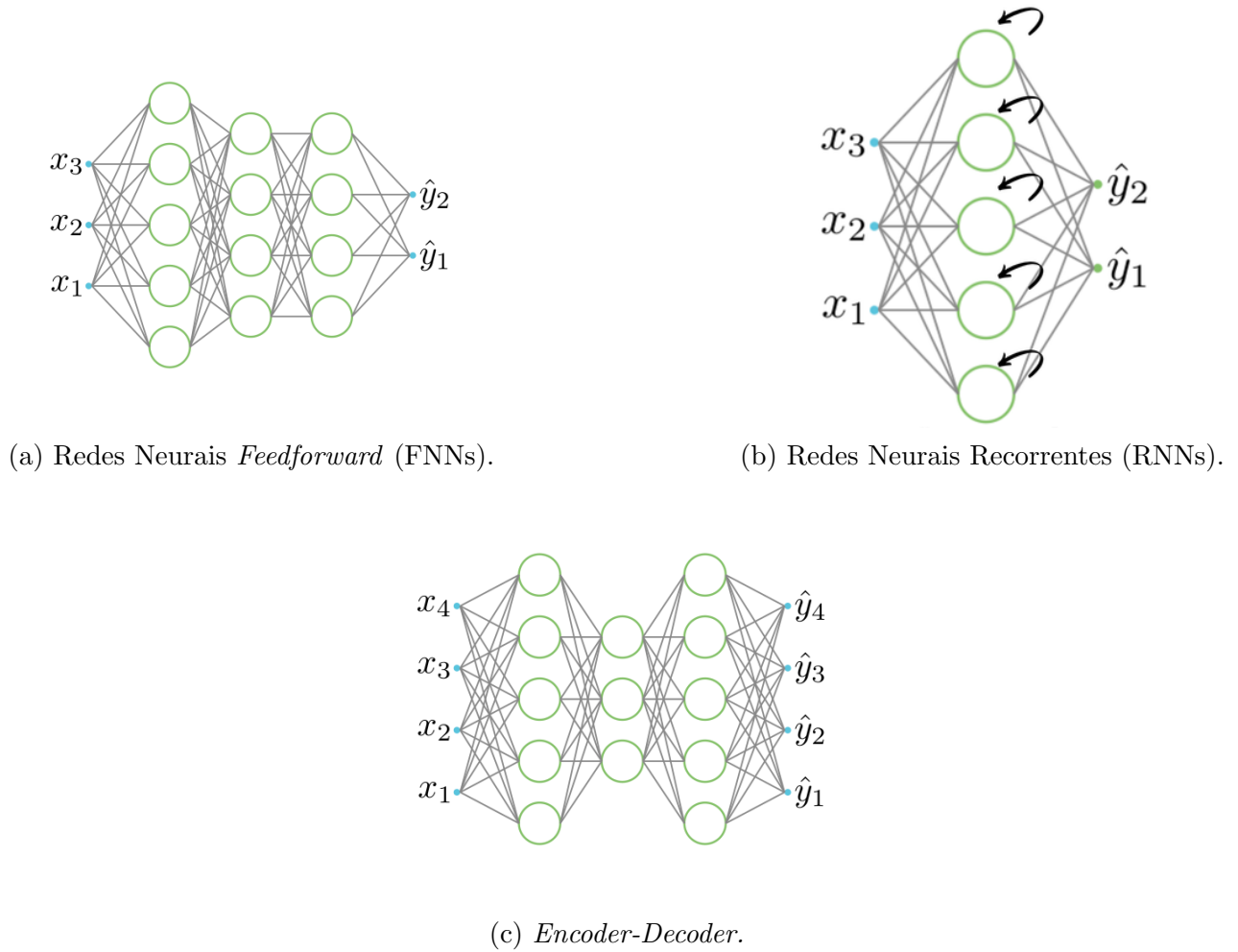
Existem diversas arquiteturas de redes neurais, cada uma projetada para lidar com diferentes tipos de problemas e dados. Algumas das arquiteturas mais comuns incluem ((BOOK, 2025a)):

- **Redes Neurais Feedforward (FNNs):** São as redes neurais mais simples, onde a informação flui em uma única direção, da camada de entrada para a camada de saída, passando pelas camadas ocultas. FNNs são amplamente utilizadas para tarefas de classificação e regressão.

- Redes Neurais Convolucionais (RNCs): São projetadas para processar dados com uma estrutura de grade, como imagens. Estas utilizam operações de convolução para extrair características espaciais dos dados de entrada. RNCs são amplamente utilizadas em tarefas de visão computacional, como reconhecimento de objetos e detecção de faces.
- Redes Neurais Recorrentes (RNNs): São projetadas para lidar com dados sequenciais, onde a saída em um determinado momento depende das entradas anteriores. RNNs possuem conexões recorrentes que permitem que informações sejam mantidas ao longo do tempo. Estas são amplamente utilizadas em tarefas como processamento de linguagem natural e reconhecimento de fala.
- Redes Neurais Generativas Adversariais (GANs): Consistem em duas redes neurais que competem entre si: um gerador e um discriminador. O gerador cria dados falsos, enquanto o discriminador tenta distinguir entre dados reais e falsos. GANs são amplamente utilizadas para geração de imagens realistas e outras tarefas criativas.
- *Encoder-Decoder*: São arquiteturas que consistem em duas partes principais: o *encoder*, que comprime os dados de entrada em uma representação latente, e o *decoder*, que reconstrói os dados a partir dessa representação. Essas redes são amplamente utilizadas em tarefas como tradução automática e geração de texto.

É possível utilizar duas arquiteturas em conjunto, como abordado na Seção 3.1.

Figura 1.7: Arquiteturas de Redes Neurais.



Fonte: Elaboração do Autor.

As Figuras 1.7(a) e 1.7(c) ilustram a arquitetura de uma Rede Neural *Feedforward*, com o funcionamento semelhante ao desenvolvido na equação (1.9). A Figura 1.7(b) mostra a arquitetura de uma Rede Neural Recorrente, estas redes possuem conexões recorrentes, ou seja, a saída de uma unidade é realimentada como entrada para a mesma unidade ou para unidades anteriores, permitindo que informações sejam mantidas ao longo do tempo, por não estar no escopo deste trabalho, não serão abordados os detalhes matemáticos de funcionamento das RNNs.

1.2.2 *Backpropagation*

Assim como no perceptron, as redes neurais utilizam algoritmos de aprendizagem para ajustar seus pesos e vieses com o objetivo de minimizar uma função de custo. O algoritmo mais comum utilizado para esse propósito é o *backpropagation* (retropropagação) (HECHT-NIELSEN, 1992) e (WERBOS, 1990), que é uma extensão do método da descida do gradiente. O *backpropagation* é utilizado para calcular os gradientes da função de custo em relação aos pesos e vieses de todas as camadas da rede neural, permitindo a atualização eficiente desses parâmetros durante

o treinamento. O algoritmo funciona propagando o erro da saída da rede de volta para as camadas anteriores, ajustando os pesos com base na contribuição de cada unidade para o erro total.

Esse processo é repetido iterativamente até que a função de custo seja minimizada, resultando em uma rede neural treinada capaz de realizar previsões precisas em novos dados. Essa propagação do erro é feita utilizando a regra da cadeia do cálculo diferencial, que permite calcular as derivadas parciais de funções compostas. O algoritmo de *backpropagation* pode ser resumido nos seguintes passos, apresentados no Algoritmo 2.

Entrada: Dados de treinamento (x, y) , taxa de aprendizado η , pesos W e vieses b

Saída: Pesos W e vieses b atualizados

Repita

Passo para frente: propagar x pela rede (camada a camada) e obter $\hat{y}$;

Cálculo do Erro: calcular o erro comparando $\hat{y}$ com y usando uma função de custo ;

Passo para trás: retropropagar o erro e calcular os gradientes $\nabla_W C$ e $\nabla_b C$ (regra da cadeia);

Atualização: atualizar pesos e vieses (descida do gradiente);

$W \leftarrow W - \eta \nabla_W C$;

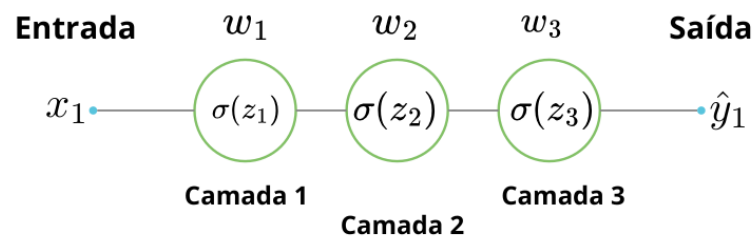
$b \leftarrow b - \eta \nabla_b C$;

até critério de parada (*ex.: n^o de épocas ou convergência*);

Algoritmo 2: Etapas do treinamento via *backpropagation*.

Na Figura 1.8 ilustra uma rede neural simples e a equação (1.10) mostra o cálculo do gradiente para ajustar o peso da primeira camada (w_1) utilizando a regra da cadeia,

Figura 1.8: Ilustração do processo de *backpropagation* em uma rede neural.



Fonte: Elaboração do Autor.

$$\nabla_w C = \frac{\partial C(w, b)}{\partial w_1} = \frac{C(w, b)}{\sigma(z_3)} \cdot \frac{\sigma(z_3)}{z_3} \cdot \frac{z_3}{\sigma(z_2)} \cdot \frac{\sigma(z_2)}{z_2} \cdot \frac{z_2}{\sigma(z_1)} \cdot \frac{\sigma(z_1)}{z_1} \cdot \frac{z_1}{w_1}, \quad (1.10)$$

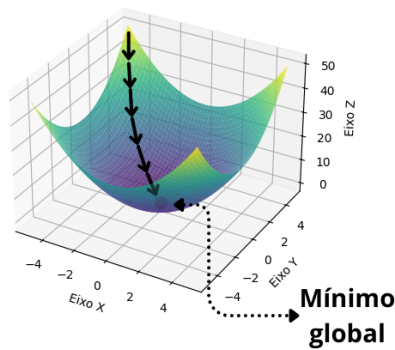
em que $z_1 = w_1 \cdot x + b_1$, $z_2 = \sigma(z_1) \cdot w_2 + b_3$ e $z_3 = \sigma(z_2) \cdot w_3 + b_3$ são as saídas das camadas ocultas e $C(w, b)$ é a função de custo do erro quadrático médio, ou seja, $C(w, b) = \frac{(\hat{y} - \sigma(z_3))^2}{n}$, n sendo o número de amostras no conjunto de dados.

1.2.3 Gradiente Descendente Estocástico

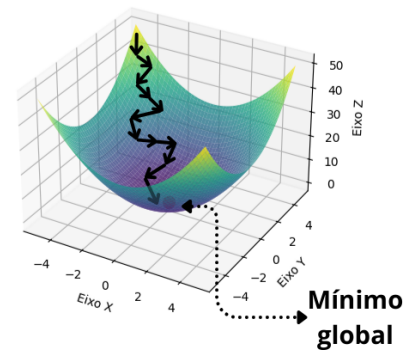
O Gradiente Descendente Estocástico (SGD - Stochastic Gradient Descent) (RUDER, 2016) é uma variação do método de descida do gradiente tradicional, amplamente utilizado no treinamento de redes neurais e outros modelos de aprendizado de máquina. A principal diferença entre o SGD e o método de descida do gradiente padrão é a forma como os gradientes são calculados e atualizados durante o treinamento. No método de descida do gradiente tradicional, os gradientes são calculados usando todo o conjunto de dados de treinamento, ou seja, os pesos são atualizados uma vez em cada época de treinamento, o que pode ser computacionalmente caro e demorado, especialmente para grandes conjuntos de dados. Em contraste, o SGD calcula os gradientes usando apenas uma amostra aleatória (ou um pequeno lote) do conjunto de dados em cada iteração, ou seja, caso o conjunto de dados tenha um tamanho de 1000 e o tamanho do lote seja 10, o SGD atualizará os pesos 100 vezes por época. Isso torna o processo de atualização dos pesos mais rápido e eficiente, permitindo que o modelo aprenda mais rapidamente. Apesar da Descida do Gradiente padrão fornecer uma direção mais precisa para o mínimo global da função de custo, o SGD é capaz de chegar próximo do mínimo de forma mais rápida e eficiente.

A Figura 1.9(a) ilustra como funciona o Gradiente Descendente Estocástico, em que cada seta preta indica uma atualização dos pesos do modelo. A Figura 1.9(b) ilustra o caminho percorrido pelo método de descida do gradiente tradicional, enquanto a Figura 1.9(c) ilustra o caminho percorrido pelo SGD.

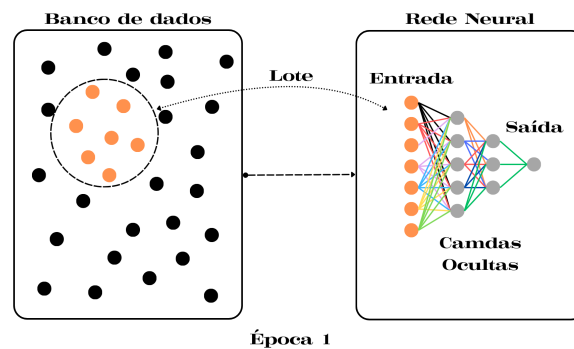
Figura 1.9: Comparação entre o método de descida do gradiente tradicional e o Gradiente Descendente Estocástico.



(a) Descida do Gradiente Tradicional.



(b) Gradiente Descendente Estocástico.



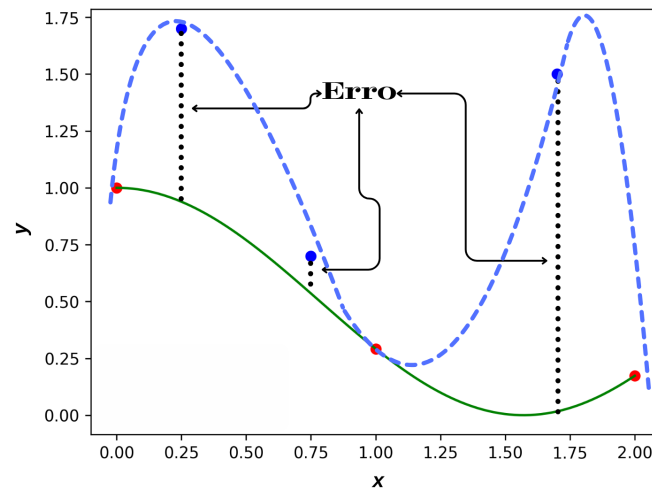
(c) Separando em lotes o banco de dados para o SGD.

Fonte: Elaboração do Autor.

1.2.4 *Overfitting*

O *Overfitting* (BOOK, 2025b) (do inglês “Sobreajuste” ou “Ajuste Excessivo”) é um problema comum em aprendizado de máquina, onde um modelo aprende os detalhes e o ruído dos dados de treinamento a tal ponto que este se torna incapaz de generalizar para novos dados. Isso significa que o modelo tem um desempenho excelente nos dados de treinamento, mas falha ao fazer previsões precisas em dados não vistos anteriormente.

O *overfitting* ocorre quando o modelo é muito complexo em relação à quantidade e qualidade dos dados disponíveis para treinamento. Modelos com muitos parâmetros, como redes neurais profundas, são particularmente suscetíveis ao *overfitting*, especialmente quando o conjunto de dados de treinamento é pequeno ou contém ruído significativo. Na Figura 1.10 é apresentada uma ilustração do *overfitting*, em que os pontos vermelhos são os dados de treinamento e os pontos azuis são os dados de teste, a curva verde representa um modelo que se ajusta perfeitamente aos dados de treinamento, mas falha em generalizar para novos dados, enquanto a curva azul seria um modelo ideal mais geral.

Figura 1.10: Ilustração do *Overfitting*.

Fonte: Elaboração do Autor.

1.2.5 Hiperparâmetros

Um algoritmo de aprendizado pode ser visto como uma função que recebe dados de treinamento como entrada e produz como saída uma função ou modelo (isto é um conjunto de funções). No entanto, na prática muitos algoritmos de aprendizado possuem parâmetros adicionais que não são aprendidos diretamente a partir dos dados de treinamento, mas que influenciam o processo de aprendizado. Esses parâmetros são chamados de hiperparâmetros e sua escolha adequada é crucial para o desempenho do modelo (BENGIO, 2012). Alguns exemplos comuns de hiperparâmetros incluem:

- **Taxa de Aprendizado:** Controla o tamanho dos passos dados durante a atualização dos pesos do modelo. Uma taxa de aprendizado muito alta pode levar a oscilações e impedir a convergência, enquanto uma taxa muito baixa pode resultar em um processo de treinamento lento. Valores típicos para a taxa de aprendizado variam entre 0,01 e 0,1, dependendo do problema e do algoritmo utilizado.
- **Número de Épocas:** Define quantas vezes o algoritmo de aprendizado passará por todo o conjunto de dados de treinamento. Um número muito alto pode levar ao *overfitting*, enquanto um número muito baixo pode resultar em *underfitting* (subajuste). Uma técnica comum para determinar o número ideal de épocas é o uso do *Early Stopping* (do inglês Parada Antecipada), que monitora o desempenho do modelo em um conjunto de validação e interrompe o treinamento quando o desempenho começa a piorar.
- **Tamanho do Lote:** Refere-se ao número de amostras de dados usadas para calcular o gradiente em cada iteração do treinamento. Tamanhos de lote menores podem levar a atualizações mais ruidosas, enquanto tamanhos maiores podem resultar em um treinamento mais estável, embora mais lento. Os valores escolhidos geralmente variam entre 1 e 32, sendo que valores acima de 10 tiram proveito da Unidade Central de Processamento

(do inglês *Central Processing Unit* - CPU) e da Unidade de Processamento Gráfico (do inglês *Graphics Processing Unit* - GPU), que oferece ganho de velocidade dos produtos matriz-matriz em relação aos produtos matriz-vetor.

- **Arquitetura do Modelo:** Inclui o número de camadas, o número de unidades em cada camada e o tipo de funções de ativação utilizadas. Modelos mais complexos podem capturar padrões mais sofisticados, mas também são mais propensos ao *overfitting*.

1.2.6 Regularização

A regularização (BOOK, 2025c) é uma técnica utilizada para prevenir o *overfitting* em modelos de aprendizado de máquina, adicionando uma penalidade à função de custo que o modelo tenta minimizar durante o treinamento. Essa penalidade incentiva o modelo a manter os pesos dos parâmetros pequenos, o que ajuda a evitar que ele se ajuste excessivamente aos dados de treinamento. Existem várias técnicas de regularização, sendo as mais comuns:

- **Regularização L1 (Norma 1):** Adiciona uma penalidade proporcional à soma dos valores absolutos dos pesos dos parâmetros à função de custo. Isso pode levar a soluções esparsas, onde alguns pesos são reduzidos a zero, efetivamente removendo certas características do modelo. A equação da função de custo com regularização L1 é dada por:

$$C(w, b) = C_0(w, b) + \frac{\lambda}{n} \sum_i |w_i|, \quad (1.11)$$

em que $C_0(w, b)$ é a função de custo original, λ é o parâmetro de regularização que controla a força da penalidade, w_i são os pesos dos parâmetros do modelo e n é o número de amostras no conjunto de dados.

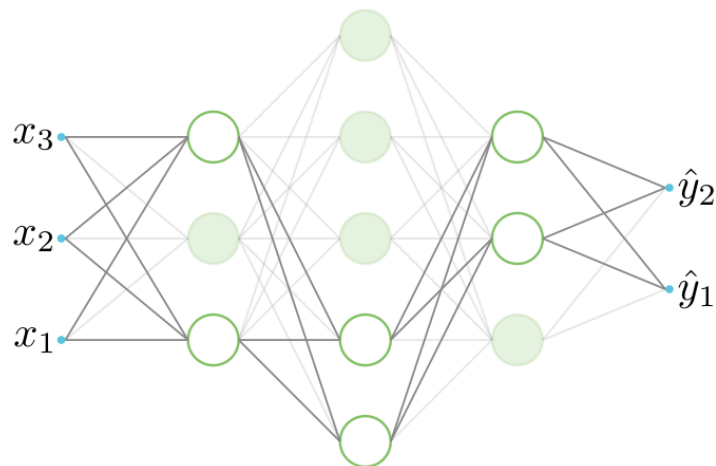
- **Regularização L2 (Norma 2):** Adiciona uma penalidade proporcional à soma dos quadrados dos pesos dos parâmetros à função de custo. Isso incentiva o modelo a distribuir os pesos de forma mais uniforme, evitando que alguns pesos se tornem muito grandes. A equação da função de custo com regularização L2 é dada por:

$$C(w, b) = C_0(w, b) + \frac{\lambda}{n} \sum_i w_i^2, \quad (1.12)$$

em que os termos são definidos da mesma forma que na regularização L1.

- **Dropout:** Durante o treinamento, essa técnica desativa aleatoriamente uma fração das unidades em cada camada da rede neural para cada época. Isso força a rede a aprender representações mais robustas e reduz a dependência de unidades específicas, ajudando a prevenir o *overfitting*. Em termos gerais, ao aplicar o *dropout* estamos treinando uma coleção de sub-redes dentro da rede original, o que pode promover a generalização do modelo.

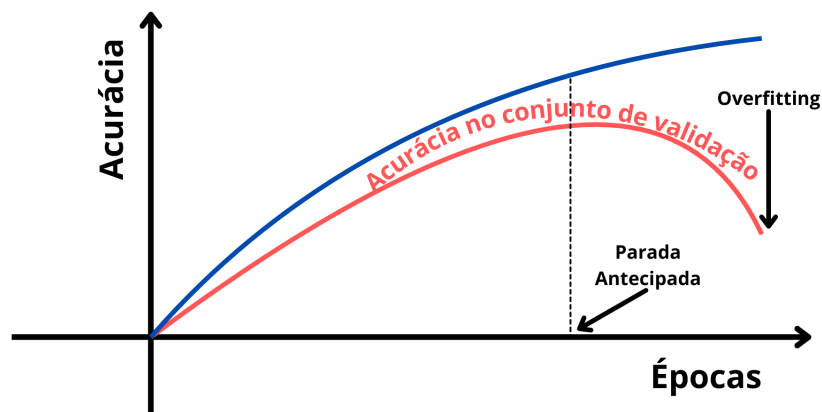
Figura 1.11: Ilustração do *Dropout* removendo 60% das unidades aleatoriamente durante uma época de treinamento.



Fonte: Elaboração do Autor.

- *Early Stopping* (Parada Antecipada): Monitora o desempenho do modelo em um conjunto de validação durante o treinamento e interrompe o processo quando o desempenho começa a piorar. Isso evita que o modelo continue a se ajustar aos dados de treinamento além do ponto ideal. Como ilustrado na Figura 1.12, o ponto de parada ideal é onde a acurácia na validação começa a diminuir, indicando o início do *overfitting*.

Figura 1.12: Ilustração do *Early Stopping*.



Fonte: Adaptado de (BOOK, 2025c).

1.2.7 *Momentum* e Adam

O *Momentum* (Momento) (SUTSKEVER, 2013) é uma técnica utilizada para acelerar o processo de treinamento de redes neurais, especialmente em situações onde a função de custo apresenta regiões íngremes ou planas. A ideia principal do *momentum* é acumular uma média

ponderada dos gradientes anteriores para suavizar as atualizações dos pesos, permitindo que o modelo mantenha uma direção consistente durante o treinamento. Isso ajuda a evitar oscilações e acelera a convergência para o mínimo da função de custo. Dada uma função de custo $C(w, b)$ a ser minimizada, o *momentum* clássico é definido por:

$$\begin{cases} v_{t+1} = \beta v_t - 1 + \epsilon \nabla C(w_t, b_t) \\ \theta_{t+1} = \theta_t + v_{t+1}, \end{cases} \quad (1.13)$$

em que $\epsilon > 0$ é a taxa de aprendizagem, $\beta \in [0, 1]$ é o coeficiente de *momentum*, v_t é a velocidade acumulada no tempo t , $\theta_t = (w_t, b_t)$ representa os parâmetros do modelo (pesos e vieses) no tempo t e $\nabla C(w_t, b_t)$ é o gradiente da função de custo em relação aos parâmetros do modelo. O coeficiente de *momentum* β controla a influência dos gradientes anteriores nas atualizações atuais. *Adaptive Moment Estimation - Adam* (RUDER, 2016) é um método que calcula taxas de aprendizagem (η) adaptativas para cada parâmetro do modelo, combinando as vantagens do *momentum* e do método de descida do gradiente. Atualmente, o Adam é um dos otimizadores mais populares para treinamento de redes neurais, devido à sua eficiência e capacidade de lidar com grandes conjuntos de dados e parâmetros.

1.2.8 *Vanishing Gradient*

O problema do *vanishing gradient* (gradiente que desaparece) ocorre durante o treinamento de redes neurais profundas, em que os gradientes calculados durante o processo de *backpropagation* tornam-se extremamente pequenos à medida que são propagados para as camadas mais próximas da entrada. Isso acontece devido à multiplicação repetida de pequenas derivadas associadas às funções de ativação e aos pesos das camadas. Como resultado, os pesos das camadas iniciais da rede recebem atualizações insignificantes, dificultando o aprendizado efetivo dessas camadas. Como neste trabalho serão utilizadas redes neurais profundas, é importante entender o impacto do problema do *vanishing gradient* e as técnicas para mitigá-lo.

Exemplo 1.2. Considere uma rede neural profunda com várias camadas ocultas, em que cada camada utiliza a função de ativação sigmoid logística, definida como:

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (1.14)$$

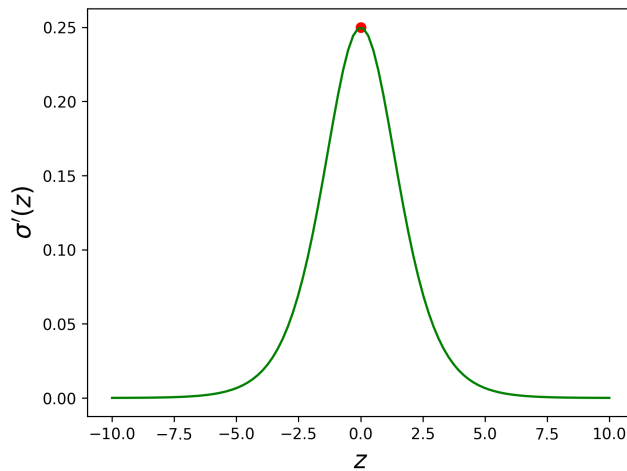
A derivada dessa função é dada por:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)). \quad (1.15)$$

Durante o processo de *backpropagation*, o gradiente da função de custo em relação aos pesos das camadas iniciais é calculado utilizando a regra da cadeia. Isso envolve a multiplicação das derivadas das funções de ativação de cada camada, como mostra a equação (1.10). Como a derivada da função sigmoid é sempre menor que 0,25 (o valor máximo ocorre em $z = 0$), a multiplicação repetida dessas pequenas derivadas resulta em gradientes que se tornam cada vez

menores à medida que são propagados para trás através das camadas da rede. Por exemplo, se uma rede possui 10 camadas ocultas, o gradiente pode ser reduzido a um valor extremamente pequeno, como 10^{-6} ou menor, tornando as atualizações dos pesos nas camadas iniciais praticamente insignificantes.

Figura 1.13: Gráfico da derivada da função sigmoid logística.



Fonte: Elaboração do Autor.

O problema do *vanishing gradient* é particularmente comum em redes neurais recorrentes (RNNs) e em redes *feedforward* profundas que utilizam funções de ativação como a sigmoid ou a tangente hiperbólica. Para contornar o problema do *vanishing gradient*, várias técnicas podem ser empregadas, incluindo:

- **Funções de Ativação Alternativas:** Funções de ativação como ReLU e suas variantes (Leaky ReLU, Parametric ReLU) têm derivadas constantes para valores positivos, o que ajuda a manter os gradientes maiores durante o *backpropagation*.
- **Batch Normalization:** Normaliza as ativações de cada camada durante o treinamento, o que pode ajudar a estabilizar os gradientes e acelerar o processo de aprendizado.

1.2.9 Função de Perda *Binary Cross-Entropy*

O *Vanishing Gradient* acontece principalmente em redes neurais profundas que utilizam funções de ativação como a *sigmoid* ou a tangente hiperbólica, uma das formas de contornar esse problema é utilizando a função de perda *Binary Cross-Entropy* (BCE) (ELHARROUSS *et al.*, 2025), que é comumente utilizada em tarefas de classificação binária. A função de perda BCE é definida como:

$$\text{BCE}(y, \hat{y}) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)], \quad (1.16)$$

em que y_i é o valor de saída verdadeiro da i -ésima amostra (0 ou 1), $\hat{y}_i$ é a previsão da rede para a i -ésima amostra (probabilidade de pertencer à classe 1) e n é o número total de amostras no

conjunto de dados. A função de perda BCE é especialmente eficaz para lidar com o problema do *vanishing gradient*, pois ela penaliza fortemente as previsões incorretas, o que pode ajudar a manter os gradientes maiores durante o processo de *backpropagation*.

1.2.10 Exemplos

- Dígitos Manuscritos: Para exemplificar o funcionamento de como uma rede neural faz o reconhecimento de imagens, considere a tarefa de classificação de dígitos manuscritos utilizando o conjunto de dados MNIST (LECUN, 1998). O conjunto de dados MNIST consiste em 70.000 imagens em escala de cinza de dígitos manuscritos (0 a 9), cada uma com uma resolução de 28×28 pixels, como mostra a Figura 1.14.

Figura 1.14: Exemplos de dígitos manuscritos do conjunto de dados MNIST.



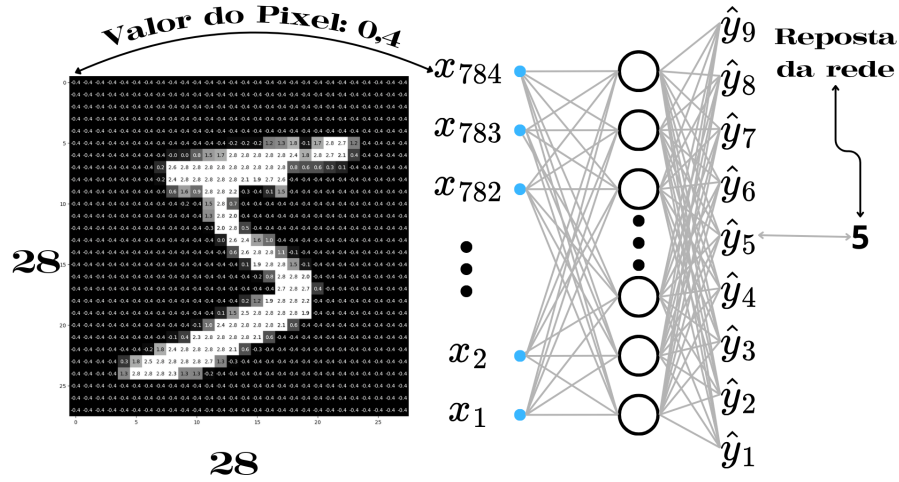
Fonte: Adaptado de (LECUN, 1998).

Cada imagem é representada como uma matriz de 28×28 , em que cada pixel tem um valor de intensidade entre 0 (preto) e 255 (branco). Para o reconhecimento dos dígitos utilizou-se uma rede *feedforward* e os métodos presentes em (1.2.2), (1.2.3), (1.2.6) e (1.2.8) para o treinamento e otimização dos pesos. Para compor a camada de entrada da rede neural, cada imagem é “achatada” em um vetor de 784 elementos ($28 \times 28 = 784$), onde cada elemento corresponde à intensidade de um pixel. O objetivo da rede neural *feedforward* com camadas totalmente conectadas é aprender a classificar corretamente cada imagem em uma das 10 classes correspondentes aos dígitos de 0 a 9. Como a camada de saída da rede neural deve ter 10 unidades (um para cada classe), a arquitetura da rede pode ser definida como $[784, 30, 9]$, onde a camada de entrada tem 784 unidades, a camada oculta tem 30 unidades (valor escolhido aleatoriamente) e a camada de saída tem 9 unidades, como mostra a Figura 1.15.

A função de ativação utilizada é a função sigmoid logística vista na equação (1.3), que é comumente usada em tarefas de classificação. A saída da rede é um vetor de 10 elementos,

Figura 1.15: Arquitetura da rede neural para classificação de dígitos manuscritos.

Inicialmente, é enviado o número de pixels da imagem (784) para a camada de entrada (unidade $x_{784} = 0,4$ intensidade do pixel), que é processada por uma camada oculta com 30 unidades, e a camada de saída tem 9 unidades, correspondendo às 10 classes de dígitos (0 a 9).



Fonte: Elaboração do Autor.

em que cada elemento representa a probabilidade da imagem pertencer a uma das classes de dígitos. A classe com a maior probabilidade é escolhida como a previsão da rede para a imagem de entrada (para a Figura 1.15 a resposta da rede é 5, que foi a unidade com a maior probabilidade).]

Para o treinamento da rede é utilizado o algoritmo de *backpropagation* (Algoritmo 2) e o Gradiente Descendente Estocástico, com uma taxa de aprendizagem de 0,1 e a normalização $L1$ visto na equação (1.11). O treinamento é realizado para 30 épocas. O banco de dados é dividido em um conjunto de treinamento com 60.000 imagens e um conjunto de teste com 10.000 imagens, aleatoriamente. Após o treinamento, a rede neural é avaliada no conjunto de teste para medir sua acurácia, que é a proporção de previsões corretas em relação ao total de previsões feitas:

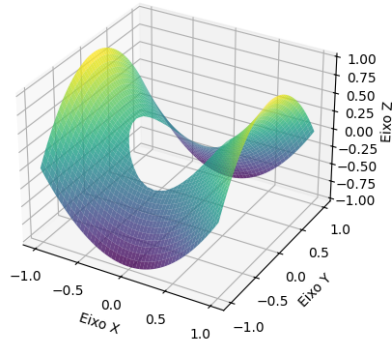
$$\text{Acurácia} = \frac{\text{Número de Previsões Corretas}}{\text{Número Total de Previsões}}. \quad (1.17)$$

A Figura 1.16 mostra o fluxo de dados durante o processo de treinamento e avaliação da rede neural para classificação de dígitos manuscritos.

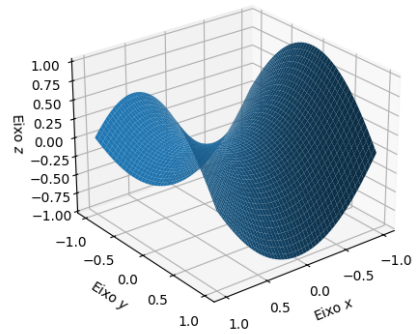
Com as configurações mencionadas, a rede neural alcançou uma acurácia de aproximadamente 95% no conjunto de teste, o que indica que esta foi capaz de aprender a classificar corretamente a maioria das imagens de dígitos manuscritos.

- **Reconstrução de uma Figura Geométrica:** Pode-se usar redes neurais para reconstruir malhas geométricas (SENA; JAFELICE; SANTOS, 2024). Neste caso, utilizou-se uma rede *Multi layer perceptron* (MLP) com duas unidades na camada de entrada, dez neurônios na camada oculta e um neurônio na camada de saída, para aproximar o gráfico do parapo-

Figura 1.17: Paraboloide hiperbólico.

Superfície: $z = x^2 - y^2$ com corte em $x^2 + y^2 \leq 0.25$ 

(a)

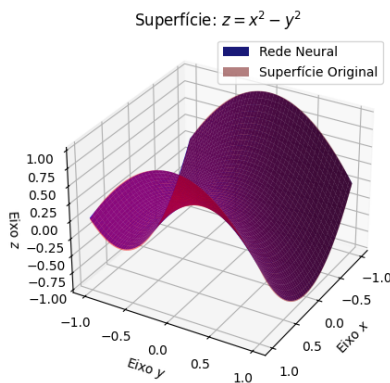
Aproximação da Superfície: $z = x^2 + y^2$ 

(b)

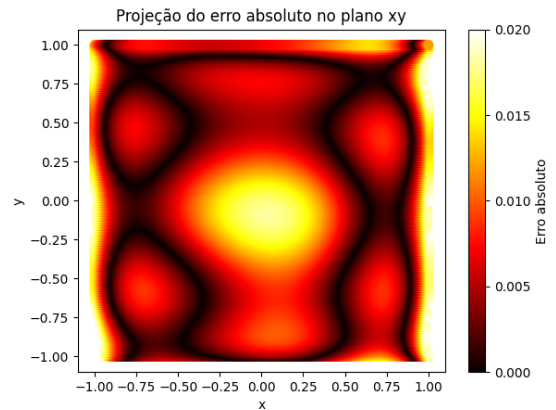
Fonte: Elaboração do Autor.

solução analítica é exibida na Figura 1.18(b), em que os pontos de E com as cores mais claras estão próximos de 0.02 e os mais escuros, próximos de 0.

Figura 1.18: Gráfico da superfície e aproximação da superfície obtida pela MLP e a projeção do E no plano xy.



(a)



(b)

Fonte: Elaboração do Autor.

1.3 Rede Neural para Análise de Imagens

1.3.1 Córtex Visual Humano e Visão Computacional

O córtex visual humano é a parte do cérebro responsável pelo processamento das informações visuais recebidas pelos olhos (STANFIELD, 2014). Este está localizado na parte posterior do cérebro, na região occipital. O córtex visual é organizado em várias áreas, cada uma especia-

lizada em processar diferentes aspectos da visão, como cor, forma, movimento e profundidade. Essas áreas trabalham em conjunto para interpretar e compreender o mundo visual ao nosso redor. A estrutura do córtex visual é altamente complexa, com bilhões de unidades interconectadas que formam redes neurais intrincadas. Essas redes são capazes de aprender e adaptar-se com base nas experiências visuais, permitindo-nos reconhecer objetos, rostos e padrões visuais de maneira eficiente. Uma característica importante do córtex visual é a presença de campos receptivos, que são regiões específicas do campo visual onde a estimulação de uma unidade resulta em uma resposta. Esses campos receptivos são organizados de forma hierárquica, com unidades em níveis mais baixos respondendo a características simples, como bordas e texturas, enquanto unidades em níveis mais altos respondem a características mais complexas, como formas e objetos completos. É por meio desse mecanismo e também pela capacidade de aprendizagem que o ser humano é capaz de reconhecer e interpretar o mundo visual ao seu redor.

Traduzir a visão para uma representação digital é um processo extremamente complexo. Em parte, porque é um problema inverso, no qual busca-se recuperar algumas variáveis desconhecidas a partir de informações insuficientes para especificar completamente a solução. Portanto, deve-se recorrer a modelos baseados na física, modelos probabilísticos ou aprendizado de máquina com grandes conjuntos de exemplos para resolver ambiguidades entre possíveis soluções. Entretanto, modelar o mundo visual em toda a sua rica complexidade é muito mais difícil do que, por exemplo, modelar o trato vocal que produz sons da fala (SZELISKI, 2010).

É surpreendente que humanos e animais façam isso com tanta facilidade, enquanto algoritmos de visão computacional são tão propensos a erros. Pessoas que nunca trabalharam na área frequentemente subestimam a dificuldade do problema. Essa percepção equivocada de que a visão deveria ser fácil remonta aos primeiros dias da inteligência artificial, quando se acreditava inicialmente que as partes cognitivas (prova lógica e planejamento) da inteligência eram intrinsecamente mais difíceis do que os componentes perceptivos (MÜLLER, 2008)³.

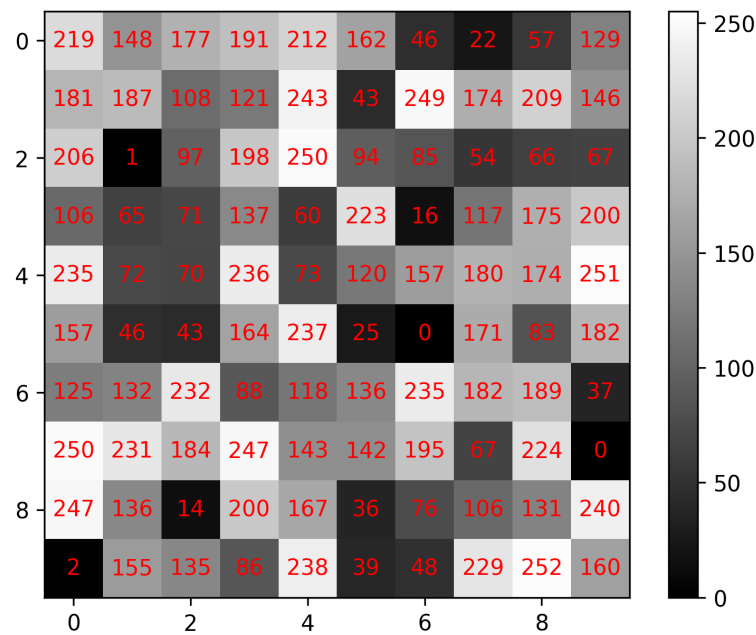
Uma imagem virtual é uma representação visual de um objeto, cena ou conceito, composta por uma matriz de pixels que contém informações sobre cor e intensidade. Cada pixel é uma célula na matriz da imagem que possui um valor específico. As imagens podem ser em escala de cinza, em que cada pixel tem um valor de intensidade entre 0 (preto) e 255 (branco), como mostra a Figura 1.19, ou em cores, em que cada pixel é representado por uma combinação de valores para as cores primárias (vermelho, verde e azul - RGB), uma imagem colorida pode ser interpretada como a sobreposição de três matrizes, cada matriz representando uma das cores primárias.

1.3.2 Redes Neurais Convolucionais

As redes neurais convolucionais (RNCs ou do inglês *Convolutional Neural Networks* - CNNs) são uma classe especial de redes neurais projetadas para processar dados com uma estrutura de grade e que tenham uma relação de vizinhança entre os elementos, como imagens (TREVISAN,

³(MÜLLER, 2008) cita (CREVIER, 1993) como a fonte original. Entretanto o artigo original foi escrito por Seymour Papert (1966) e envolveu uma turma de estudantes.

Figura 1.19: Exemplo de uma imagem/matriz 10×10 em escala de cinza, em que cada pixel tem um valor de intensidade entre 0 (preto) e 255 (branco).



Fonte: Elaboração do Autor.

2021), foram baseadas do Cortex Visual Humano (1.3.1). Estas são amplamente utilizadas em tarefas de visão computacional, como reconhecimento de objetos, detecção de faces e segmentação⁴ de imagens. As RNCs são inspiradas na organização do córtex visual, onde diferentes regiões do cérebro são responsáveis por processar diferentes partes do campo visual.

1.3.3 Convolução

Exemplo 1.3. *Supondo que na tentativa de localizar uma nave com um sensor a laser (GO-ODFELLOW; BENGIO; COURVILLE, 2016), este sensor retorna uma única saída $x(t)$, a posição da nave no tempo t . $x(t)$ e t são valores reais, ou seja, é possível adquirir diferentes leituras do sensor em qualquer instante de tempo. Supondo que o sensor seja ruidoso, para obter pouco ruído da estimativa da posição da nave é possível fazer várias medições e calcular a média destas, entretanto, medições mais recentes são mais relevantes, então, pode-se fazer uma média ponderada que dê pesos às medições mais recentes. Seja esta média a função $w(a)$, em que a é o tempo em que a medição foi obtida. Ao aplicar $w(a)$ à $x(t)$, para todo t , é obtida a função s , que é a estimativa suavizada da posição da nave, ou seja:*

$$s(t) = \int x(a)w(t-a)da. \quad (1.18)$$

⁴Segmentação de imagens é uma modificação feita na área de visão computacional que envolve a divisão de uma imagem em regiões ou segmentos distintos, cada um representando um objeto ou parte de um objeto. O objetivo da segmentação é atribuir uma etiqueta a cada pixel da imagem, indicando a qual classe ou categoria ele pertence.

Essa operação é chamada de convolução (SOVIERZOSKI, 2010). A operação de convolução é tipicamente denotada com um asterístico:

$$s(t) = (x * w)(t).$$

No exemplo 1.3, é preciso que w seja uma função densidade de probabilidade válida para que a saída seja uma média ponderada e w precisa ser igual a zero para $a < 0$. Estas restrições são particulares do exemplo, em geral, convolução é definida para quaisquer funções e integrais.

Na terminologia da rede convolucional, o primeiro termo (x) é frequentemente chamado de *input* (entrada), o segundo termo (w) é chamado de *kernel* (núcleo) ou filtro, e o resultado (s) é chamado de *feature map* (mapa de características).

A operação de convolução é comumente aplicada a dados provenientes de domínios discretos (dados de computador). Se (x) e (w) são definidas somente para valores inteiros de t , a convolução discreta é definida por:

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a)w(t-a), \quad (1.19)$$

em muitos materiais, a convolução discreta pode ser representada por:

$$s[t] = (x * w)[t] = \sum_{a=-\infty}^{\infty} x[a]w[t-a]$$

Exemplo 1.4. Seja $f(t) = e^{-t}u(t)$ e $g(t) = u(t)$, onde $u(t)$ é a função limiar equação (1.2). A convolução $(f * g)(t)$ é dada por:

$$(f * g)(t) = \int_{-\infty}^{\infty} e^{-a}u(a)u(t-a)da.$$

Para $t < 0$, a função limiar $u(t-a)$ é zero para todos os valores de a , o que resulta em $(f * g)(t) = 0$. Para $t \geq 0$, a função limiar $u(t-a)$ é igual a 1 para $a \leq t$ e zero para $a > t$. Portanto, a convolução pode ser reescrita como:

$$(f * g)(t) = \int_0^t e^{-a}da.$$

Calculando a integral, obtem-se:

$$(f * g)(t) = [-e^{-a}]_0^t = 1 - e^{-t}.$$

Portanto, a convolução entre as funções $f(t)$ e $g(t)$ é dada por:

$$(f * g)(t) = \begin{cases} 0, & t < 0 \\ 1 - e^{-t}, & t \geq 0 \end{cases} \quad (1.20)$$

Intuitivamente, a função g é invertida e deslocada por t e depois multiplicada ponto a ponto com a função f , e o resultado é integrado sobre todo o domínio contínuo. Para domínios discretos, o Exemplo 1.5 mostra o processo de convolução entre duas sequências finitas.

Exemplo 1.5. *Seja $x[n] = \{1, 2, 3\}$ e $w[n] = \{0, 1\}$, em que $x[0] = 1$, $x[1] = 2$, $x[2] = 3$, $w[0] = 0$ e $w[1] = 1$ e $w[n] = x[n] = 0$ para outros valores de n . A convolução $(x * w)[n]$ é dada por:*

$$s[n] = (x * w)[n] = \sum_{a=0}^3 x[a]w[n-a].$$

Calculando os valores para cada n , obtém-se:

$$(n = 0) \quad (x * w)[0] = x[0]w[0] + x[1]w[-1] + x[2]w[-2] = 1 \cdot 0 + 2 \cdot 0 + 3 \cdot 0 = 0$$

$$(n = 1) \quad (x * w)[1] = x[0]w[1] + x[1]w[0] + x[2]w[-1] = 1 \cdot 1 + 2 \cdot 0 + 3 \cdot 0 = 1$$

$$(n = 2) \quad (x * w)[2] = x[0]w[2] + x[1]w[1] + x[2]w[0] = 1 \cdot 0 + 2 \cdot 1 + 3 \cdot 0 = 2$$

$$(n = 3) \quad (x * w)[3] = x[0]w[3] + x[1]w[2] + x[2]w[1] = 1 \cdot 0 + 2 \cdot 0 + 3 \cdot 1 = 3$$

Portanto, a convolução entre as sequências $x[n]$ e $w[n]$ é dada por:

$$(x * w)[n] = \{0, 1, 2, 3\}. \quad (1.21)$$

Proposição 1.1. *O tamanho do vetor resultante da convolução entre dois vetores de comprimento M e N é dado por $M + N - 1$.*

Demonstração. Seja $x[n] \neq 0$ para $0 \leq n \leq M - 1$ e $w[n] \neq 0$ para $0 \leq n \leq N - 1$. Duas sequências finitas de comprimentos M e N respectivamente. Então, a convolução $(x * w)[n]$ é dada por:

$$y[n] = (x * w)[n] = \sum_{a=-\infty}^{\infty} x[a]w[n-a],$$

para que a soma seja diferente de zero, é necessário que $x[a] \neq 0$ e $w[n-a] \neq 0$, ou seja:

$$0 \leq a \leq M - 1 \quad \text{e} \quad 0 \leq n - a \leq N - 1.$$

Somando a na segunda desigualdade é obtido:

$$a \leq n \leq a + N - 1$$

Como a pode variar entre 0 e $M - 1$, quando $a = 0$, então $n \geq 0$ e quando $a = M - 1$, então $n \leq M - 1 + N - 1 = M + N - 2$. Portanto:

$$0 \leq n \leq M + N - 2,$$

o que implica que o vetor resultante $(y[n])$ tem comprimento $M + N - 1$ □

No Exemplo 1.5, a convolução entre os vetores $x[n]$ e $w[n]$ resultou em um vetor de comprimento 4, que é igual a $3 + 2 - 1 = 4$. Para voltar ao comprimento original do vetor $x[n]$, pode-se fazer um truncamento do vetor resultante, ou seja, remover a mesma quantidade de elementos do início e do final do vetor resultante, o que é conhecido como convolução “*same*”⁵ (ou mesmo em português). No exemplo 1.5, removendo o primeiro e o último elemento do vetor resultante, obtém-se a convolução *same*:

$$(x * w)_{same}[n] = \{1, 2\}.$$

Pode-se pensar na operação de convolução discreta como o processo de multiplicação e soma, em que o kernel w vai sendo “passado” sobre o vetor x ponto a ponto, como mostra a Figura 1.20.

Figura 1.20: Ilustração da operação de convolução discreta. O kernel w é deslizado sobre o vetor x , e em cada posição, a soma ponderada dos valores de x cobertos pelo kernel é calculada para gerar o valor correspondente na saída.

$$\begin{array}{ccc}
 \begin{array}{c} (1, 2, 3) \\ | \\ (1, 0) \end{array} \} & = 1 \cdot 0 = 0 & \longrightarrow & \begin{array}{c} (1, 2, 3) \\ | \\ (1, 0) \end{array} \} & = 1 \cdot 1 + 2 \cdot 0 = 1 \\
 & & & & \searrow & & & & \nearrow \\
 \begin{array}{c} (1, 2, 3) \\ | \\ (1, 0) \end{array} \} & = 2 \cdot 1 + 3 \cdot 0 = 2 & \longrightarrow & \begin{array}{c} (1, 2, 3) \\ | \\ (1, 0) \end{array} \} & = 3 \cdot 1 = 3
 \end{array}$$

Vetor resultante: $(0, 1, 2, 3)$

Fonte: Elaboração do Autor.

No contexto de redes neurais convolucionais, a convolução é aplicada a dados discretos, como imagens representadas por matrizes de pixels. Neste caso, o sinal x é uma matriz bidimensional que representa a imagem de entrada e o *kernel* w é uma matriz menor que atua como um filtro para extrair características específicas da imagem. A operação de convolução em dados discretos é realizada deslizando o *kernel* sobre a imagem e calculando a soma ponderada dos valores dos pixels cobertos pelo *kernel*. A saída da convolução é uma nova matriz, chamada de Mapa de Características, que destaca as características extraídas pela aplicação do *kernel*. No

⁵A convolução *same* é usada quando a dimensão do dado de entrada deve ser mantida na saída, quando isso não é necessário, a convolução pode receber outro nome, como por exemplo, convolução *valid* (CHOWDHURY, 2024), utilizada, quando os valores da saída são somente calculados quando o *kernel* está totalmente contido na entrada.

caso de uma convolução bidimensional discreta, a operação é definida como:

$$S[i, j] = (X * W)[i, j] = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} X[m, n]W[i - m, j - n], \quad (1.22)$$

convolução é uma operação comutativa, ou seja:

$$S[i, j] = (X * W)[i, j] = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} X[i - m, j - n]W[m, n],$$

considerando que X é a matriz da imagem de entrada, W é a matriz do kernel e S é o mapa de características resultante.

Exemplo 1.6. *Considere a convolução de duas matrizes, em que a matriz de entrada*

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

é convoluída com o kernel

$$W = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

para produzir a saída S . Então, como nos exemplos anteriores, é necessário inverter o kernel W e depois deslizar o kernel sobre a matriz de entrada X . Neste exemplo, a convolução é same.

$$W \rightarrow \hat{W} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix},$$

$\hat{W} = J_i W J_j$ em que J_i e J_j são matrizes de permutação que realizam a rotação de 180 graus do kernel original W . Para facilitar o calculo da convolução, pode-se separar X com blocos de 2×2 (tamanho do kernel) e calcular a convolução para cada bloco. Para o bloco superior esquerdo de X até o bloco inferior direito, temos:

$$\begin{aligned} S(0, 0) &= \begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix} \odot \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} = (1)(-1) + (2)(0) + (4)(0) + (5)(1) = 4, \\ S(0, 1) &= \begin{bmatrix} 2 & 3 \\ 5 & 6 \end{bmatrix} \odot \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} = (2)(-1) + (3)(0) + (5)(0) + (6)(1) = 4, \\ S(1, 0) &= \begin{bmatrix} 4 & 5 \\ 7 & 8 \end{bmatrix} \odot \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} = (4)(-1) + (5)(0) + (7)(0) + (8)(1) = 4, \\ S(1, 1) &= \begin{bmatrix} 5 & 6 \\ 8 & 9 \end{bmatrix} \odot \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} = (5)(-1) + (6)(0) + (8)(0) + (9)(1) = 4, \end{aligned}$$

em que a operação $\odot$ significa a multiplicação de cada elemento correspondente das matrizes e

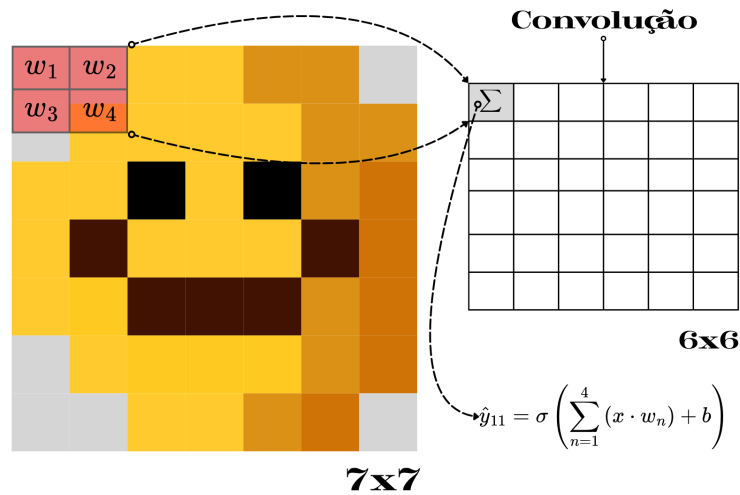
a soma dos resultados. Portanto, a saída da convolução é dada por:

$$S = \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix}. \quad (1.23)$$

Observa-se que a saída da convolução é uma matriz de 2×2 . O cálculo da dimensão da matriz de saída é feito da seguinte forma: $Largura_S = largura_X - largura_W + 1 = 3 - 2 + 1 = 2$ e $Altura_S = altura_X - altura_W + 1 = 3 - 2 + 1 = 2$.

Na Figura 1.21 é ilustrado um exemplo de convolução bidimensional, em que uma matriz de entrada X (matriz 7×7) é convoluída com um *kernel* W (matriz 2×2) para produzir a saída S (matriz 6×6). W é deslizado sobre a matriz de entrada, e em cada posição, a soma ponderada dos valores dos pixels cobertos pelo *kernel* é calculada e o resultado é aplicado em uma função de ativação (σ) para gerar o valor correspondente na matriz de saída, assim como no Exemplo 1.6. No caso da Figura 1.21, é utilizado a convolução válida, ou seja, a saída é gerada apenas para as posições onde o *kernel* pode ser completamente aplicado à matriz de entrada sem ultrapassar seus limites.

Figura 1.21: Exemplo de convolução bidimensional. A matriz de entrada X (matriz 7×7) é convoluída com o *kernel* W (matriz 2×2) para produzir a saída S (matriz 6×6). O *kernel* é “deslizado” sobre a matriz de entrada, formando sub-matrizes, e em cada posição, a soma ponderada dos valores dos pixels cobertos pelo *kernel* é calculada e o resultado é aplicado em uma função de ativação (σ) para gerar o valor correspondente ($\hat{y}_{11}$) na matriz de saída. b sendo o viés adicionado à soma ponderada.



Fonte: Elaboração do Autor.

O cálculo da largura e da altura da imagem de saída pode ser feito utilizando as seguintes fórmulas:

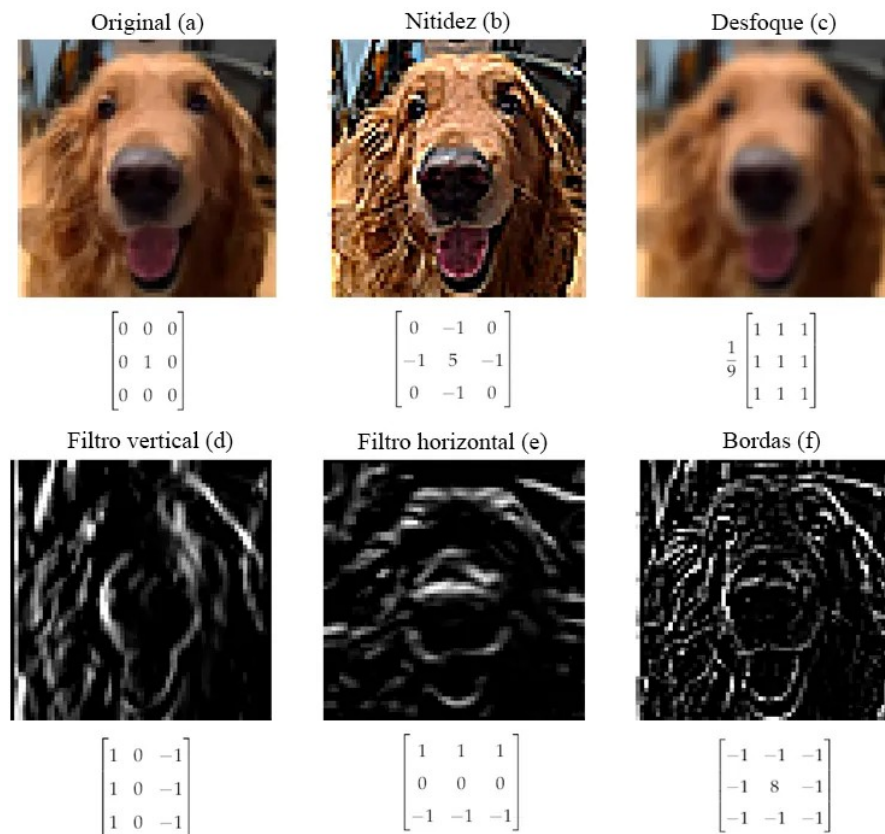
$$w_{out} = w_{in} - w_{ker} + 1, \quad (1.24)$$

$$h_{out} = h_{in} - h_{ker} + 1, \quad (1.25)$$

em que w_{out} e h_{out} são a largura e a altura da imagem de saída, w_{in} e h_{in} são a largura e a

altura da imagem de entrada, e w_{ker} e h_{ker} são a largura e a altura do *kernel*, respectivamente. A Figura 1.22 mostra diferentes aplicações de *kernels* em uma imagem.

Figura 1.22: Aplicações de diferentes *kernels* em uma imagem. A imagem original é representada em (a), juntamente com o *kernel* identidade, que não faz modificações. Em (b) a imagem passou por um *kernel* que aumenta a nitidez reforçando o pixel central e aumentando a diferença dele em relação à sua vizinhança. Em (c) foi usado um *kernel* de desfoque, que substitui um pixel pela média dele com seus vizinhos. A imagem em (d) é o resultado de aplicar na imagem em escala de cinza um *kernel* que reforça suas linhas verticais, enquanto (e) reforça suas linhas horizontais. O *kernel* em (f) reforça os contornos (bordas) da imagem.



Fonte: Adaptado de (TREVISAN, 2021).

Normalmente os *kernels* são muito menores do que as imagens (geralmente 3×3), e através da operação de convolução um *kernel* é utilizado para processar toda uma imagem, portanto, diz-se que os parâmetros são compartilhados (pesos e viés). Por conta dessa representação reduzida, cada camada da rede convolucional deve aprender uma quantidade menor de parâmetros. Em CNNs, normalmente considera-se a convolução como uma camada da rede, e cada camada normalmente possui múltiplos *kernels*, produzindo a mesma quantidade de mapas de atributos, cada um responsável por extrair diferentes características da imagem de entrada. Durante o treinamento da rede, os valores dos *kernels* são ajustados para otimizar a capacidade da rede de reconhecer padrões específicos nas imagens. Por exemplo, uma camada convolucional com 3 *kernels* que recebe como entrada uma imagem em escala de cinza de dimensões $(w_{in} \times h_{in} \times 1)$ produzirá uma saída com dimensões $(w_{out} \times h_{out} \times 3)$, onde w_{out} e h_{out} são calculados conforme as fórmulas apresentadas em (1.24) e (1.25). A saída não é mais uma imagem comum, mas sim

um conjunto de três mapas de características, cada um representando a resposta da imagem de entrada a um dos três *kernels* aplicados. Encadeando os mapas de atributos, a rede pode aprender representações hierárquicas das características presentes nas imagens, desde bordas e texturas simples até formas e objetos mais complexos.

1.3.4 Correlação Cruzada

As redes neurais convolucionais geralmente não utilizam a definição tradicional de convolução (GOODFELLOW; BENGIO; COURVILLE, 2016), mas sim uma operação chamada correlação cruzada, definida como:

$$S[i, j] = (X * W)[i, j] = \sum_m \sum_n X[i + m, j + n] W[m, n], \quad (1.26)$$

em que X é a matriz da imagem de entrada, W é a matriz do *kernel* e S é o mapa de características resultante. Isso ocorre porque no processo de treinamento os valores do *kernel* não são definidos manualmente, mas sim aprendidos por meio dos processos de otimização dos pesos. Por conta disso, a rede aprende a melhor configuração do *kernel* sem a necessidade de realizar a rotação de 180 graus, sendo exatamente o que a definição de correlação cruzada descreve, já que W não depende de parâmetros negativos.

1.3.5 Kernel Multicamadas

Em uma imagem com mais de um canal, como no caso de uma imagem VVA (vermelho, verde, azul) do inglês RGB (*Red, Green, Blue*), a convolução acontece com um *kernel* em cada canal da imagem, um conjunto de canais é chamado de filtro. Por exemplo, considere uma imagem RGB de dimensões $(w_{in} \times h_{in} \times 3)$ e um *kernel* de dimensões $(k_w \times k_h \times 3)$. A convolução é realizada separadamente em cada canal (vermelho, verde e azul) da imagem, utilizando o respectivo canal do *kernel*. Após a aplicação do *kernel* em cada canal, os resultados são somados ou é aplicada uma média para produzir um único mapa de características. Portanto, a saída da convolução terá dimensões $(w_{out} \times h_{out} \times 1)$, onde w_{out} e h_{out} são calculados conforme as fórmulas apresentadas em (1.24) e (1.25).

1.3.6 Padding

Durante a aplicação da convolução, é comum que a dimensão espacial (largura e altura) da imagem de saída seja menor do que a da imagem de entrada. Isso ocorre porque o *kernel* não pode ser aplicado nas bordas da imagem sem ultrapassar seus limites. Para mitigar essa redução de dimensão, uma técnica chamada *padding* (O'SHEA; NASH, 2015) é frequentemente utilizada. O *padding* envolve adicionar uma borda de pixels ao redor da imagem original antes de aplicar a convolução. Esses pixels adicionais podem ser preenchidos com zeros (*zero-padding*) ou com valores replicados dos pixels mais próximos da borda da imagem original, (*reflection padding*). O uso de *padding* permite que o *kernel* seja aplicado em todas as regiões da imagem,

incluindo as bordas, preservando assim as dimensões espaciais da imagem de entrada na saída. A Figura 1.23 ilustra a aplicação de diferentes tipos de *padding* em uma imagem.

Figura 1.23: Aplicação de *padding* em uma imagem. Em (a) a imagem original passou por zero *padding*. Em (b) a imagem original passou por *reflection padding*. A imagem em (c) passou por um *padding* constante, semelhante em (a), mas com valor diferente de zero.



Fonte: Adaptado de (TREVISAN, 2021).

O cálculo da largura e da altura da imagem de saída com *padding* pode ser ajustado conforme as seguintes fórmulas:

$$\begin{aligned} w_{out} &= w_{in} - w_{kernel} + 2w_{pad} + 1, \\ h_{out} &= h_{in} - h_{kernel} + 2h_{pad} + 1, \end{aligned} \quad (1.27)$$

em que w_{pad} e h_{pad} são a quantidade de pixels adicionados como *padding* na largura e na altura, respectivamente.

1.3.7 *Stride*

O *stride* (passo) é um parâmetro utilizado nas operações de convolução e *pooling* em redes neurais convolucionais que determina o número de pixels que o *kernel* de *pooling* se desloca ao longo da imagem de entrada. Em outras palavras, o *stride* define o quanto o *kernel* avança a cada aplicação da operação. Um *stride* de 1 significa que o *kernel* se mova um pixel por vez, enquanto um *stride* maior que 1 faz com que o *kernel* pule pixels, resultando em uma redução mais rápida das dimensões espaciais da imagem. O uso do *stride* afeta diretamente o tamanho da saída gerada pela operação de convolução. O cálculo da largura e da altura da imagem de saída com *stride* pode ser ajustado conforme as seguintes fórmulas:

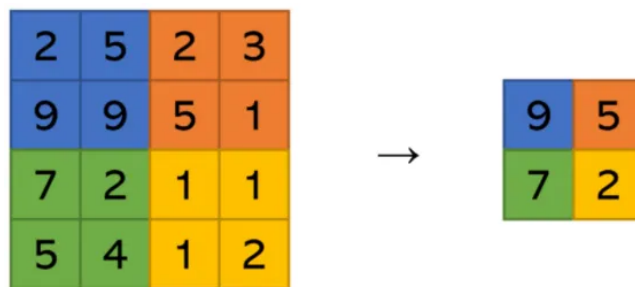
$$\begin{aligned} w_{out} &= \frac{w_{in} - w_{kernel} + 2w_{pad}}{s_w} + 1, \\ h_{out} &= \frac{h_{in} - h_{kernel} + 2h_{pad}}{s_h} + 1, \end{aligned} \quad (1.28)$$

em que s_w e s_h são os valores do *stride* na largura e na altura, respectivamente.

1.3.8 Pooling

O *pooling* (em tradução livre, *agrupamento*) é uma operação utilizada em redes neurais convolucionais para reduzir as dimensões espaciais (largura e altura) dos mapas de características, mantendo as informações mais relevantes. Essa redução ajuda a diminuir a complexidade computacional da rede. Existem diferentes tipos de operações de *pooling*, sendo as mais comuns o *Max Pooling* e o *Average Pooling* (Agrupamento Médio, técnica que leva em consideração a média dos valores dos pixels em cada bloco coberto pelo *kernel* de *pooling*). A técnica de *pooling* substitui agrupamentos de pixels por um valor que os represente. A Figura 1.24 ilustra o funcionamento do *max pooling* de tamanho 2, em que os pixels são agrupados em quadrados de tamanho 2x2 e são substituídos pelo maior valor de intensidade de pixel do grupo. Em CNN's o *stride* do *pooling* é geralmente igual ao tamanho do *kernel* de *pooling*, o que resulta em uma redução de dimensão espacial pela metade a cada aplicação de *pooling*. Por exemplo, um mapa de características de dimensões 8×8 aplicado a um *max pooling* com *kernel* de tamanho 2×2 e *stride* de 2 resultará em um mapa de características reduzido para dimensões 4×4 .

Figura 1.24: Exemplo de *Max pooling* com tamanho de 2x2. A imagem original é reduzida para a imagem menor após a aplicação do *Max pooling*. Cada bloco de 2x2 pixels na imagem original é substituído pelo valor máximo dentro desse bloco na imagem reduzida.



Fonte: Adaptado de (TREVISAN, 2021).

O *max pooling* pode ser implementado utilizando o Algoritmo 3, em que X é a matriz de entrada, k é o tamanho do *kernel* de *pooling* e Y é a matriz de saída resultante do *pooling*. O algoritmo percorre a matriz de entrada em blocos de tamanho $k \times k$, calculando o valor máximo dentro de cada bloco e atribuindo esse valor à posição correspondente na matriz de saída Y . O resultado é uma matriz reduzida que mantém as características mais importantes da imagem original.

O cálculo da largura e da altura da imagem de saída após a aplicação do *pooling*, quando o *stride* é igual ao tamanho do *kernel*, pode ser realizado conforme as seguintes fórmulas:

$$\begin{aligned} w_{out} &= \lfloor \frac{w_{in}}{k} \rfloor, \\ h_{out} &= \lfloor \frac{h_{in}}{k} \rfloor, \end{aligned} \tag{1.29}$$

em que k é o tamanho do *kernel* de *pooling*, e $\lfloor \cdot \rfloor$ denota a função piso, que arredonda para baixo o resultado da divisão. Como consequência imediata da aplicação do *pooling*, a informação fica

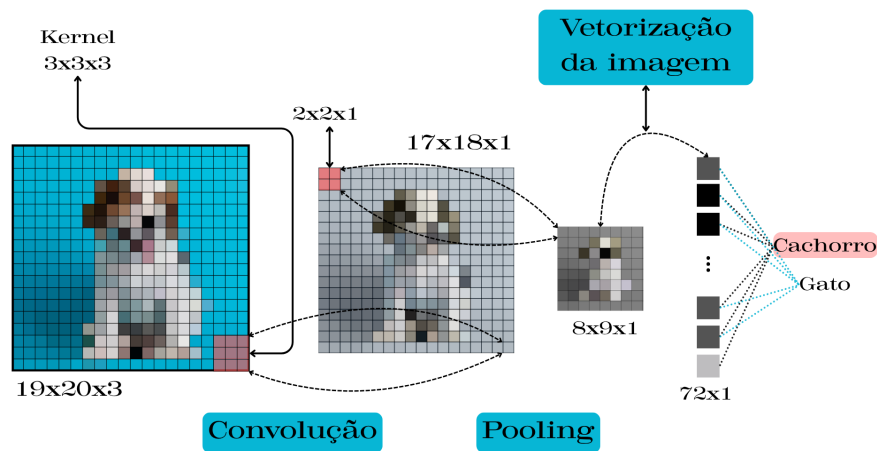
Entrada: X (matriz de entrada), k (tamanho do *kernel* de *pooling*)
Saída: Y (matriz de saída)
 $w_{out} \leftarrow \lfloor w_{in}/k \rfloor$;
 $h_{out} \leftarrow \lfloor h_{in}/k \rfloor$;
 $Y \leftarrow$ matriz de zeros de tamanho $w_{out} \times h_{out}$;
Para $i \leftarrow 0$ **Até** $w_{out} - 1$ **Faça**
 Para $j \leftarrow 0$ **Até** $h_{out} - 1$ **Faça**
 $Y[i][j] \leftarrow \max(X[i * k : (i + 1) * k][j * k : (j + 1) * k]);$
 Fim
Fim

Algoritmo 3: Max *pooling*

mais condensada, o que pode levar a uma perda de detalhes finos na imagem. No entanto, essa perda é compensada pela capacidade de aumentar a quantidade dos mapas de característica da imagem, melhorando sua capacidade de generalização em tarefas de reconhecimento de padrões.

Exemplo 1.7. A Figura 1.25 apresenta uma arquitetura típica de uma Rede Neural Convolutiva (CNN), com uma camada de convolução e uma camada de pooling. Como a imagem tem três canais (RGB) de dimensões $19 \times 20 \times 3$ o kernel de convolução também tem três canais ($3 \times 3 \times 3$), a aplicação da convolução resulta em um mapa de características com um canal ($17 \times 18 \times 1$). O mapa de características é então processado por uma camada de pooling ($2 \times 2 \times 1$), que reduz suas dimensões espaciais ($8 \times 9 \times 1$). Após essa etapa a rede transforma o mapa de características em um vetor unidimensional (72×1), que é processado por camadas totalmente conectadas, visto na Seção 1.2 para realizar a classificação do objeto em cachorro ou gato, com a rede escolhendo a classe com maior probabilidade como resultado final (cachorro).

Figura 1.25: Arquitetura típica de uma Rede Neural Convolutiva com uma camada de convolução e uma camada de *pooling*, em que é enviado para rede a imagem de um cachorro e é feita a classificação do objeto na imagem.



Fonte: Elaboração do Autor.

1.3.9 Convolução Transposta

A convolução transposta⁶, também conhecida como deconvolução, é uma operação utilizada em redes neurais convolucionais para aumentar as dimensões espaciais (largura e altura) dos mapas de características. Essa operação é frequentemente empregada em tarefas de geração de imagens, em que o objetivo é reconstruir imagens a partir de representações compactas. A convolução transposta funciona de maneira inversa à convolução válida. Enquanto a convolução reduz as dimensões espaciais da imagem de entrada, a convolução transposta expande essas dimensões. Isso é alcançado gerando uma saída com dimensões calculadas da seguinte forma:

$$\begin{aligned} w_{out} &= (w_{in} - 1) \cdot s_w - 2w_{pad} + w_{kernel}, \\ h_{out} &= (h_{in} - 1) \cdot s_h - 2h_{pad} + h_{kernel}, \end{aligned} \quad (1.30)$$

em que w_{in} e h_{in} são a largura e a altura da imagem de entrada, s_w e s_h são os valores do *stride* na largura e na altura, w_{pad} e h_{pad} são a quantidade de pixels adicionados como *padding* na largura e na altura (geralmente os valores para o *padding* são zero), e w_{kernel} e h_{kernel} são a largura e a altura do *kernel*, respectivamente. Cada entrada da imagem de saída é calculada como uma soma ponderada dos valores da imagem de entrada multiplicados pelos pesos do *kernel*. A convolução transposta pode ser implementada utilizando o seguinte Algoritmo 4.

Entrada: X (matriz de entrada), W (*kernel*), s_w (*stride* na largura), s_h (*stride* na altura), w_{pad} (*padding* na largura), h_{pad} (*padding* na altura)

Saída: Y (matriz de saída)

$w_{out} \leftarrow (w_{in} - 1) \cdot s_w - 2w_{pad} + w_{kernel};$
 $h_{out} \leftarrow (h_{in} - 1) \cdot s_h - 2h_{pad} + h_{kernel};$
 $Y \leftarrow$ matriz de zeros de tamanho $w_{out} \times h_{out};$

Para $i \leftarrow 0$ **Até** $w_{in} - 1$ **Faça**

Para $j \leftarrow 0$ **Até** $h_{in} - 1$ **Faça**

Para $m \leftarrow 0$ **Até** $w_{kernel} - 1$ **Faça**

Para $n \leftarrow 0$ **Até** $h_{kernel} - 1$ **Faça**

$Y[i \cdot s_w + m - w_{pad}][j \cdot s_h + n - h_{pad}] += X[i][j] \cdot W[m][n];$

Fim

Fim

Fim

Fim

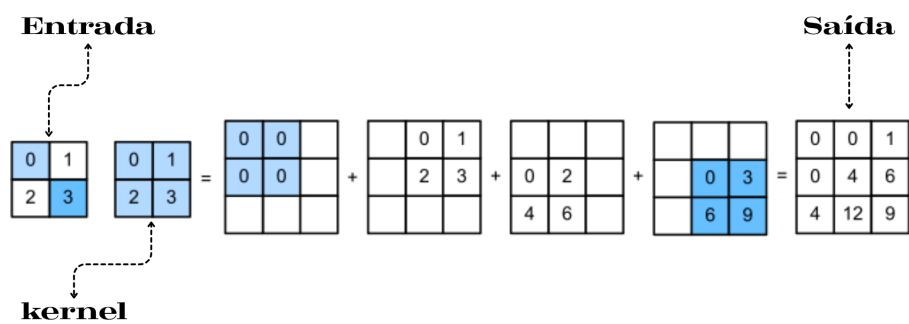
Algoritmo 4: Convolução Transposta

A Figura 1.26 exemplifica a operação de convolução transposta, em que uma matriz de entrada X (matriz 2×2) é operada com um *kernel* W (matriz 2×2) para produzir a saída Y (matriz 3×3).

Colocar o exeplo dos dígitos manuscritos

⁶O nome sugere uma operação inversa à convolução, entretanto não é isso que acontece. Esse nome provavelmente veio da ideia de “inverter” a operação de convolução no sentido de expandir as dimensões espaciais da entrada e não diminuir.

Figura 1.26: Exemplo da operação de convolução transposta.



Fonte: Elaboração do Autor.

Capítulo 2

Teoria dos Conjuntos Fuzzy

2.1 Teoria dos Conjuntos Fuzzy

O capítulo 2 apresenta a teoria dos conjuntos fuzzy e os sistemas baseados em regras fuzzy, que são fundamentais para o desenvolvimento do modelo de segmentação de imagens proposto neste trabalho. A seção 2.1 introduz os conceitos básicos da teoria dos conjuntos fuzzy, incluindo a definição de conjuntos fuzzy, operações entre conjuntos fuzzy, normas triangulares, níveis de um conjunto fuzzy e números fuzzy. A seção 2.2 apresenta os sistemas baseados em regras fuzzy, descrevendo suas componentes e os métodos de inferência e defuzzificação utilizados para produzir saídas a partir de entradas dadas por conjuntos fuzzy.

A teoria dos conjuntos fuzzy, introduzida por Lotfi A. Zadeh em 1965 (ZADEH, 1965), é uma extensão da teoria clássica dos conjuntos, permitindo a representação de incerteza e imprecisão. Diferentemente dos conjuntos clássicos, onde um elemento pertence ou não a um conjunto, nos conjuntos fuzzy cada elemento possui um grau de pertinência, variando entre 0 e 1. Essa abordagem é especialmente útil em situações onde as fronteiras entre categorias não são claramente definidas, permitindo uma modelagem mais flexível e realista de fenômenos complexos. As definições e propriedades dos conjuntos fuzzy presentes neste capítulo foram baseadas em (JAFELICE; BARROS; BASSANEZI, 2023).

2.1.1 Conjuntos Fuzzy

Um *subconjunto fuzzy* A de um conjunto universo X é definido por uma função de pertinência μ_A que atribui a cada elemento $x \in X$ um valor $\mu_A(x) \in [0, 1]$, representando o grau de pertencimento de x ao conjunto A . Assim:

$$\mu_A : X \rightarrow [0, 1].$$

O valor $\mu_A(x) = 0$ indica que x não pertence ao conjunto A , enquanto $\mu_A(x) = 1$ indica que x pertence completamente ao conjunto. Valores intermediários representam graus parciais de pertencimento, permitindo uma representação mais rica e flexível de conceitos vagos ou imprecisos.

Exemplo 2.1. Considere o conjunto universo X representando a temperatura em graus Celsius, e o conjunto fuzzy A representando a categoria “quente”. A função de pertinência μ_A pode ser definida da seguinte forma:

$$\mu_A(x) = \begin{cases} 0 & \text{se } 0 \leq x \leq 20 \\ \frac{x-20}{10} & \text{se } 20 < x < 30 \\ 1 & \text{se } x \geq 30 \end{cases}$$

Neste exemplo, temperaturas de 20°C ou menos não são consideradas quentes ($\mu_A(x) = 0$), enquanto temperaturas de 30°C ou mais são consideradas completamente quentes ($\mu_A(x) = 1$). Temperaturas entre 20°C e 30°C têm um grau de pertinência que varia linearmente entre 0 e 1, refletindo a natureza gradual do conceito de “quente”.

2.1.2 Operações entre Conjuntos Fuzzy

As operações entre conjuntos fuzzy, como união, interseção e complemento, são definidas de maneira a preservar a estrutura fuzzy. A união de dois conjuntos clássicos A e B é dada por:

$$A \cup B = \{x \in U; x \in A \text{ ou } x \in B\},$$

enquanto a interseção é definida por:

$$A \cap B = \{x \in U; x \in A \text{ e } x \in B\}.$$

O complemento de um conjunto fuzzy A é dado por:

$$\bar{A} = \{x \in U; x \notin A\}.$$

Essas três operações têm respectivamente as funções características, $\forall x \in X$

$$\mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x)), \quad \mu_{A \cap B}(x) = \min(\mu_A(x), \mu_B(x)), \quad \mu_{\bar{A}}(x) = 1 - \mu_A(x).$$

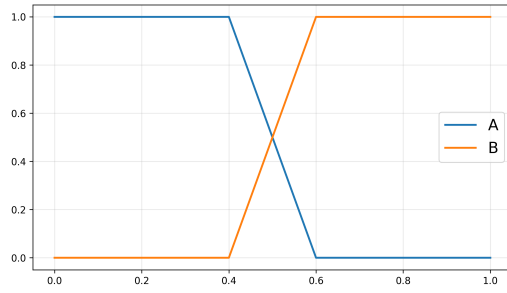
Definição 2.1. Sejam A e B conjuntos fuzzy em um conjunto universo X . A união, interseção e complemento de A e B , denotadas por $A \cup B$, $A \cap B$ e $\bar{A}$, respectivamente, são as funções de pertinência, presentes na Figura 2.1, definidas por:

$$\mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x)) \quad \text{para todo } x \in X,$$

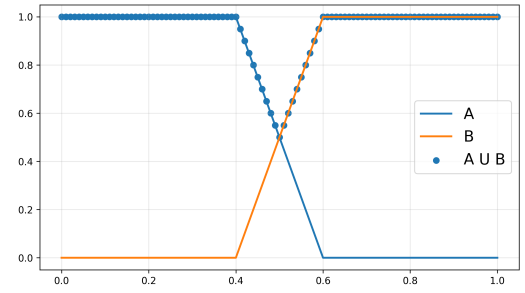
$$\mu_{A \cap B}(x) = \min(\mu_A(x), \mu_B(x)) \quad \text{para todo } x \in X,$$

$$\mu_{\bar{A}}(x) = 1 - \mu_A(x) \quad \text{para todo } x \in X.$$

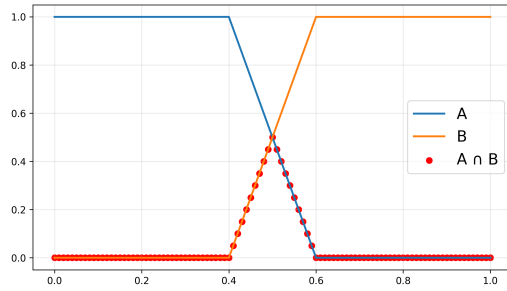
Figura 2.1: Operações entre conjuntos fuzzy.



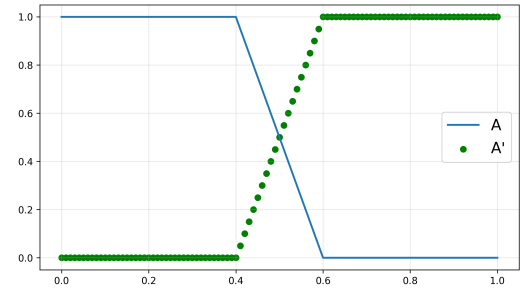
(a) Conjuntos fuzzy A e B.



(b) União e interseção de conjuntos fuzzy A e B.



(c) Interseção de conjuntos fuzzy A e B.



(d) Complemento de um conjunto fuzzy A.

Fonte: Elaboração do Autor.

2.1.3 Normas Triangulares

As normas triangulares são generalizações dos operadores união e intersecção.

Definição 2.2. Uma co-norma triangular (*s-norma*) é uma operação binária $S : [0, 1] \times [0, 1] \rightarrow [0, 1]$ que satisfaz as seguintes propriedades:

- *Comutatividade:* $xSy = ySx$
- *Associatividade:* $xS(ySz) = (xSy)Sz$
- *Monotonicidade:* Se $x \leq x'$ e $y \leq y'$, então $xSy \leq x'Sy'$
- *Condição de fronteira:* $xS0 = x$ e $xS1 = 1$

Exemplo 2.2. Soma Limitada: $S : [0, 1] \times [0, 1] \rightarrow [0, 1]$ Com $xSy = \min(1, x + y)$

Definição 2.3. Uma norma triangular (*t-norma*) é uma operação binária $T : [0, 1] \times [0, 1] \rightarrow [0, 1]$ que satisfaz as seguintes propriedades:

- *Comutatividade:* $xTy = yTx$
- *Associatividade:* $xT(yTz) = (xTy)Tz$
- *Monotonicidade:* Se $x \leq x'$ e $y \leq y'$, então $xTy \leq x'Ty'$
- *Condição de fronteira:* $0Tx = 0$ e $1Tx = x$

Exemplo 2.3. Diferença Limitada: $T : [0, 1] \times [0, 1] \rightarrow [0, 1]$ Com $xTy = \max(0, x + y - 1)$

2.1.4 Níveis de um Conjunto Fuzzy

Definição 2.4. *Sejam A um conjunto fuzzy e $\alpha \in (0, 1]$. O α -nível de A , denotado por $[A]^\alpha$, é o conjunto clássico definido por:*

$$[A]^\alpha = \{x \in X \mid \mu_A(x) \geq \alpha\}.$$

Definição 2.5. *O nível zero de um conjunto fuzzy A do universo X , em que X é um espaço topológico, é o fecho topológico do suporte de A , ou seja:*

$$[A]^0 = \overline{\text{supp}(A)}.$$

Definição 2.6. *O suporte de um conjunto fuzzy A são todos os elementos de X que têm grau de pertinência maior que zero em A , ou seja:*

$$\text{supp}(A) = \{x \in X \mid \mu_A(x) > 0\}.$$

Exemplo 2.4. *Seja A o conjunto fuzzy representando a categoria “quente” do Exemplo 2.1, com a função de pertinência definida por:*

$$\mu_A(x) = \begin{cases} 0 & \text{se } x \leq 20 \\ \frac{x-20}{10} & \text{se } 20 < x < 30 \\ 1 & \text{se } x \geq 30. \end{cases}$$

O α -nível de A para $\alpha = 0,5$ é dado por:

$$[A]^{0,5} = \{x \in \mathbb{R} \mid \mu_A(x) \geq 0,5\}.$$

Neste caso, tem-se:

$$\mu_A(x) \geq 0,5 \implies \frac{x-20}{10} \geq 0,5 \implies x \geq 25.$$

Portanto, o α -nível de A para $\alpha = 0,5$ é o intervalo $[25, \infty)$, representando as temperaturas que são consideradas “quentes” com um grau de pertinência de pelo menos 0,5. O suporte de A é dado por:

$$\text{supp}(A) = \{x \in \mathbb{R} \mid \mu_A(x) > 0\}.$$

Neste caso, tem-se:

$$\mu_A(x) > 0 \implies \frac{x-20}{10} > 0 \implies x > 20.$$

Portanto, o suporte de A é o intervalo $(20, \infty)$, representando as temperaturas que têm um grau de pertinência maior que zero em A , ou seja, as temperaturas que são consideradas “quentes” em algum grau.

2.1.5 Números Fuzzy

Um número fuzzy é um subconjunto fuzzy do conjunto dos números reais, caracterizado por uma função de pertinência que atribui a cada número real um grau de pertencimento. Os números fuzzy são frequentemente utilizados para representar quantidades imprecisas ou vagamente definidas, como “muito quente” ou “pouco tempo”.

Definição 2.7. *Um conjunto fuzzy N é chamado de número fuzzy quando o conjunto universo, em que N é definido, é o conjunto dos números reais $\mathbb{R}$, e a função de pertinência μ_N é tal que:*

- $\mu_N(x) = 1$ para pelo menos um valor x de $\text{supp}(N)$;
- $[N]^\alpha$ é um intervalo fechado para todo $\alpha \in (0, 1]$;
- O suporte de N é limitado.

Exemplo 2.5. *Um número fuzzy A é dito trapezoidal se o gráfico de sua função de pertinência tem a forma de um trapézio. A função de pertinência de um número fuzzy trapezoidal pode ser definida por quatro parâmetros a, b, c, d (com $a \leq b \leq c \leq d$) da seguinte forma:*

$$\mu_A(x) = \begin{cases} \frac{x-a}{b-a}, & \text{se } a < x < b \\ 1, & \text{se } b \leq x < c \\ \frac{d-x}{d-c}, & \text{se } c \leq x < d \\ 0, & \text{caso contrário} \end{cases}$$

Neste exemplo, o número fuzzy A é representado por um trapézio definido pelos parâmetros a, b, c, d . Para $a < x < b$, a função de pertinência aumenta linearmente de 0 a 1, indicando um aumento gradual no grau de pertencimento. Para $b \leq x < c$, a função de pertinência é constante em 1, indicando que esses valores pertencem completamente ao conjunto fuzzy. Para $c \leq x < d$, a função de pertinência diminui linearmente de 1 a 0, indicando uma diminuição gradual no grau de pertencimento.

2.2 Sistema Baseado em Regras Fuzzy

Um sistema Baseado em Regras Fuzzy (SBRF) (JAFELICE; BARROS; BASSANEZI, 2023) é um sistema que utiliza a teoria dos conjuntos fuzzy para produzir saídas a partir de entradas dadas por conjuntos fuzzy. Tais sistemas foram desenvolvidos com o objetivo de imitar o comportamento humano considerando que o conhecimento e as ações humanas são apoiadas em uma sequência de regras linguísticas, traduzidas por um conjunto de regras. As regras formuladas pelo ser humano são usualmente da forma:

SE (um conjunto de condições é satisfeito) *ENTÃO* (um conjunto de consequências pode ser inferido).

Definição 2.8. *Uma relação fuzzy R é um subconjunto fuzzy do produto cartesiano entre conjuntos universos $X_1, X_2, \dots, X_n$, ou seja, $R \subseteq X_1 \times X_2 \times \dots \times X_n$. O produto cartesiano fuzzy $A_1 \times A_2 \times \dots \times A_n$ dos subconjuntos fuzzy $A_1, A_2, \dots, A_n$ de $X_1, X_2, \dots, X_n$, respectivamente, é a relação fuzzy R cuja função de pertinência é dada por:*

$$\mu_R(x_1, x_2, \dots, x_n) = \min(\mu_{A_1}(x_1), \mu_{A_2}(x_2), \dots, \mu_{A_n}(x_n)),$$

2.2.1 Regras e Inferência Fuzzy

Uma regra fuzzy é uma sentença da forma “SE X é A ENTÃO Y é B ”, em que A e B são conjuntos fuzzy em X e Y , respectivamente. Tal regra pode ser interpretada como uma relação fuzzy R entre os conjuntos A e B cuja a função de pertinência $\mu_R(x, y)$ depende de $\mu_A(x)$ e $\mu_B(y)$ para cada $(x, y) \in X \times Y$. Em geral utiliza-se a função:

$$\mu_R(x, y) = \min(\mu_A(x), \mu_B(y)).$$

Uma variável linguística é uma variável cujo o valor é expresso qualitativamente por termos linguísticos, que são representados por conjuntos fuzzy e quantitativamente por uma função de pertinência. Por exemplo, a variável linguística “temperatura” pode assumir os termos linguísticos “fria”, “morna” e “quente” representados por conjuntos fuzzy com funções de pertinência que descrevem o grau de pertencimento de cada temperatura a esses termos.

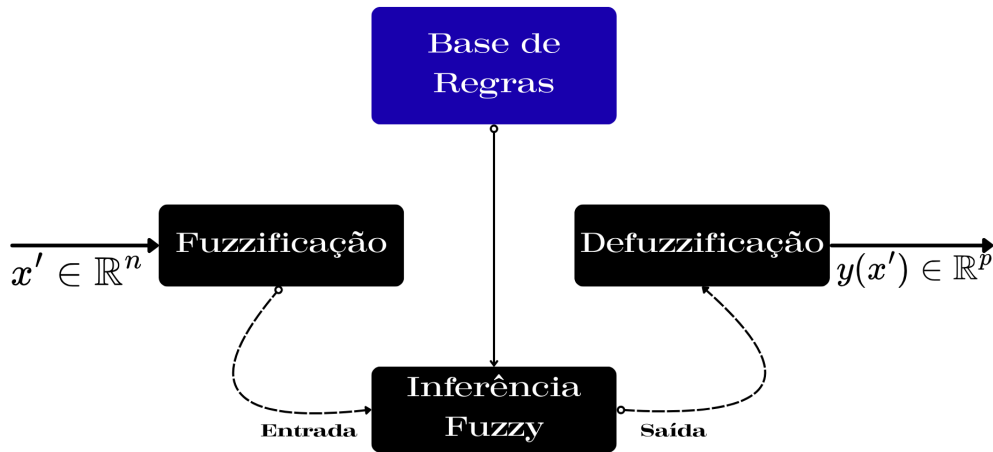
2.2.2 Sistema Baseado em Regras Fuzzy

Um SBRF é composto por quatro componentes: um processador de entrada que realiza a fuzzificação dos dados de entrada, uma coleção de regras, chamada base de regras, uma máquina de inferência fuzzy e um processador de saída que fornece um número real como saída. Estes componentes são conectados como mostrado na Figura 2.2.

O SBRF pode ser visto com um mapeamento entre a entrada e a saída da forma $y = f(x)$, $x \in \mathbb{R}^n$ e $y \in \mathbb{R}^p$. Os componentes do SBRF realizam as seguintes funções:

- O **processador de entrada** (fuzzificação) realiza a fuzzificação dos dados de entrada, ou seja, traduz em conjuntos fuzzy os valores de entrada. A atuação de um especialista é necessária para definir as funções de pertinência que serão utilizadas na fuzzificação, bem como os termos linguísticos que serão associados a cada variável de entrada.
- A **base de regras** contém um conjunto de regras fuzzy que descrevem o comportamento do sistema, essas regras são formuladas por especialistas com base em seu conhecimento sobre o domínio do problema. A base de regras descreve relações entre as variáveis linguísticas para serem utilizadas pela máquina de inferência fuzzy.
- A **máquina de inferência fuzzy** fornece a saída a partir de cada entrada fuzzy e da relação definida pela base de regras. A máquina de inferência fuzzy pode ser vista como

Figura 2.2: Arquitetura de um Sistema Baseado em Regras Fuzzy (SBRF).



Fonte: Adaptado de (JAFELICE; BARROS; BASSANEZI, 2023).

um mecanismo de mapeamento entre os conjuntos fuzzy de entrada e os conjuntos fuzzy de saída, utilizando as regras da base de regras para inferir a saída correspondente a cada entrada. Neste trabalho é utilizado o método de inferência de Mamdani, que é um dos métodos mais comuns para sistemas baseados em regras fuzzy com variáveis linguísticas.

- **Método de inferência de Mamdani:** Uma regra *SE* (antecedente) *ENTÃO* (consequente) é definida pelo produto cartesiano fuzzy dos conjuntos fuzzy que compõem o antecedente e o consequente da regra. O método de Mamdani agrega as regras através do operador lógico *OU*, que é modelado pelo operador do máximo e, em cada regra, o operador lógico *E* é modelado pelo operador do mínimo. As regras são:

Regra 1 : SE (x é A_1 E y é B_1) ENTÃO (z é C_1);

Regra 2 : SE (x é A_2 E y é B_2) ENTÃO (z é C_2).

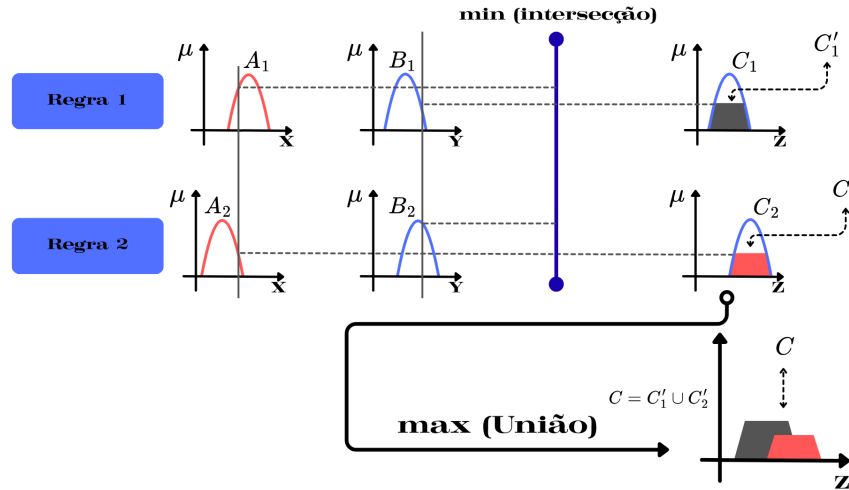
A Figura 2.3 ilustra um exemplo do método de Mamdani, em que duas regras são aplicadas para inferir a saída z a partir das entradas x e y .

Com base no Exemplo 2.1 uma regra que poderia ser formulada utilizando o método de Mamdani é a seguinte:

Regra 1 : SE (x é A) ENTÃO (z é C),

em que A é o conjunto fuzzy representando a categoria “quente” e C é um conjunto fuzzy representando a categoria “baixo”, da variável de saída “temperatura do ar condicionado”. Assim, a regra estabelece que se a temperatura x é considerada quente, ou seja, a função de pertinência de x é alta para o conjunto fuzzy A , então z tem uma função de pertinência

Figura 2.3: Exemplo de inferência fuzzy utilizando o método de Mamdani.



Fonte: Adaptado de (JAFELICE; BARROS; BASSANEZI, 2023).

alta para o conjunto fuzzy C .

- **Método de Takagi-Sugeno:** Neste caso, o consequente de cada regra é uma função das variáveis de entrada. Por exemplo, pode-se supor que a função que mapeia a entrada e a saída para cada regra é uma combinação linear das entradas, ou seja, $z = a_1x_1 + a_2x_2 + \dots + a_nx_n + b$. Assim, as regras do método de Takagi-Sugeno podem ser formuladas da seguinte forma:

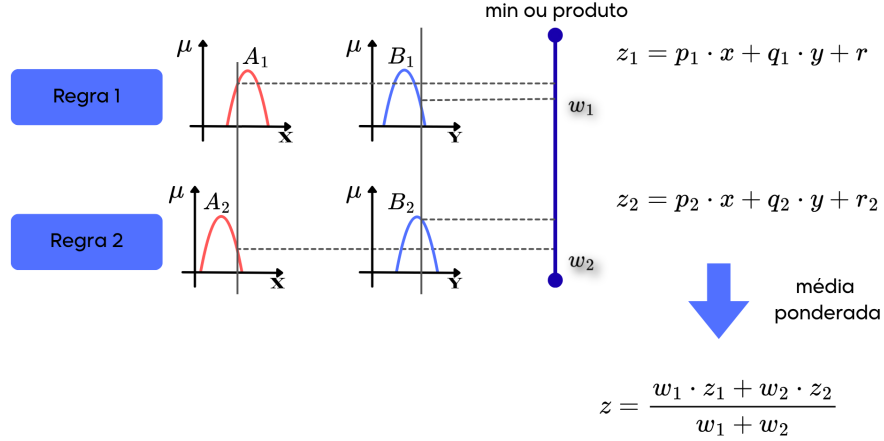
Regra r : SE $(x_1 \text{ é } A_1 \text{ E } x_2 \text{ é } A_2 \dots x_n \text{ é } A_n)$ ENTAO $(z = f_r(x_1, x_2, \dots, x_n))$.

A Figura 2.4 ilustra um exemplo do método de Takagi-Sugeno, em que duas regras são aplicadas para inferir a saída z a partir das entradas x e y . A saída do sistema é obtida pela média ponderada das saídas de cada regra, usando-se o grau de ativação destas regras como ponderação.

- **O processador de saída (defuzzificação)** realiza a defuzzificação, ou seja, converte os conjuntos fuzzy de saída em números reais. Em sistemas fuzzy, em geral a saída é um conjunto fuzzy. Assim, devemos escolher um método de defuzzificação para obter um número real a partir do conjunto fuzzy de saída. O método de defuzzificação mais utilizado é o método de Centro de gravidade, que calcula o centro de massa do conjunto fuzzy de saída para obter um valor real representativo da saída do sistema.

- **Método de Centro de Gravidade:** O método de centro de gravidade é semelhante a média ponderada para distribuições de dados, com a diferença que os pesos são os valores $C(z_i)$ que indicam o grau de compatibilidade do valor z_i com o conceito modelado pelo conjunto fuzzy C . Para domínios discretos, o método de centro de

Figura 2.4: Exemplo de inferência fuzzy utilizando o método de Takagi-Sugeno.



Fonte: Adaptado de (JAFELICE; BARROS; BASSANEZI, 2023).

gravidade é dado por:

$$G(C) = \frac{\sum_{i=0}^n z_i \times C(z_i)}{\sum_{i=0}^n C(z_i)}.$$

Exemplo 2.6. Considere um sistema de controle de velocidade de um ventilador com duas variáveis de entrada: temperatura x (em °C) e umidade y (em %). O objetivo é determinar a velocidade z do ventilador (em RPM) utilizando um SBRF com o método de Takagi-Sugeno.

As variáveis linguísticas são definidas pelas seguintes funções de pertinência trapezoidais:

- Temperatura “Baixa” (A_1):

$$\mu_{A_1}(x) = \begin{cases} 1, & \text{se } x \leq 20 \\ \frac{25-x}{5}, & \text{se } 20 < x < 25 \\ 0, & \text{se } x \geq 25. \end{cases}$$

- Temperatura “Média” (A_2):

$$\mu_{A_2}(x) = \begin{cases} 0, & \text{se } x \leq 20 \text{ ou } x \geq 35 \\ \frac{x-20}{5}, & \text{se } 20 < x < 25 \\ 1, & \text{se } 25 \leq x \leq 30 \\ \frac{35-x}{5}, & \text{se } 30 < x < 35. \end{cases}$$

- Temperatura “Alta” (A_3): $\mu_{A_3}(x) = 1 - \mu_{A_1}(x) - \mu_{A_2}(x)$.

- Umidade “Baixa” (B_1):

$$\mu_{B_1}(y) = \begin{cases} 1, & \text{se } y \leq 20 \\ \frac{40-y}{20}, & \text{se } 20 < y < 40 \\ 0, & \text{se } y \geq 40 \end{cases}$$

- Umidade “Média” (B_2): $\mu_{B_2}(y) = \begin{cases} 0, & \text{se } y \leq 20 \text{ ou } y \geq 80 \\ \frac{y-20}{20}, & \text{se } 20 < y < 40 \\ 1, & \text{se } 40 \leq y \leq 60 \\ \frac{80-y}{20}, & \text{se } 60 < y < 80. \end{cases}$

- Umidade “Alta” (B_3): $\mu_{B_3}(y) = 1 - \mu_{B_1}(y) - \mu_{B_2}(y)$.

As regras fuzzy pelo método de Takagi-Sugeno são:

$$\textbf{Regra 1:} \text{ SE } (x \text{ é } A_1 \text{ E } y \text{ é } B_1) \text{ ENTÃO } z_1 = 5x + 2y,$$

$$\textbf{Regra 2:} \text{ SE } (x \text{ é } A_2 \text{ E } y \text{ é } B_2) \text{ ENTÃO } z_2 = x + y.$$

$$\textbf{Regra 3:} \text{ SE } (x \text{ é } A_3 \text{ E } y \text{ é } B_3) \text{ ENTÃO } z_3 = 2x + 3y.$$

Para as entradas $x = 35^\circ\text{C}$ e $y = 70\%$, os graus de pertinência são:

$$\mu_{A_1}(35) = \frac{35-20}{20} = 0,75, \quad \mu_{A_2}(35) = 0,25, \quad \mu_{A_3}(35) = 1 - \mu_{A_1}(35) - \mu_{A_2}(35) = 0.$$

$$\mu_{B_1}(70) = \frac{70-40}{40} = 0,75, \quad \mu_{B_2}(70) = 0,25, \quad \mu_{B_3}(70) = 1 - \mu_{B_1}(70) - \mu_{B_2}(70) = 0.$$

Os graus de ativação de cada regra, utilizando o operador do mínimo, são:

$$w_1 = \min(\mu_{A_1}(35), \mu_{B_1}(70)) = \min(0,75; 0,75) = 0,75,$$

$$w_2 = \min(\mu_{A_2}(35), \mu_{B_2}(70)) = \min(0,25; 0,25) = 0,25.$$

$$w_3 = \min(\mu_{A_3}(35), \mu_{B_3}(70)) = \min(0; 0) = 0.$$

As saídas de cada regra são:

$$z_1 = 5 \cdot 35 + 2 \cdot 70 = 175 + 140 = 315,$$

$$z_2 = 35 + 70 = 105.$$

$$z_3 = 2 \cdot 35 + 3 \cdot 70 = 70 + 210 = 280.$$

A saída final é obtida pela média ponderada:

$$\begin{aligned}
 z &= \frac{w_1 \cdot z_1 + w_2 \cdot z_2 + w_3 \cdot z_3}{w_1 + w_2 + w_3} = \frac{0,75 \times 315 + 0,25 \times 105 + 0 \times 280}{0,75 + 0,25 + 0} \\
 &= \frac{236,25 + 26,25 + 0}{1} = 262,5 \text{ RPM.}
 \end{aligned}$$

Portanto, para uma temperatura de 35 °C e umidade de 70%, o sistema Takagi-Sugeno determina que a velocidade do ventilador deve ser de 262,5 RPM, uma boa velocidade para refrescar o ambiente.

Capítulo 3

U-Net e U-Net com Sistema Adaptativo de Inferência Neuro-Fuzzy

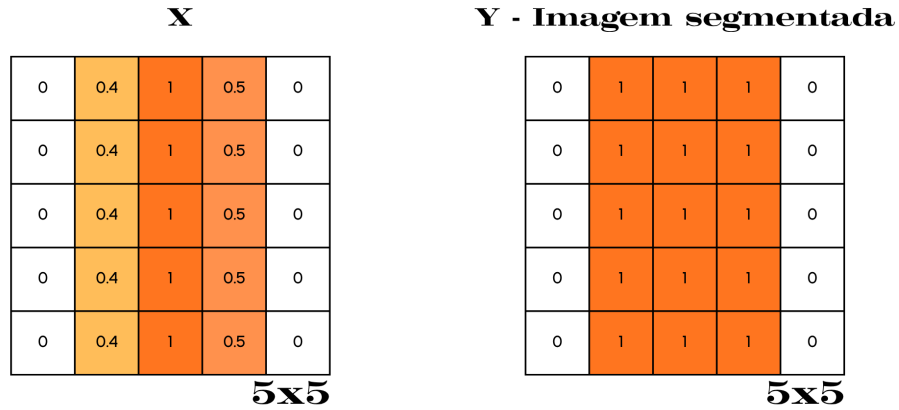
3.1 U-net: Rede Convolutacional para Segmentação de Imagens

Modificando a arquitetura tradicional de uma CNN, é possível obter variadas redes neurais com objetivos específicos. O uso típico de redes convolucionais ocorre em tarefas de classificação, em que o objetivo é atribuir uma etiqueta a uma imagem inteira. No entanto, em muitas tarefas visuais, especialmente no processamento de imagens biomédicas, a saída desejada deve incluir informações espaciais detalhadas, como a localização precisa de estruturas dentro da imagem. Para atender a essa necessidade, foi proposta a arquitetura U-Net (RONNEBERGER; FISCHER; BROX, 2015), que é uma rede neural convolutacional projetada para segmentação de imagens. A U-Net é composta por uma parte de contração (*encoder*, composta por camadas de convolução e *pooling*) e uma parte de expansão (*decoder*, composta por camadas de convolução transposta), formando uma estrutura em forma de U (origem do nome U-Net). O *encoder* é responsável por extrair características da imagem de entrada, enquanto o *decoder* reconstrói a imagem, utilizando as características extraídas pelo *encoder*. A arquitetura U-Net é especialmente eficaz em tarefas de segmentação semântica, em que o objetivo é classificar cada pixel da imagem em uma categoria específica, permitindo a identificação precisa de objetos e estruturas dentro da imagem. O Exemplo 3.1 apresenta uma simplificação visual do funcionamento da U-Net antes de ser introduzido os detalhes da arquitetura completa.

Exemplo 3.1. *Seja uma imagem de entrada normalizada X de dimensões 5×5 e uma resposta desejada Y de dimensões 5×5 , a Figura 3.1 apresenta as duas imagens, em que 1 representa a cor mais brilhante e 0 representa o fundo branco.*

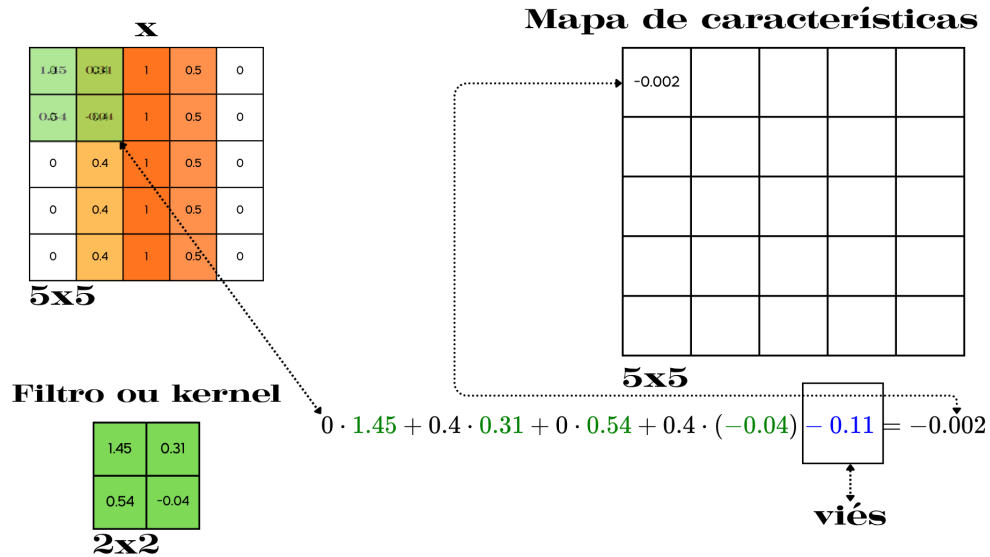
Na imagem de saída esperada, pode-se entender os valores de pixel iguais a 1 como regiões de interesse e os valores iguais a 0 como fundo. O primeiro passo para gerar a imagem de saída é aplicar a operação de convolução na imagem de entrada X utilizando um kernel que neste exemplo é uma matriz 2×2 , como mostra a Figura 3.2.

Figura 3.1: Imagem de entrada X e resposta desejada Y .



Fonte: Elaboração do Autor.

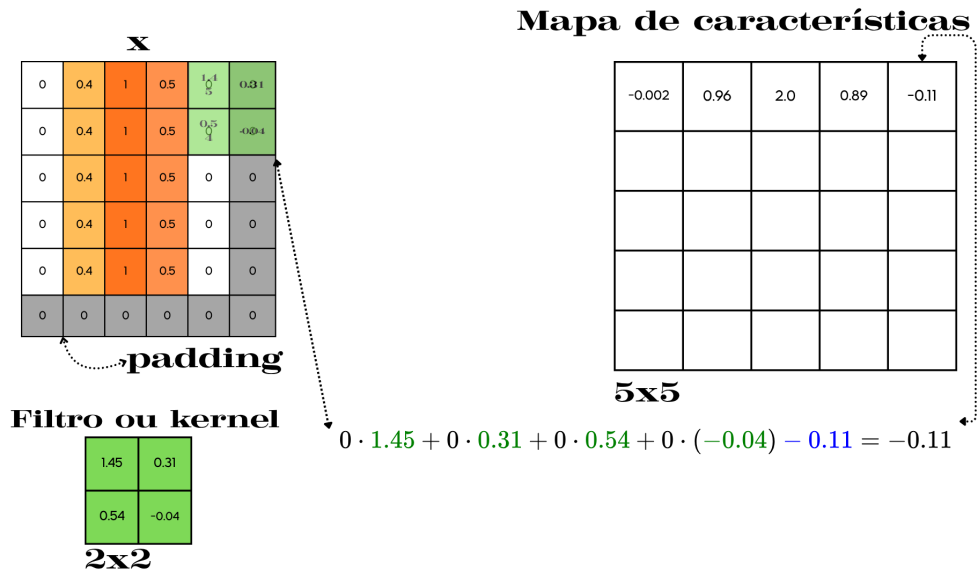
Figura 3.2: Aplicação de um *kernel* de convolução na imagem de entrada X . O resultado da convolução é um mapa de características que destaca as regiões de interesse na imagem.



Fonte: Elaboração do Autor.

Os valores do *kernel* e do *viés* que apareceram neste exemplo já foram otimizados para destacar as regiões de interesse na imagem. Para que o mapa de características resultante da convolução tenha as mesmas dimensões da imagem de entrada, é necessário aplicar a técnica de *padding*, adicionando uma borda de zeros ao redor da imagem de entrada X , como mostra a Figura 3.3.

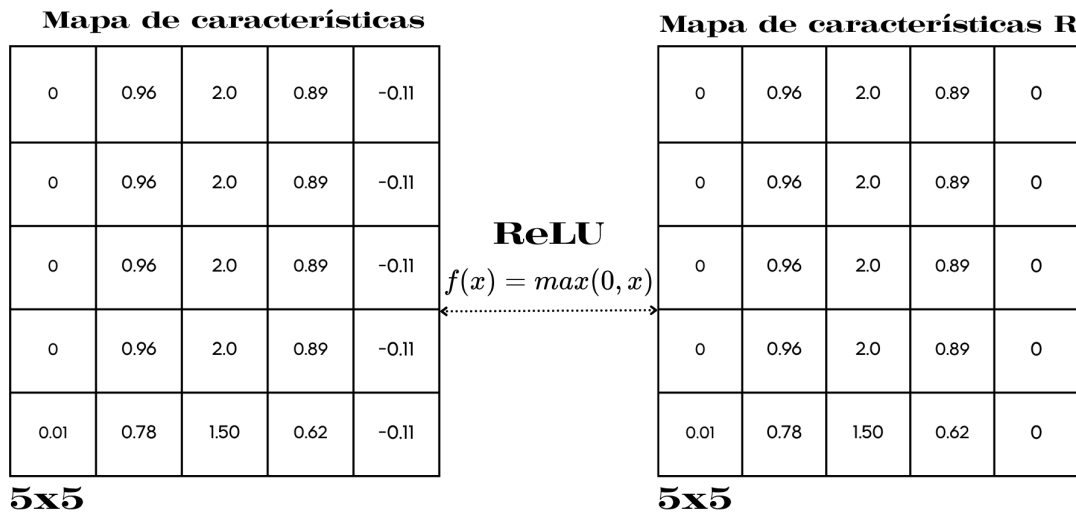
Figura 3.3: Aplicação de padding na imagem de entrada X para preservar suas dimensões durante a operação de convolução.



Fonte: Elaboração do Autor.

Após o kernel ser aplicado na imagem de entrada X com o uso de padding, é gerado um mapa de características, entretanto para completar a operação, é necessário aplicar uma função de ativação em cada valor do mapa, como mostra a Figura 3.4. A função de ativação utilizada é a função ReLU, vista na Equação 1.4.

Figura 3.4: Aplicação da função de ativação ReLU no mapa de características resultante da convolução.



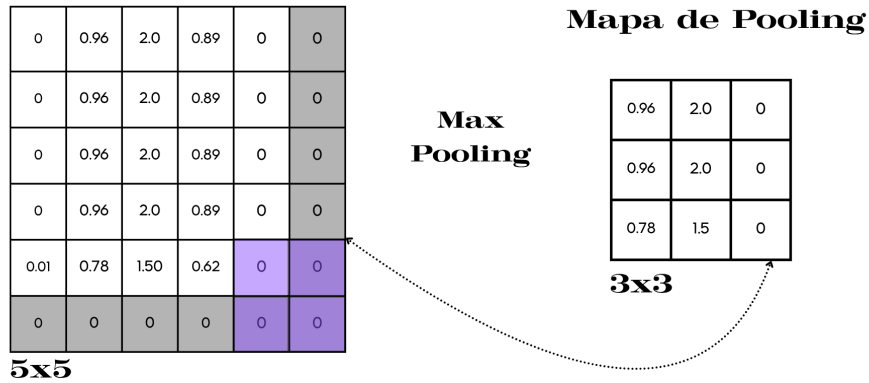
Fonte: Elaboração do Autor.

O mapa de características resultante da aplicação da função de ativação é então processado

por uma camada de *Max pooling* de dimensões 2×2 e *stride* 2, que reduz suas dimensões espaciais pela metade, como mostra a Figura 3.5.

Figura 3.5: Aplicação de uma camada de *Max pooling* no mapa de características resultante.

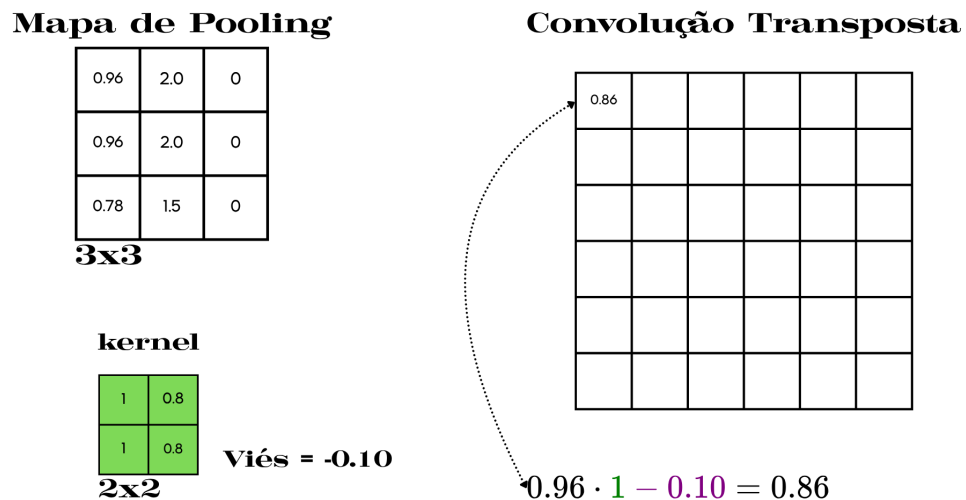
Mapa de características R



Fonte: Elaboração do Autor.

Com o mapa de pooling gerado, para voltar as dimensões originais da imagem de entrada, é necessário aplicar a operação de *convolução transposta*, utilizando um kernel de dimensões 2×2 , como mostra a Figura 3.6.

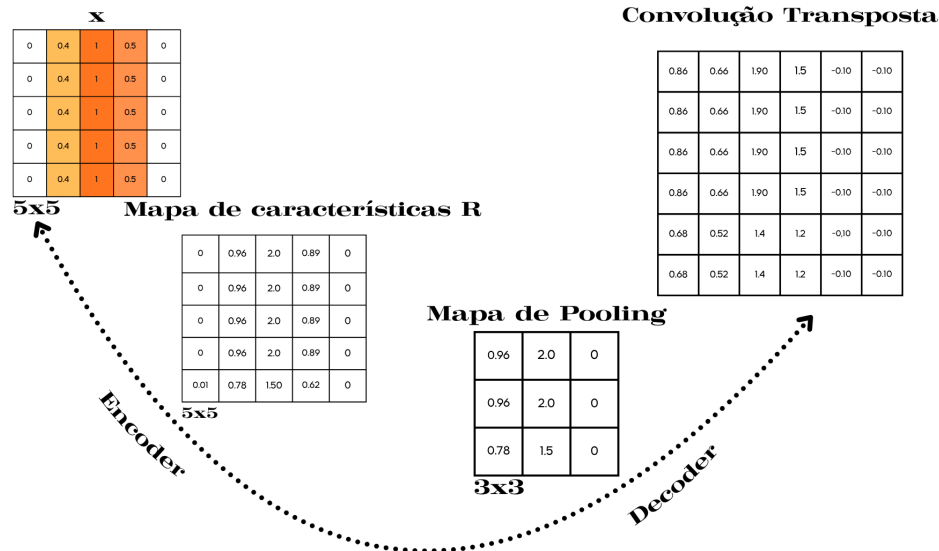
Figura 3.6: Aplicação da operação de convolução transposta no mapa de *pooling* para expandir suas dimensões espaciais e voltar às dimensões originais da imagem de entrada.



Fonte: Elaboração do Autor.

O nome *U-Net* vem da forma em U da arquitetura, em que o encoder diminui as dimensões espaciais da imagem de entrada, enquanto o decoder as expande, formando uma estrutura simétrica, como mostra a Figura 3.7.

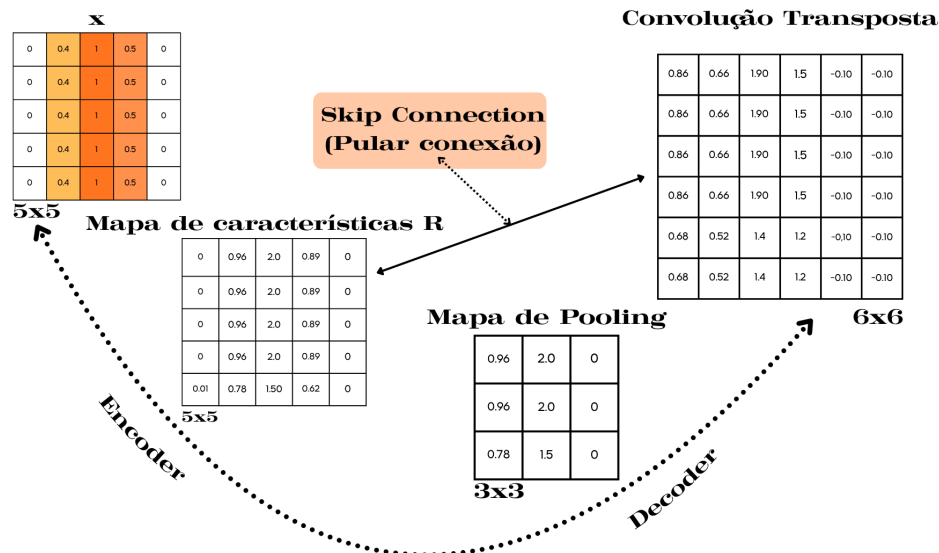
Figura 3.7: Estrutura em forma de U da arquitetura U-Net. O *encoder* é responsável por extrair características da imagem de entrada, enquanto o *decoder* reconstrói a imagem, utilizando as características extraídas pelo *encoder*.



Fonte: Elaboração do Autor.

A característica mais importante da arquitetura U-Net é a presença de conexões de atalho (*skip connections*) entre as camadas correspondentes do encoder e do decoder, presente na Figura 3.8. Essa operação concatena as imagens correspondentes, entretanto para que as ima-

Figura 3.8: Exemplo de conexões de atalho (*skip connections*) na arquitetura U-Net.

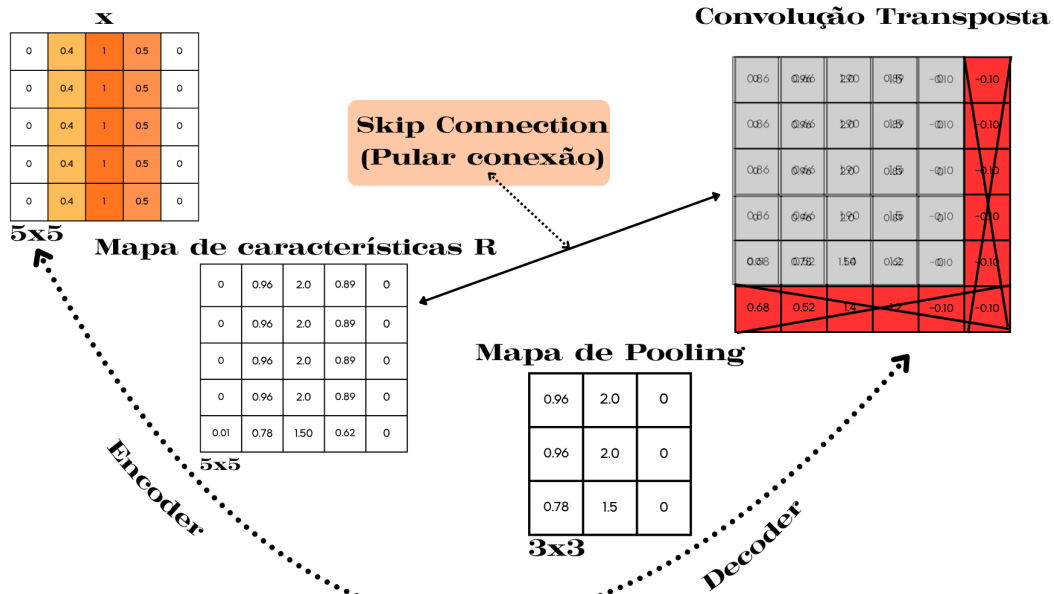


Fonte: Elaboração do Autor.

gens possam ser concatenadas, é necessário que elas tenham as mesmas dimensões, portanto, é necessário aplicar a técnica de *copy (copiar) and (e) crop (cortar)*, em que as características extraídas pelo decoder são copiadas e recortadas para se encaixar nas dimensões da camada correspondente no encoder, a sobreposição é feita começando de cima para baixo, da esquerda para

a direita, com o vermelho sinalizando a área que será removida após a sobreposição, (Figura 3.9).

Figura 3.9: Exemplo de conexões de atalho (*skip connections*) na arquitetura U-Net. As conexões de atalho permitem que as informações de baixo nível, como bordas e texturas, sejam diretamente transferidas do *encoder* para o *decoder*, facilitando a reconstrução precisa da imagem segmentada.



Fonte: Elaboração do Autor.

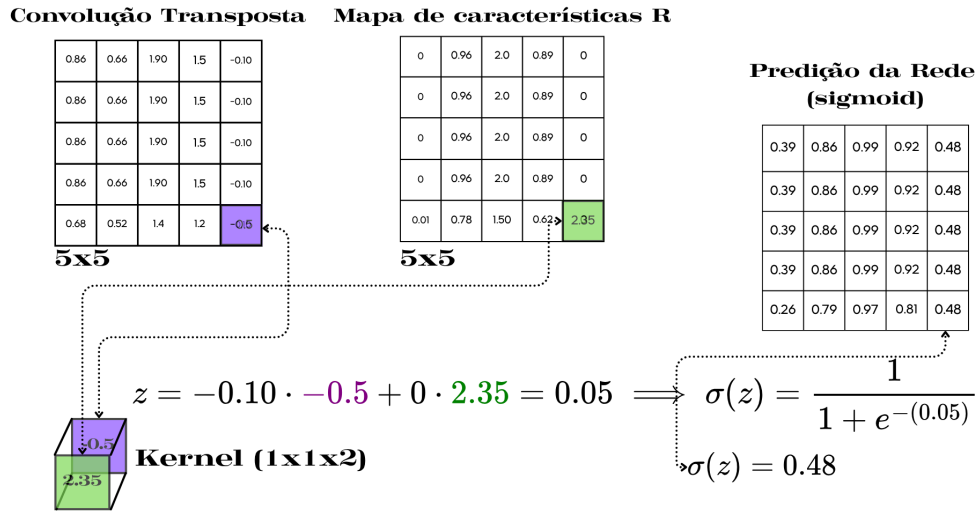
Dependendo da convolução utilizada, o *copy and crop* pode ser aplicado a imagem do encoder e caso as imagens tenham mais de um canal, a operação de concatenação leva em consideração apenas a altura e a largura das imagens, ou seja, as imagens precisam ter as mesmas dimensões, os canais de cada imagem são somados no processo, por exemplo:

$$X_1 = (A, L, C_1) \oplus X_2 = (A, L, C_2) = (A, L, C_1 + C_2) = Y,$$

em que A é a altura, L é a largura e C é a quantidade de canais de cada imagem e $\oplus$ representa a operação de concatenação ao longo do eixo dos canais.

Para finalizar a concatenação, é necessário aplicar uma última camada de convolução. Como as imagens estão sobrepostas, por conta da concatenação, o kernel fica com dimensões $(1 \times 1 \times 2)$, um canal para cada imagem, como mostra a Figura 3.10. No fim da convolução, é necessário aplicar uma função de ativação, neste caso a sigmoid logística vista na Equação 1.3, para gerar a imagem de saída Y com valores entre 0 e 1, representando a probabilidade de cada pixel pertencer à classe segmentada. Neste exemplo, quanto mais próximo de 1, mais brilhante é o pixel, indicando uma maior probabilidade de não estar no plano de fundo da imagem, enquanto valores próximos de 0 indicam uma maior probabilidade de pertecerem ao plano de fundo.

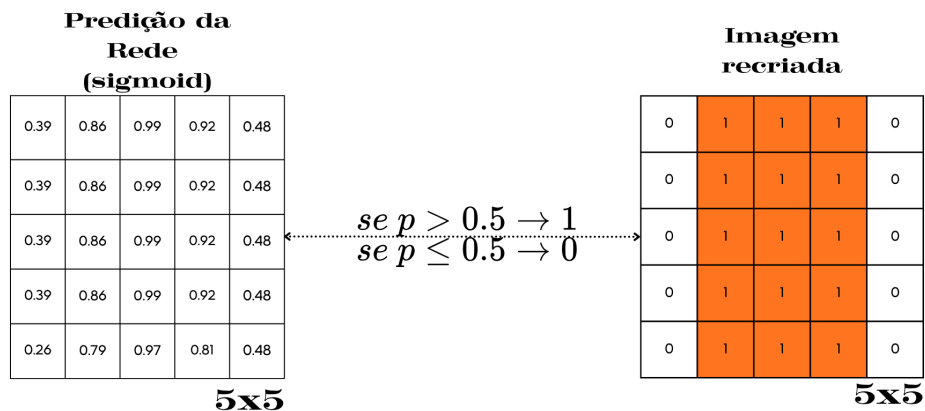
Figura 3.10: Aplicação de uma última camada de convolução para gerar a imagem de saída Y . O *kernel* utilizado tem dimensões $(1 \times 1 \times 2)$, um canal para cada imagem resultante da concatenação.



Fonte: Elaboração do Autor.

Por fim, como a imagem segmentada, presente na Figura 3.1, tem valores 0 e 1, é necessário aplicar um limiar (*threshold*) na imagem gerada após a aplicação da função sigmoid. Como mostra a Figura 3.11.

Figura 3.11: Aplicação de um limiar (*threshold*) para gerar a imagem binária final. A imagem segmentada resultante da aplicação da função de ativação sigmoid tem valores entre 0 e 1, representando a probabilidade de cada pixel pertencer à classe segmentada. O limiar é aplicado para converter esses valores em uma imagem binária, onde pixels com valores acima do limiar são classificados como pertencentes à classe segmentada (1), enquanto pixels com valores abaixo do limiar são classificados como pertencentes ao plano de fundo (0).



Fonte: Elaboração do Autor.

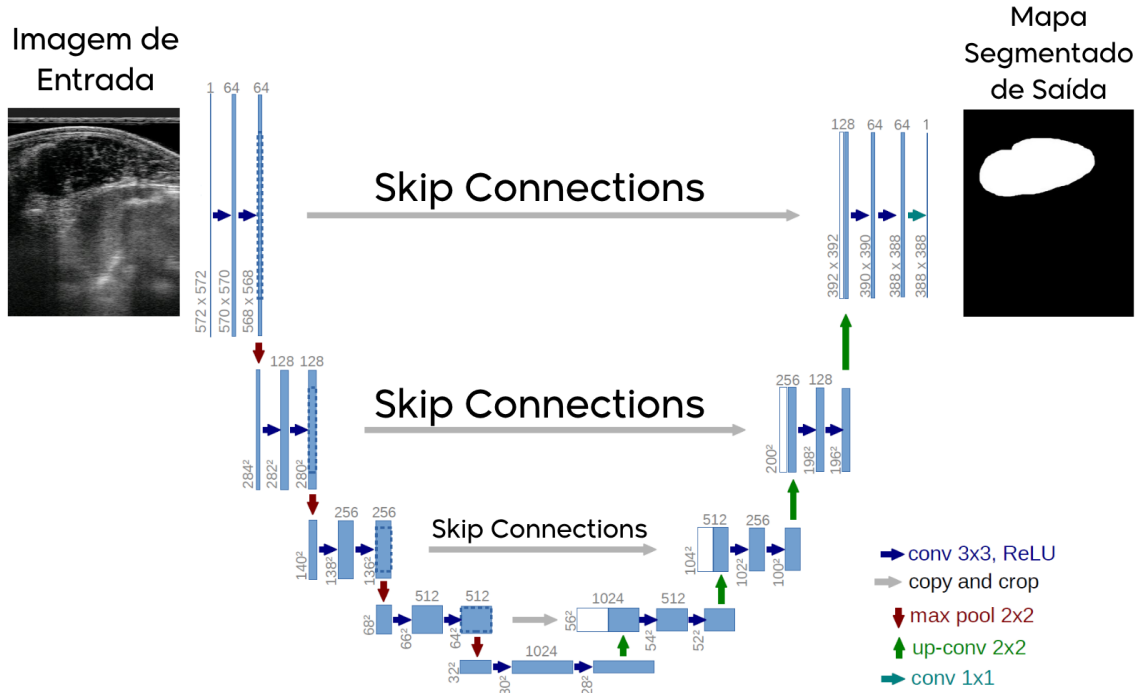
3.1.1 U-Net

A rede convolucional desenvolvida por Ronneberger (RONNEBERGER; FISCHER; BROX, 2015) é bem similar ao Exemplo 3.1, entretanto, a arquitetura completa da U-Net é composta por múltiplas camadas de convolução e *pooling* no *encoder*, e múltiplas camadas de convolução transposta no *decoder*, no total são quatro camadas de convolução e *pooling* no *encoder* e quatro camadas de convolução transposta no *decoder*, como mostra a Figura 3.12. O número de filtros em cada camada de convolução aumenta à medida que se avança no *encoder*, começando com 64 filtros na primeira camada e dobrando a quantidade a cada camada subsequente, chegando a 1024 filtros na última camada do *encoder*. No *decoder*, o número de filtros é reduzido pela metade a cada camada subsequente, começando com 512 filtros na primeira camada do *decoder* e terminando com 64 filtros na última camada antes da saída. As conexões de atalho (*skip connections*) entre as camadas correspondentes do *encoder* e do *decoder* permitem que sejam feitas as concatenações das imagens. A Figura 3.12 apresenta a arquitetura completa da U-Net, cada bloco representa uma etapa de processamento, o número na parte superior do bloco indica o número de filtros utilizados na camada de convolução, enquanto o número na parte inferior do bloco indica as dimensões espaciais da imagem resultante daquela etapa. A seta azul escuro representa a operação de convolução com filtros de dimensão 3×3 e função de ativação *ReLU*, a seta vermelha representa a operação de *pooling* com *kernel* de tamanho 2×2 e stride de 2, a seta cinza representa a operação de *Copy and Crop* para realizar a concatenação das imagens, e a seta verde representa a operação de convolução transposta com filtros de dimensão 2×2 . A última camada de convolução utiliza um *kernel* de dimensão 1×1 e função de ativação *sigmoid logística* (dependendo da aplicação) para gerar a imagem de saída segmentada. Como não é utilizado nenhum processo de *padding* as dimensões da imagem são reduzidas em cada etapa de convolução. O corte feito para a concatenação (sinalizado por uma linha azul pontilhada) nas imagens do *encoder* (lado direito da Figura 3.12) é feito no centro da imagem, ou seja, a parte central da imagem do *encoder* é mantida para a concatenação, enquanto as bordas são removidas. O total de parâmetros a serem otimizados é de aproximadamente 27.548,993 dependendo da utilização do viés ou não. Para as duas primeiras camadas de convolução do *encoder* o número de parâmetros pode ser calculado da seguinte forma:

$$\begin{aligned} 1^{\text{a}} \text{ convolução} &= ((3 \times 3 \times 1) + 1) \times 64 = 640 \\ 2^{\text{a}} \text{ convolução} &= ((3 \times 3 \times 64) + 1) \times 64 = 36.928. \end{aligned}$$

O filtro da segunda convolução tem 64 canais, um para cada mapa de características gerado pela primeira convolução, multiplica-se novamente por 64, pois cada filtro gera apenas um mapa de características.

Figura 3.12: Arquitetura completa da U-Net.



Fonte: Adaptado de (RONNEBERGER; FISCHER; BROX, 2015).

3.1.2 Dice

Quando a região de interesse em uma imagem é pequena em comparação ao plano de fundo, a métrica da acurácia pode ser enganosa, pois a maioria dos pixels da imagem pertencem ao plano de fundo, o que pode levar a um resultado de alta acurácia mesmo que o modelo não esteja segmentando corretamente a região de interesse. Para lidar com esse problema, é comum utilizar métricas específicas para avaliação de segmentação, como o coeficiente de Dice (HARISHA, 2023), que é uma medida de similaridade entre a imagem segmentada e a imagem de referência (*ground truth* ou máscara). O coeficiente de Dice é calculado utilizando a seguinte fórmula:

$$Dice = \frac{2|A \cap B|}{|A| + |B|}, \quad (3.1)$$

em que A é o conjunto de pixels da imagem segmentada e B é o conjunto de pixels da imagem de referência. O coeficiente de Dice varia entre 0 e 1, onde um valor de 1 indica uma segmentação perfeita (todas as regiões de interesse foram corretamente identificadas) e um valor de 0 indica que não há sobreposição entre a imagem segmentada e a imagem de referência.

Exemplo 3.2. Seja $A = \{1, 2, 3, 4, 5, 6\}$ e $B = \{1, 2, 3, 4, 9\}$. A interseção entre os conjuntos A e B é $A \cap B = \{1, 2, 3, 4\}$, o que significa que existem 4 elementos em comum entre os dois conjuntos. O número total de elementos em A é $|A| = 6$ e o número total de elementos em B

é $|B| = 5$. Substituindo esses valores na fórmula do coeficiente de Dice, temos:

$$Dice = \frac{2|A \cap B|}{|A| + |B|} = \frac{2 \times 4}{6 + 5} = \frac{8}{11} \approx 0,727.$$

Portanto, o coeficiente de Dice para os conjuntos A e B é aproximadamente 0,727, indicando uma boa sobreposição entre os dois conjuntos, mas não uma correspondência perfeita.

A métrica Dice é bastante utilizada em conjunto com a arquitetura U-Net para avaliar a qualidade da segmentação gerada pela rede. Neste trabalho esta será amplamente utilizada.

3.1.3 U-Net *Multitask* com Inferência Fuzzy

Como mostrado na seção 3.1 do capítulo 1, a U-Net é uma arquitetura de rede neural convolucional projetada para tarefas de segmentação de imagens, especialmente em contextos biomédicos. Para melhorar a capacidade de segmentação, pode-se fazer modificações na arquitetura original. A modificação proposta neste trabalho acontece na camada de *baseline* ou *bottleneck*, localizada no final do *encoder*. As principais modificações na arquitetura da U-Net são as seguintes:

- U-Net *Multitask*: Esta modificação envolve a adição de uma nova camada de classificação à arquitetura original da U-Net, permitindo que a rede aprenda a realizar tanto a segmentação quanto a classificação simultaneamente. A camada de classificação é composta por unidades adicionais que processam as características extraídas pela U-Net para produzir uma saída de classificação (ZHU; ROHLING; SALCUDEAN, 2022). Um exemplo de aplicação é a segmentação de imagens de cancer de mama feitas por ultrassom, em que a U-Net é utilizada para segmentar a região de interesse (tumor) e a camada de classificação é utilizada para classificar o tipo do tumor (benigno (0) ou maligno (1)), a entrada da rede seria (IMG, [0,1]), em que IMG representa a imagem de entrada e [0,1] representa os rótulos de classificação. A segunda entrada da rede geralmente está em *OneHotEncoder* (Data Science Academy, 2025) que é uma técnica de codificação de rótulos categóricos em vetores binários, no caso do exemplo, benigno seria codificado para [1, 0] e [0, 1] para maligno. O funcionamento da camada pode-se ser entendido da seguinte forma: No final do *encoder* a rede gera uma imagem (B, H, W, C) (*baseline*), em que B é o tamanho do lote, H e W são as dimensões da imagem e C é o número de canais. Caso a entrada da rede seja (IMG, $[x_0, \dots, x_n]$), para $x_k = 0$ e $x_j = 1$, $k \neq j$ e $j = 0, \dots, n$. É feita a operação.

$$\sum_{i=0}^C \tau \left(\sum_{j=0}^n x_j \times w_i \right) = [\hat{y}_0, \hat{y}_1, \dots, \hat{y}_C]_{1 \times C},$$

em que w_i são os pesos treináveis da camada de classificação e τ é a função de ativação utilizada. A saída da camada de classificação é um vetor de tamanho C , em que cada elemento $\hat{y}_i$ representa a probabilidade da imagem pertencer à classe i . Para finalizar, a imagem (B, H, W, C) é multiplicada pelo vetor de classificação $[\hat{y}_0, \hat{y}_1, \dots, \hat{y}_C]_{1 \times C}$ para

obter uma nova imagem (B^*, H^*, W^*, C^*) , em que cada canal é ponderado pela probabilidade da classe correspondente. Após esse processo a imagem é enviada para o *decoder* da U-Net para obter a segmentação final ou, no caso deste trabalho, é enviado para a camada de inferência fuzzy, como mostrado na Figura 3.14.

- Camada de Inferência Fuzzy: Esta camada é responsável por aplicar regras fuzzy para refinar as previsões da U-Net (VERMA *et al.*, 2026). Uma das técnicas utilizadas na maioria das redes neurais é a adição de um *threshold* (limite) na saída de cada camada ou na saída final da rede, ou seja, se a rede tem como saída uma imagem com pixels pertencentes ao intervalo $[0, 1]$, para criar uma mapa de segmentação, cada pixel deve satisfazer a condição:

$$\text{pixel} = \begin{cases} 1, & \text{se pixel} \geq \text{threshold} \\ 0, & \text{caso contrário} \end{cases}$$

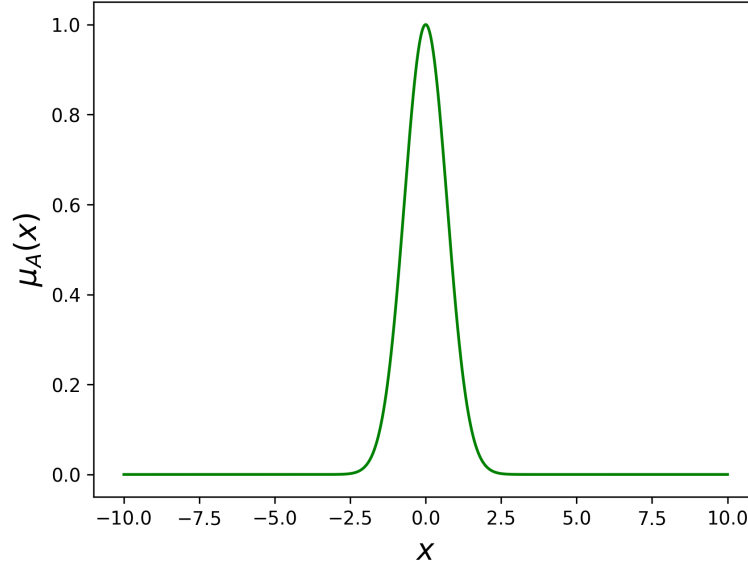
A adição de uma camada fuzzy adiciona incerteza e flexibilidade ao processo de segmentação, permitindo que a rede aprenda a lidar com casos ambíguos ou ruidosos, como por exemplo um pixel que está próximo ao limite entre duas classes. A camada de inferência fuzzy pode ser implementada utilizando o método de Takagi-Sugeno, já que em grande parte, o consequente pode ser modelado como uma função linear das entradas. Assim como o método anterior, a camada fuzzy pode ser aplicada no *baseline* da U-Net, permitindo que as regras fuzzy sejam aplicadas às características extraídas pelo *encoder* antes de serem processadas pelo *decoder*. O processo pode ser esquematizado da seguinte forma:

- Fuzzificação: O *baseline* é tem dimensões (B, H, W, C) em que B é o tamanho do lote, H e W são as dimensões da imagem e C é o número de canais. Cada pixel do *baseline* receberá um grau de pertinência para cada conjunto fuzzy definido previamente (ao inicializar a rede), ou seja, as dimensões do *baseline* serão transformadas para (B, H, W, C, F) , em que F é o número de conjuntos fuzzy definidos para cada pixel. As funções de pertinência são gaussianas:

$$\mu_A(x) = e^{-\frac{(x-c)^2}{2\sigma^2}},$$

em que c é o centro da função de pertinência e σ é o desvio padrão, controlando a largura da função, a Figura 3.13 mostra a função de pertinência com centro em $c = 0$ e largura $\sigma = 1$.

Figura 3.13: Função de pertinência gaussiana com centro em $c = 0$ e largura $\sigma = 1$.



Fonte: Elaboração do Autor.

Cada pixel terá um total de três c e um σ que serão treináveis utilizando o método de *backpropagation*.

- Inferência: As regras fuzzy (a quantidade de regras é definida na inicialização da rede) são aplicadas utilizando o método de Takagi-Sugeno. Como cada pixel do *baseline* tem três valores (funções de pertinência) para cada conjunto fuzzy em cada canal, então $(B, H, W, C, F) \rightarrow (B, H, W, C \times F)$ com isso, uma matriz de regras é definida, sendo a quantidade de colunas igual a quantidade de regras pré-definidas e a quantidade de linhas igual a $C \times F$. Cada elemento da matriz de regras é um peso treinável que indica a importância de cada conjunto fuzzy para cada regra. O cálculo de cada regra é dado por:

$$\text{Regra}_r = \tau \left(\sum_{i=0}^{C \times F} \mu_{A_i}(x) \times w_{i,r} \right),$$

em que τ é a função de ativação utilizada, $w_{i,r}$ é o peso treinável da regra r para o conjunto fuzzy A_i e $\mu_{A_i}(x_j)$ é o grau de pertinência do pixel x_j ao conjunto fuzzy A_i , para $j = 0, \dots, C \times F - 1$ (x_j é um elemento do vetor x , com x sendo um pixel da imagem $(H, W, C \times F)$ o tamanho de x é $C \times F$). A saída da inferência é uma nova imagem (B, H, W, R) , em que R é o número de regras pré-definidas.

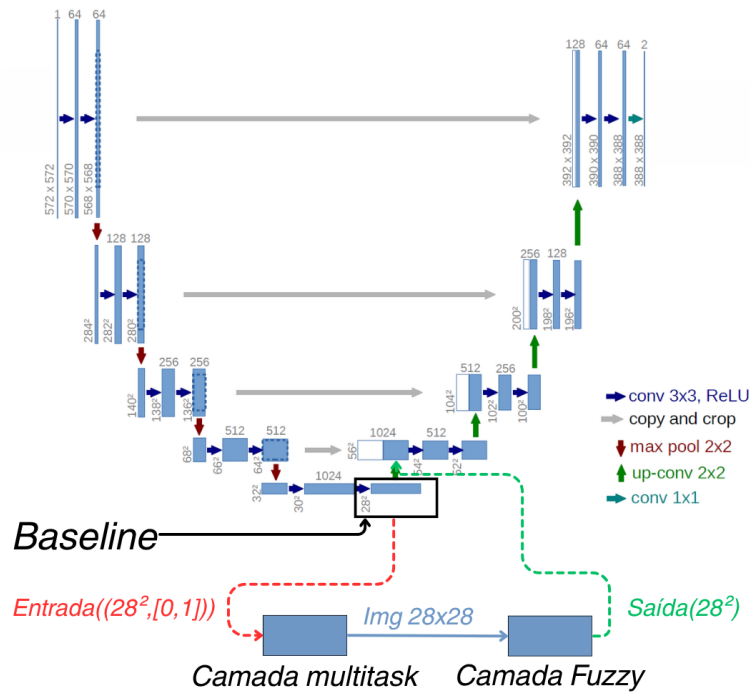
- Defuzzificação: Para obter a imagem final a ser processada pelo *decoder* da U-Net (já que $(B, H, W, C) \neq (B, H, W, R)$), é necessário realizar a defuzzificação. O método de defuzzificação utilizado é o método da média ponderada. A defuzzificação é dada por:

$$\text{Pixel}_{\text{defuzzificado}} = \frac{\sum_{r=0}^R \text{Regra}_r \times z_r}{\sum_{r=0}^R \text{Regra}_r},$$

em que Regra_r é o valor da regra r obtido na etapa de inferência e z_r é o valor associado à regra r , que pode ser um vetor de pesos treináveis ou uma função das entradas. Para que a saída tenha dimensões (B, H, W, C) , a matriz dos pesos (Z) deve ter dimensões (R, C) . A saída da defuzzificação é uma nova imagem (B, H_f, W_f, C) , em que $H_f = H$ e $W_f = W$, por conta do treinamento, inicialmente, a imagem de saída da defuzzificação é só ruído e ao decorrer do treinamento esse ruído vai diminuindo, para suprimir os efeitos do ruído, é somada a imagem de saída da defuzzificação com a imagem do *baseline* para obter uma nova imagem $(B, H + H_f, W + W_f, C)$.

Após essa etapa a imagem é processada pelo *decoder* da U-Net para obter a segmentação final. A figura 3.14 ilustra a arquitetura da U-Net com a adição de uma camada *Multitask* e uma camada de inferência fuzzy.

Figura 3.14: Arquitetura da U-Net com a adição de uma camada *Multitask* e uma camada de inferência fuzzy.



Fonte: Adaptado de (RONNEBERGER; FISCHER; BROX, 2015).

As setas mostram o fluxo do *baseline* pelas camadas adicionais, começando da seta vermelha e terminando na seta verde.

Capítulo 4

Segmentação de Imagens de Ultrassom Bovino: Metodologia e Resultados

Neste capítulo serão desenvolvidos os principais conceitos abordados nos capítulos anteriores. Primeiramente, será feita uma descrição detalhada do problema, incluindo as características das raças Nelore e Caracu, as medidas de interesse (AOL, EGL e EGG), a utilização do TensorFlow/Keras para a implementação do modelo, o processo de aquisição e pré-processamento das imagens de ultrassom e a criação das máscaras de segmentação. Em seguida, serão feitas comparações entre os modelos treinados, utilizando métricas de avaliação adequadas para a tarefa de segmentação de imagens. Duas redes serão implementadas, uma U-Net tradicional e uma U-Net com a integração do sistema de inferência neuro-fuzzy adaptativo (ANFIS), para avaliar o impacto da integração do ANFIS na performance do modelo.

4.1 Descrição do Problema

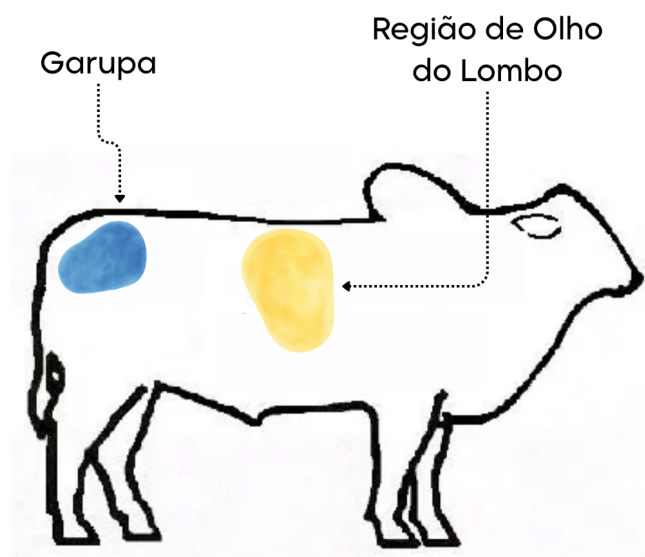
4.1.1 Raças Nelore e Caracu

A raça Nelore é originária da Índia, onde seus ancestrais, os bovinos da raças Ongole, eram criados há milênios por sua rusticidade, resistência a climas tropicais e capacidade de adaptação a ambientes variados, esses animais foram trazidos ao Brasil no final do século XIX, em um momento de expansão da pecuária nacional (ZANCANER, 2026). A raça Caracu, por sua vez, tem origem na região da Península Ibérica, chegando no Brasil logo após o descobrimento, no tempo do Brasil Colônia, os animais da raça Caracu são extremamente funcionais, com alta adaptabilidade e rusticidade, além de serem resistentes a doenças e parasitas (ABCCARACU, 2022). O Brasil é um dos maiores criadores mundiais de bovinos da raça Nelore, com uma população estimada em mais de 230 milhões de cabeças (LAMAS, 2023), enquanto a raça Caracu é menos numerosa, mas ainda assim desempenha um papel importante na pecuária brasileira, especialmente em regiões onde a resistência a condições adversas é crucial. Uma das principais formas de avaliar o desenvolvimento e a qualidade fenotípica dos animais é por meio da análise de imagens de ultrassom em que, entre outras, é possível obter informações da

carcaça do animal. Entre os recursos e metodologias existentes para avaliação de carcaças e de características associadas à qualidade da carne, a ultrassonografia se destaca, especialmente, por possibilitar o exame *in vivo*, uma única vez na vida do animal (SUGUISAWA; MATOS; SUGUISAWA, 2013), em tempo relativamente curto, com boa precisão a custos relativamente baixos. O objetivo é conhecer o potencial dos indivíduos e do rebanho como um todo, para musculosidade, precocidade de acabamento e qualidade da carne.

A avaliação é de extrema importância pois existe uma variabilidade nas características que estão relacionadas com a qualidade de desossa, que por sua vez influenciam a comercialização e os resultados econômicos dos produtores (FELICIO, 2010). Dependendo da região do corpo do animal onde a imagem de ultrassom é obtida, é possível extrair diferentes informações, neste trabalho serão utilizadas imagens de ultrassom da região do olho de lombo, que localiza-se entre a 12^a e 13^a costela (WILSON, 1998) e da garupa, como mostra a Figura 4.1.

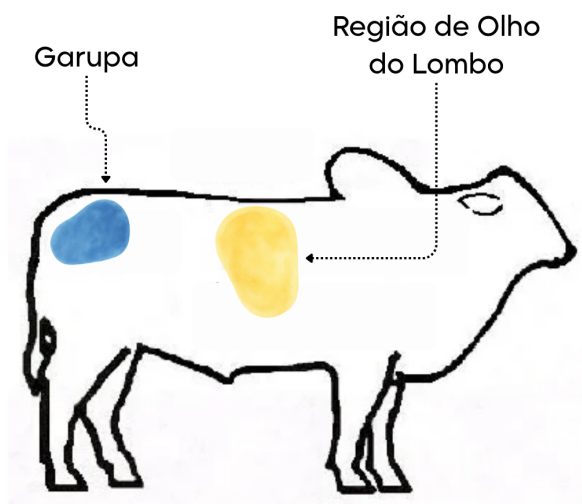
Figura 4.1: Região do olho de lombo (em amarelo), localizada entre a 12^a e 13^a costela e da garupa (em azul).



Fonte: Adaptado de (WILSON, 1998).

Na região de olho do lombo é possível obter informações de duas medidas importantes para a avaliação da carcaça, a Área de Olho de Lombo (AOL) e a Espessura de Gordura no Lombo (EGL). Na Garupa é possível obter a Espessura de Gordura na Garupa (EGG). As imagens de ultrassom foram obtidas por meio do aparelho de ultrassom Aloka SSD-500, com todos os animais sendo machos por volta de um ano de idade e estando vivos, localizados no Centro Avançado de Pesquisa Tecnológica do Agronegócio em Setãozinho-SP. A Figura 4.2(a) mostra a imagem do especialista obtendo a imagem de ultrassom da região, já a Figura 4.2(b) mostra a imagem de ultrassom, a Figura 4.2(c) mostra a imagem de ultrassom da garupa e a Figura 4.2(d) mostra o especialista obtendo a imagem de ultrassom da garupa. **esperando elas mandarem as imagens**

Figura 4.2: Aquisição da imagem de ultrassom da região do olho de lombo.



(a) Especialista obtendo a imagem de ultrassom da região do olho de lombo.



(b) Imagem de ultrassom da região do olho de lombo.



(c) Imagem de ultrassom da garupa.



(d) Especialista obtendo a imagem de ultrassom da garupa.

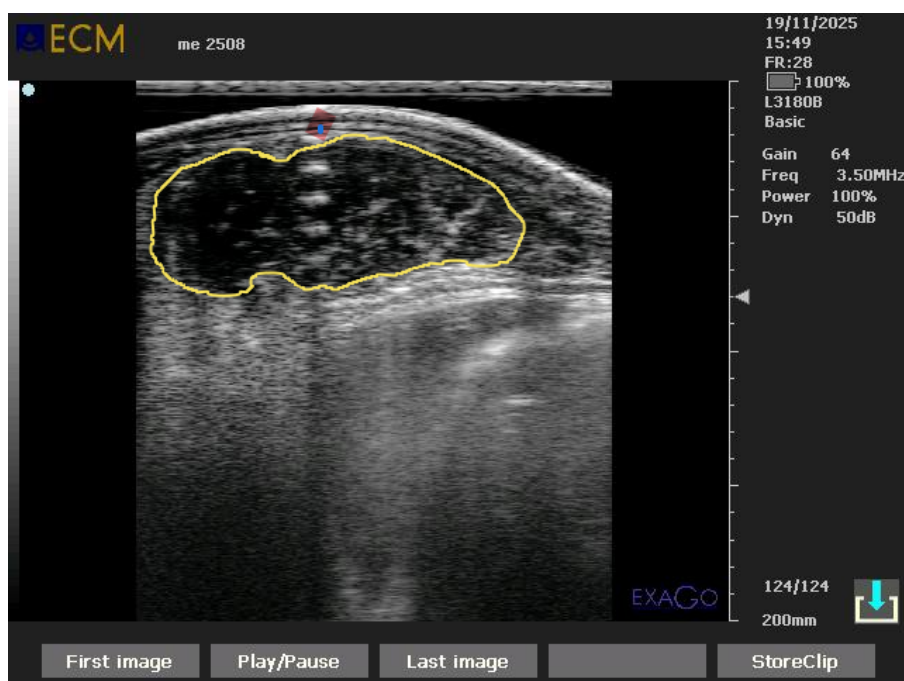
Fonte: Elaboração do Autor.

Colocar mais informações

4.1.2 Área de Olho de Lombo (AOL), Espessura de Gordura no Lombo (EGL) e Espessura de Gordura na Garupa (EGG)

A área de olho de lombo é uma medida que representa a área da seção transversal do músculo *Longissimus dorsi* (Contrafilé), medida em centímetro quadrados entre a 12^a e 13^a costela. A área está altamente correlacionada com o produto final. Essa característica tem herdabilidade moderada de 35% e indica musculosidade e rendimento da carcaça (WILSON, 1998), uma maior área é preferível pelos produtores. Do ponto de vista produtivo, pode-se afirmar que animais com valores de AOL superiores a 75 cm², ao abate, apresentam elevados rendimentos de cortes cárneos na indústria frigorífica (SUGUISAWA; MATOS; SUGUISAWA, 2013). A Figura 4.3 mostra a demarcação manual da área feita no aplicativo *ImageJ* (National Institutes of Health, 2026) em que a região interna a demarcação amarela é a AOL.

Figura 4.3: Demarcação manual da Área de Olho de Lombo (AOL) utilizando o aplicativo *ImageJ*.

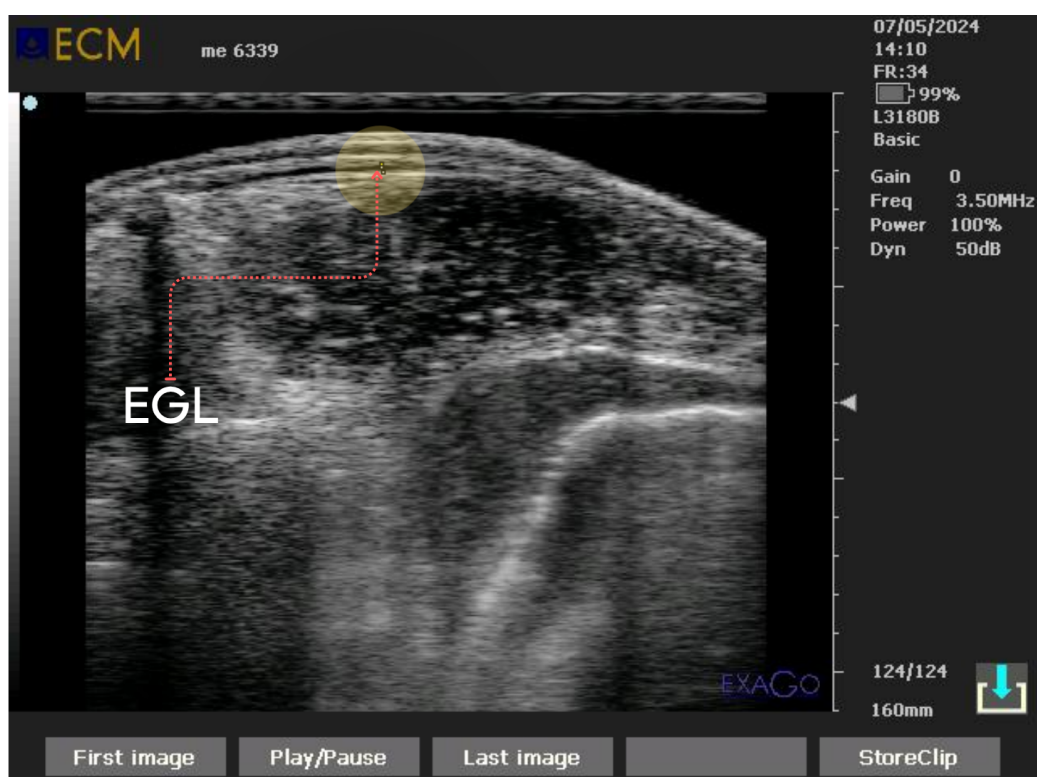


Fonte: Elaboração do Autor.

A espessura de gordura no lombo (EGL) é uma medida que representa a espessura da camada de gordura sobre o músculo *Longissimus dorsi* (Contrafilé), sendo sua espessura (EGL) medida, em milímetros, a 3/4 da distância medial do músculo, entre a 12^a e 13^a costela. Esta gordura de cobertura é extremamente importante para a proteção da carcaça contra a rápida e intensa queda de temperatura nas câmaras frias, que pode provocar o endurecimento (perda em maciez de até 5 vezes) e o escurecimento da carne em carcaças pobres em acabamento (SUGUISAWA; MATOS; SUGUISAWA, 2013), a Figura 4.3, mostra a EGL em azul dentro

da demarcação vermelha. Do ponto de vista de melhoramento genético, a característica da EGL é um importante indicativo da precocidade sexual e de terminação, ou seja, animais que iniciam a deposição de gordura mais cedo tendem a ser mais precoces, sexualmente, e tendem a apresentar carcaças prontas para o abate em menores idades. Vale ressaltar ainda que EGL apresenta características antagônicas à musculosidade (AOL) e tamanho, ou seja, a seleção exclusiva para precocidade de terminação, implicará na produção de animais com alta deposição de gordura, mas com menores proporções de cortes cárneos na carcaça e menores pesos ao abate e a idade adulta (SUGUISAWA; MATOS; SUGUISAWA, 2013). A Figura 4.4 mostra a demarcação manual feita no aplicativo *ImageJ* em que a linha amarela dentro do círculo amarelo representa a demarcação da EGL.

Figura 4.4: Demarcação manual da Espessura de Gordura no Lombo (EGL) utilizando o aplicativo *ImageJ*.

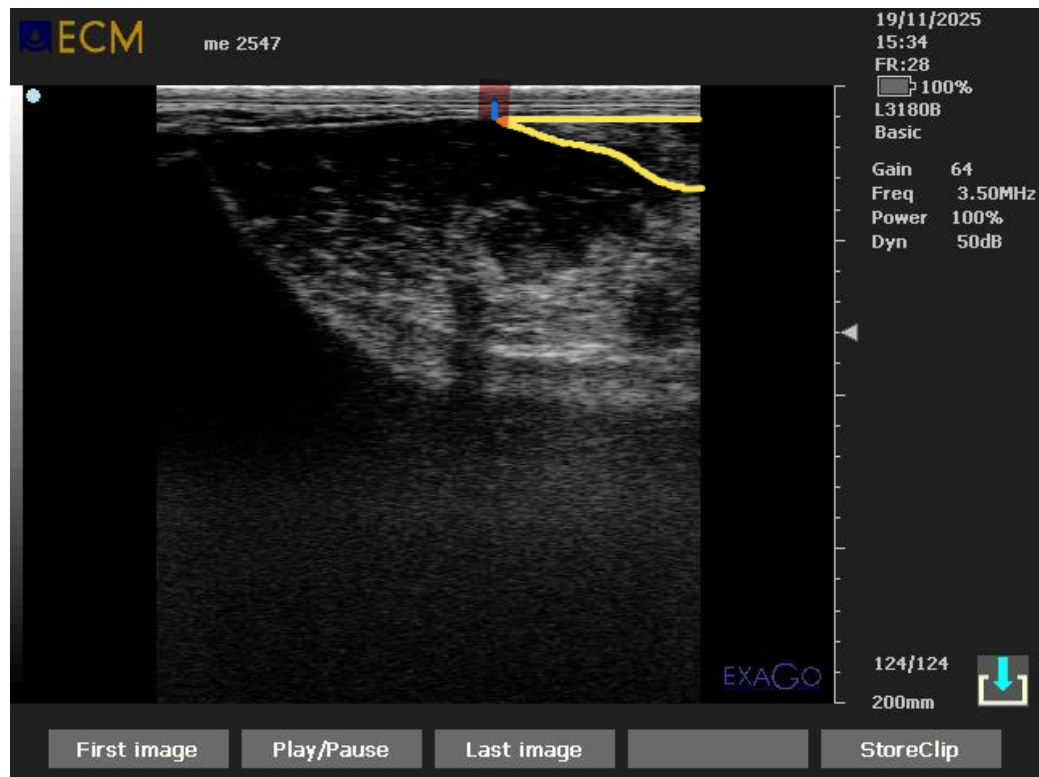


Fonte: Elaboração do Autor.

A espessura de gordura na garupa (EGG) é uma medida que representa a espessura da camada de gordura sobre a região da garupa, sendo medida em milímetros. A EGG é uma medida complementar à EGL, indicada principalmente para situações em que os animais têm a deposição de gordura subcutânea comprometida, principalmente por nutrição inadequada, como pode ocorrer, por exemplo, com animais em sistemas extensivos de pastejo. Como o processo de deposição de gordura de acabamento inicia-se das extremidades para o centro do corpo, os valores de gordura encontrados na garupa serão superiores e, conseqüentemente, minimizam possíveis erros de interpretação dado a baixa variação da medida (SUGUISAWA; MATOS; SUGUISAWA, 2013). A medida é obtida observando o músculo *Biceps Femoris*, comumente

chamado de ponta da picanha. A Figura 4.5 mostra a demarcação manual feita no aplicativo *ImageJ* em que a linha amarela dentro do círculo amarelo representa a demarcação da EGG.

Figura 4.5: Demarcação manual da Espessura de Gordura na Garupa (EGG) utilizando o aplicativo *ImageJ*.



Fonte: Elaboração do Autor.

A espessura é obtida medindo a altura da gordura que encontra-se no começo do *Biceps Femoris* (da esquerda para a direita) (WILSON, 1998).

4.1.3 Tensorflow/Keras

Para desenvolver os modelos, algumas bibliotecas foram utilizadas, sendo elas o TensorFlow e o Keras. O TensorFlow é uma biblioteca de código aberto desenvolvida pelo Google (Google LLC, 2015) para computação numérica e aprendizado de máquina. Ele é amplamente utilizado para criar e treinar modelos de aprendizado profundo, incluindo redes neurais convolucionais (CNNs) como a U-Net. O Keras (Keras Team, 2015) é uma *Application Programming Interface* - API (Interface de programação de Aplicações) de alto nível que roda sobre o TensorFlow, facilitando a construção e o treinamento de modelos de aprendizado profundo. Ele oferece uma interface simples e intuitiva para definir camadas, otimizar modelos e realizar tarefas comuns em aprendizado de máquina. Para este trabalho, utilizaremos o TensorFlow e o Keras para implementar a arquitetura da U-Net e integrar o sistema de inferência neuro-fuzzy adaptativo (ANFIS) para segmentação de imagens de ultrassom. As principais funções e classes do TensorFlow e Keras que serão utilizadas incluem:

- *Input*: Função para definir a entrada do modelo.

- *Conv2D*: Camada de convolução bidimensional para extrair características das imagens.
- *MaxPooling2D*: Camada de *pooling* para reduzir a dimensionalidade das características extraídas.
- *Conv2DTranspose*: Camada de convolução transposta para realizar a operação de expansão da imagem na arquitetura da U-Net.
- *Concatenate*: Função para concatenar as camadas de características durante a fase de expansão da U-Net.
- *Dense*: Camadas com unidades totalmente conectadas, especialmente útil para a parte de *Multitask* da U-Net.
- *Model*: Classe base para criar modelos personalizados.
- *compile*: Função para configurar o modelo para treinamento, especificando o otimizador, a função de perda e as métricas de avaliação.
- *fit*: Função para treinar o modelo com os dados de treinamento.
- *evaluate*: Função para avaliar o desempenho do modelo com os dados de teste.
- *predict*: Função para fazer previsões com o modelo treinado.
- *Reshape*: Função para transformar a dimensão dos dados.
- *Dropout*: Camada para regularização do modelo, ajudando a prevenir o *overfitting*.

Cada uma dessas funções e classes foram implementadas em um algoritmo de treinamento para a U-Net *Multitask* e a U-Net *Multitask* com inferência Fuzzy.

4.1.4 Aquisição e Pré-processamento das Imagens

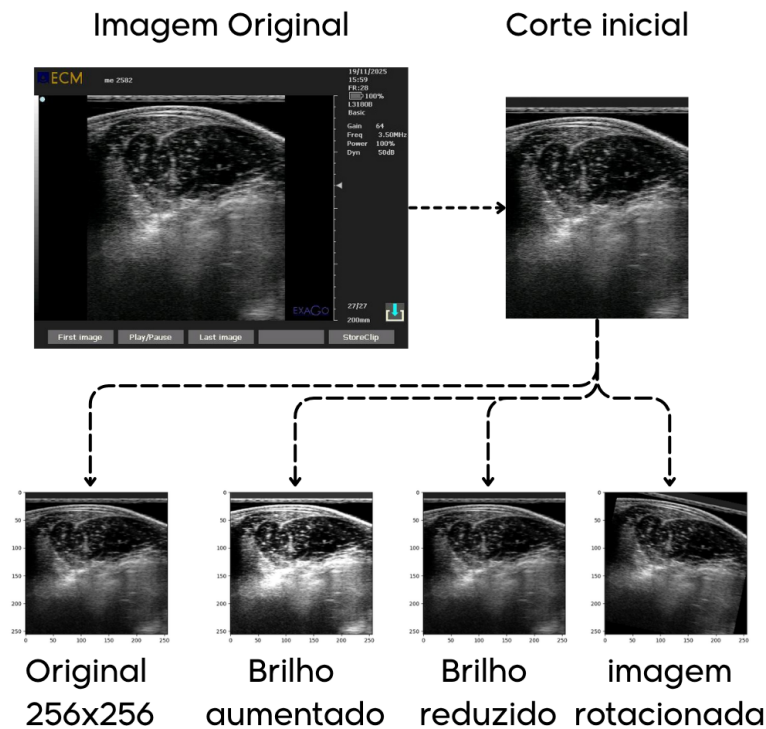
O total de imagens enviadas e demarcadas pelo Centro Avançado de Pesquisa Tecnológica do Agronegócio, foram 134 imagens de ultrassom (134 imagens da região de olho do lombo e 134 imagens da região da garupa), sendo 108 da raça Nelore e 27 da raça Caracu. Essas imagens foram divididas em conjuntos de treinamento (70%), validação (20%) e teste (10%). Devido à quantidade extremamente reduzida de imagens disponível para o treinamento de um modelo de aprendizado profundo, foi necessário aplicar um processo de aumento de dados (*data augmentation*) para ampliar o conjunto de amostras utilizado no treinamento. Além disso, foi realizado um recorte nas imagens para remover informações irrelevantes, que poderiam introduzir vieses no treinamento e comprometer o desempenho do modelo. O processo de aumento de dados incluiu as seguintes transformações:

- Rotação: As imagens foram rotacionadas em ângulos dentro do intervalo de $[-15^\circ, 15^\circ]$ para criar variações na orientação das imagens.

- Aumento e redução de brilho: As imagens foram aumentadas e reduzidas em brilho para simular diferentes condições de iluminação.

Todas as modificações feitas foram realizadas observando as características das imagens de ultrassom presentes no banco de dados. Como cada imagem foi modificada tres vezes, o processo de aumento de dados resultou em um total de 372 imagens para o treinamento do modelo (adicionando também as imagens originais), pois $70\% \times 134 = 93$ e $93 \times 4 = 372$. O pré-processamento das imagens incluiu a normalização dos valores dos pixels para o intervalo $[0, 1]$ e o redimensionamento das imagens para um tamanho fixo de 256×256^1 pixels. Além disso, as máscaras de segmentação foram criadas manualmente, com a ajuda de uma especialista, utilizando a biblioteca do Python, *label studio* (HumanSignal, 2020), em que as regiões correspondentes à AOL, EGL e EGG foram segmentadas para serem utilizadas como “rótulos” durante o treinamento do modelo. A Figura 4.6 mostra um exemplo do processo de pré-processamento.

Figura 4.6: Exemplo do processo de pré-processamento das imagens de ultrassom.



Fonte: Elaboração do Autor.

Além das imagens foram enviadas as medidas numéricas correspondentes à AOL, EGL e EGG, que foram obtidas por meio de demarcação manual utilizando o aplicativo *ImageJ* (National Institutes of Health, 2026), como mostrado nas Figuras 4.3, 4.4 e 4.5. As medidas foram enviadas em um arquivo do Excel, em que a primeira coluna continha o número de identificação do animal, a segunda coluna continha a medida da AOL, a terceira coluna continha a medida

¹As imagens após o corte inicial tinham dimensões de 333x403, a escolha de 256x256 foi feita para que a imagem não fosse muito expandida (aumento de pixels) e que não perdesse muita resolução (diminuição de pixels). Outros testes foram feitos para resoluções diferentes que resultaram em um desempenho inferior.

da EGL e a quarta coluna continha a medida da EGG. Essas medidas foram utilizadas para calcular o erro absoluto médio (MAE) entre as medidas previstas pelo modelo e as medidas reais.

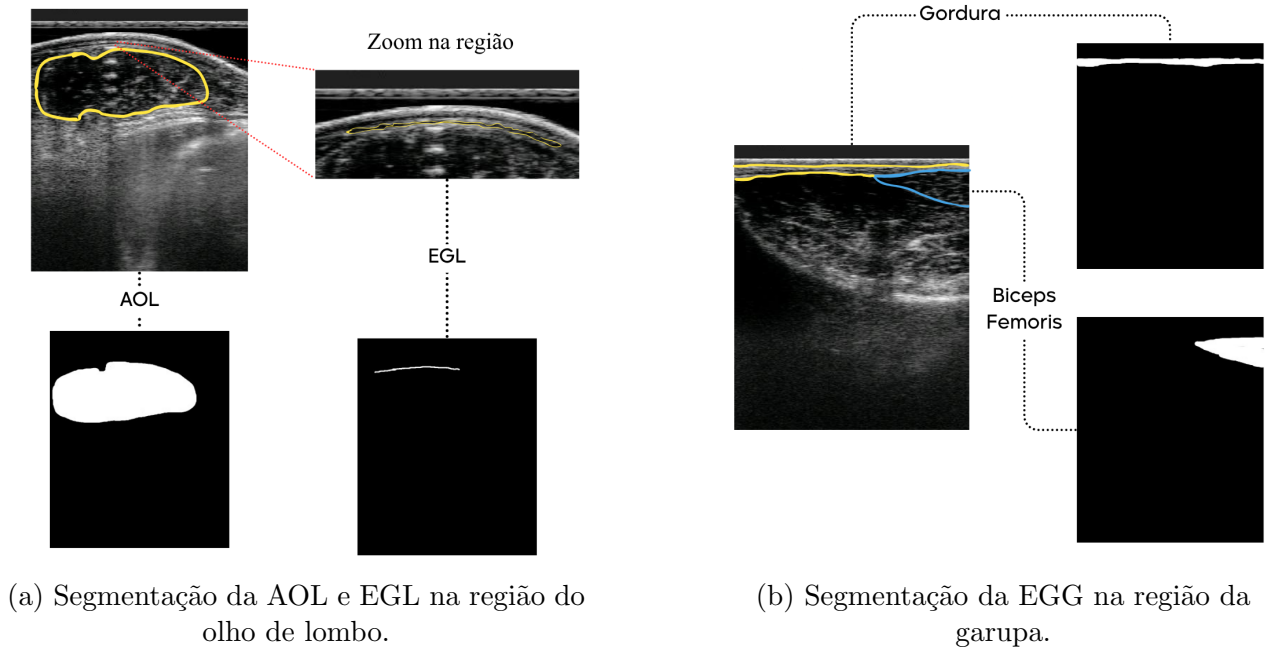
4.1.5 Máscaras

Cada medida presente nas imagens de ultrassom são números reais, por exemplo, a AOL é medida em centímetros quadrados, a EGL e a EGG são medidas em milímetros. Uma ideia inicial foi desenvolver uma *CNN* (1.3.2) que recebia as imagens de ultrassom e retornava como saída os valores numéricos correspondentes à AOL, EGL e EGG, ou seja, uma rede de regressão. No entanto, devido à quantidade extremamente baixa de imagens disponíveis para o treinamento do modelo, foi necessário realizar uma adaptação na abordagem inicial. Em vez de treinar uma *CNN* para prever os valores numéricos das medidas, optou-se por criar máscaras de segmentação para cada medida (AOL, EGL e EGG) a partir das imagens de ultrassom (MELO *et al.*, 2022). As máscaras de segmentação da AOL foram criadas demarcando a região correspondente à área do músculo, as máscaras de segmentação da EGL foram criadas demarcando a região correspondente à camada de gordura sobre o músculo da AOL (*Longissimus dorsi*) e as máscaras de segmentação da EGG foram criadas demarcando a região correspondente à camada de gordura sobre a região da garupa e também demarcando o músculo *Biceps Femoris* pois, a medida da EGG acontece em um local específico da região.

Foram treinados quatro modelos, um para cada medida, dois modelos para as máscaras da EGG. Após o treinamento, a saída do modelo é processada para extrair as medidas numéricas correspondentes. Para a AOL, a área da região segmentada é calculada e convertida para centímetros quadrados. Para a EGL, a espessura da camada de gordura é medida a partir de 3/4 da região segmentada, da esquerda para a direita, e convertida para milímetros. Para o EGG, a espessura da camada de gordura na garupa é medida na gordura que fica sobre o começo do músculo *Biceps Femoris*. Essa abordagem permitiu contornar a limitação de dados e ainda assim obter as medidas numéricas desejadas a partir das imagens de ultrassom. As máscaras de segmentação estão em preto e branco, onde as regiões em branco representam as áreas de interesse (AOL, EGL e EGG) e as regiões em preto representam o fundo da imagem. A Figura 4.7 mostra um exemplo das máscaras de segmentação criadas para as imagens de ultrassom. A Figura 4.7(a) mostra a segmentação da AOL e EGL na região do olho de lombo. A Figura 4.7(b) mostra a segmentação da EGG na região da garupa.

Cada imagem de animal presente no banco de dados vinha com um número de indentificação, como pode ser visto em cada uma das Figuras 4.2, 4.3, 4.4 e 4.5 (canto superior esquerdo começando com “me”). Animais com o primeiro dígito começando em 6, são animais da raça Nelore, enquanto animais com o primeiro dígito começando em 2, são animais da raça Caracu. Essa indentificação serviu para terminar o desenvolvimento dos dados e criar as variáveis de entrada da rede no padrão visto na seção 3.1.3, ou seja, cada variável de entrada da rede será composta por uma imagem de ultrassom e a raça do animal correspondente à imagem, sendo codificada no formato *One-Hot Encoding*, em que a raça Nelore é representada por [1, 0] e a

Figura 4.7: Exemplo das máscaras de segmentação criadas para as imagens de ultrassom.



(a) Segmentação da AOL e EGL na região do olho de lombo.

(b) Segmentação da EGG na região da garupa.

Fonte: Elaboração do Autor.

raça Caracu é representada por $[0, 1]$.

4.2 Treinamento e Métricas de Avaliação

Como foram desenvolvidos quatro modelos, o processo de treinamento foi realizado separadamente para cada modelo. O treinamento incluiu a definição da arquitetura da U-Net, a configuração dos hiperparâmetros do modelo, a escolha da função de perda, do otimizador e a execução do processo de treinamento utilizando os dados de treinamento e validação.

Cada medida (AOL, EGL e EGG) teve dois modelos, um modelo U-Net *Multitask* e um modelo de U-Net *Multitask* com fuzzy.

- U-Net: Algumas modificações foram feitas no modelo original proposto por (RONNEBERGER; FISCHER; BROX, 2015), sendo elas:
 - No *encoder* a rede é dividida em blocos, cada bloco é composto por duas camadas de convolução (*Conv2D*) com *kernels* de tamanho 3×3 , *stride* de tamanho 1, *padding* “same”² e função de ativação ReLU, seguidas por uma camada de *MaxPooling* para reduzir a dimensão da imagem, com *kernel* de tamanho 2×2 , *stride* de tamanho 2. No total foram 4 blocos no *encoder*, com o número de filtros dobrando a cada bloco, começando com 32 filtros no primeiro bloco e chegando a 128 filtros no quarto bloco.

²Na U-Net original é utilizada a convolução válida (*padding = valid*), que diminui a resolução em cada etapa de convolução e introduz menos ruído para a rede, já que não é necessário expandir as bordas da imagem (1.23), entretanto optou-se pela convolução *same* pois obteve-se melhor resultado.

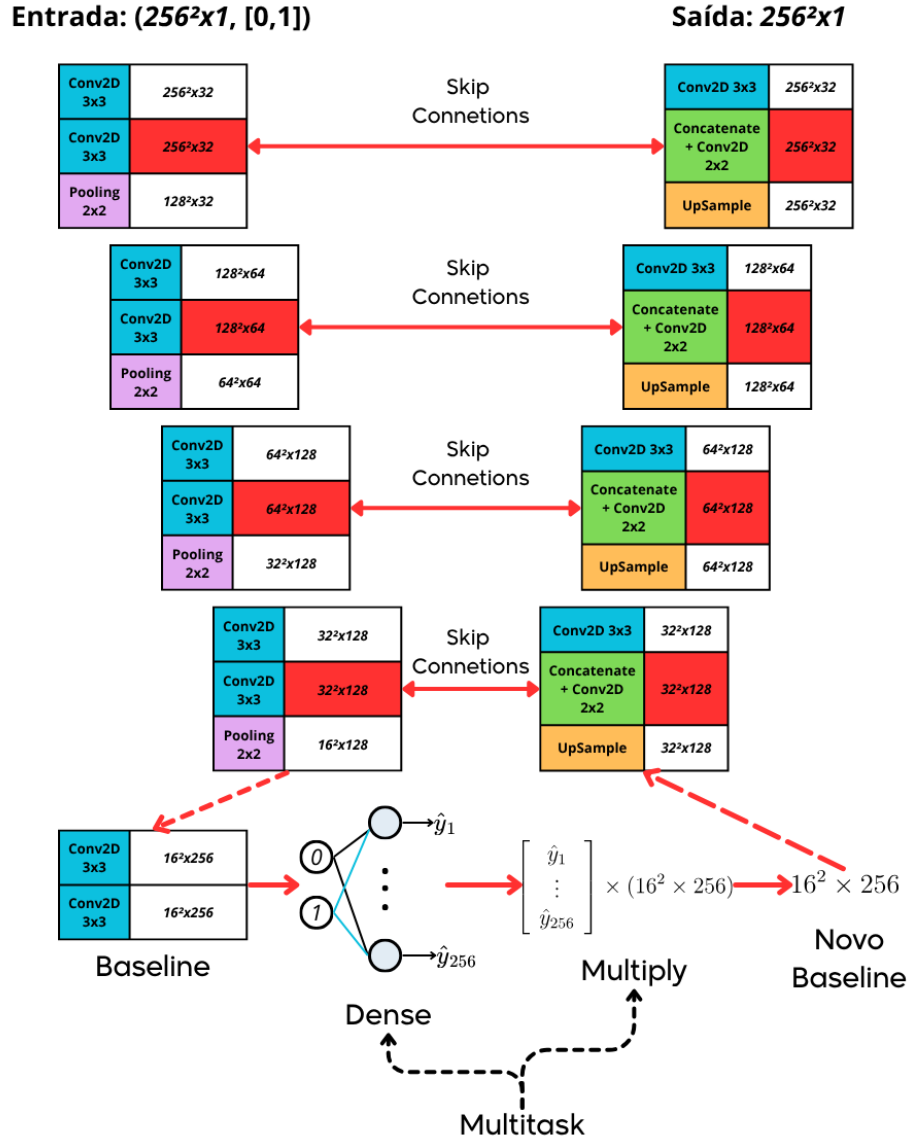
- No *baseline* tem-se duas camadas de convolução (*Conv2D*) com *kernel* de tamanho 3×3 , *stride* de tamanho 1, *padding* “same” e função de ativação ReLU.
- Para levar em consideração a raça do animal ([0,1] Nelore e [1,0] Caracu) é adicionada uma camada *Multitask* na parte de *baseline* da U-Net, de forma similar a vista na subseção 3.1.3.
- No *decoder* a rede também é dividida em blocos, cada bloco é composto por uma camada de convolução transposta (*Conv2DTranspose*) com *kernel* de tamanho 2×2 , *stride* de tamanho 2 e *padding* “same”, seguida por uma camada de concatenação para concatenar as características extraídas pelo *encoder* com as características extraídas pelo *decoder*, seguida por uma camada de convolução (*Conv2D*) com *kernel* de tamanho 3×3 , *stride* de tamanho 1, *padding* “same” e função de ativação ReLU. Como a dimensão da imagem não é reduzida em cada etapa de convolução do *encoder* não é necessário realizar a técnica de *copy and crop* vista na U-Net original (3.1), pois as características extraídas pelo *encoder* e pelo *decoder* possuem a mesma dimensão espacial como mostra a Figura 4.8, o que facilita a concatenação das características. No total foram 4 blocos no *decoder*, com o número de filtros reduzindo pela metade a cada bloco, começando com 256 filtros no primeiro bloco e chegando a 32 filtros no quarto bloco. A camada de saída é composta por uma camada de convolução (*Conv2D*) com um único *kernel* de tamanho 1×1 , *stride* de tamanho 1 e função de ativação *Sigmoid* para produzir a máscara de segmentação.

A Figura 4.8 mostra a arquitetura da U-Net utilizada e como as dimensões das imagens vão se modificando ao longo da rede. A cor azul representa as camadas de convolução com filtros de tamanho 3×3 , a cor roxa representa as camadas de *MaxPooling*, a cor laranja representa as camadas de convolução transposta (*UpSample*) com filtros de tamanho 2×2 e a cor verde representa as camadas de concatenação e convoluções com filtros de tamanho 2×2 . As setas indicam o caminho dos dados ao longo da rede, mostrando como as características extraídas pelo *encoder* são concatenadas com as características extraídas pelo *decoder* (*Skip Connection*) para produzir a máscara de segmentação final. O *Multitask* após o *baseline* é a representação visual das equações vistas na subseção (3.1.3), com *Dense* e *Multiply* sendo as funções do *Tensorflow* utilizadas para criar a camada.

Em todos os blocos foram adicionadas camadas de *Dropout* (1.2.6) com uma taxa de 0.1 para regularização do modelo e prevenção de *overfitting* (1.2.4). O modelo foi treinado utilizando o otimizador Adam com taxa de aprendizado inicial de 0.001 (1.2.7), utilizando a descida do gradiente estocástico com um tamanho de lote de 4 (1.2.3), um número máximo de épocas definido em 100 e função de custo sendo a *Binary Cross-Entropy* (1.2.9). O processo de treinamento foi monitorado utilizando os dados de validação com o *EarlyStopping* (1.2.6) para evitar também o *overfitting*.

- U-Net *Multitask* com inferência Fuzzy: Como comentado na subseção (3.1.3) a U-Net

Figura 4.8: Arquitetura da U-Net utilizada e as dimensões das imagens ao longo da rede.



Fonte: Elaboração do Autor.

Multitask com inferência Fuzzy compartilha quase todas as definições da U-Net *Multitask* (*encoder* e *decoder* iguais), entretanto, o *baseline* é alterado para incluir mais uma modificação importante, a integração do sistema de inferência neuro-fuzzy adaptativo (ANFIS). Com isso, algumas variáveis extras precisam ser definidas:

- Número de Conjuntos Fuzzy: O número de conjuntos fuzzy para cada variável de entrada do ANFIS foi definido como 3, ou seja, “Tendência baixa de pertencer a região segmentada”, “Tendência média de pertencer a região segmentada” e “Tendência alta de pertencer a região segmentada”.
- Número de Regras Fuzzy: O número de regras fuzzy foi definido como 9, considerando todas as combinações possíveis dos conjuntos fuzzy para a variável de entrada do *baseline*.

Os modelos foram avaliados utilizando as seguintes métricas de avaliação:

- *Dice Coefficient*: Principal medida para avaliar o desempenho dos modelos na segmentação das imagens, como abordado na subseção (3.1.2).
- *Acurácia*: A acurácia é uma métrica de avaliação utilizada para medir a proporção de pixels classificados corretamente pelo modelo em relação ao total de pixels na imagem. Para o contexto de segmentação de imagens (MELO *et al.*, 2022), se C_T for os pixels da região de AOL traçada pelo especialista e C o conjunto dos pixels obtidos pelo método de predição. Os pixels definidos como verdadeiros positivos são $VP = C_T \cap C$, enquanto os pixels definidos como verdadeiros negativos são $VN = \{p | p \notin C_T \wedge p \notin C\}$. Os pixels falsos positivos são $FP = \{p | p \notin C_T \wedge p \in C\}$ enquanto os pixels falsos negativos satisfazem $FN = \{p | p \in C_T \wedge p \notin C\}$. Com isso, a acurácia é calculada utilizando a fórmula (4.1).

$$\text{Acurácia} = \frac{VP + VN}{VP + VN + FP + FN}. \quad (4.1)$$

Para o contexto de segmentação de imagens, a acurácia nem sempre é tão precisa quanto o Dice. Caso a imagem de segmentação seja uma região pequena em comparação com o fundo da imagem, o modelo pode classificar a maioria dos pixels como fundo e ainda assim obter uma acurácia alta, mesmo que a segmentação da região de interesse seja ruim.

- *Número de Épocas*: O número de épocas é uma métrica que indica o número de vezes que o modelo foi treinado com todo o conjunto de dados de treinamento. Ele é importante para avaliar a convergência do modelo, ou seja, se o modelo está aprendendo e melhorando seu desempenho ao longo do tempo. Também é um importante indicador de *overfitting* (1.2.4).
- R^2 : O coeficiente de determinação (R^2) (CANDIDO, 2024) é uma métrica de avaliação utilizada para medir a proporção da variância dos dados que é explicada pelo modelo. Ele varia entre 0 e 1, em que 1 indica que o modelo explica toda a variância dos dados

e 0 indica que o modelo não explica nenhuma variância dos dados. O R^2 é calculado utilizando a seguinte fórmula:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (4.2)$$

em que y_i são as medidas reais, $\hat{y}_i$ são as medidas previstas pelo modelo e $\bar{y}$ é a média das medidas reais. O R^2 é uma métrica importante para avaliar a qualidade das previsões do modelo, especialmente em tarefas de regressão, em que o objetivo é prever valores numéricos a partir de dados de entrada. Como o objetivo é prever as medidas numéricas correspondentes à AOL, EGL e EGG a partir das imagens de ultrassom, o R^2 é uma métrica relevante para avaliar a precisão das previsões do modelo em relação às medidas reais.

- Erro Absoluto Médio (MAE): O erro absoluto médio (MAE) (MELO *et al.*, 2022) é uma métrica de avaliação utilizada para medir a média dos erros absolutos entre as medidas previstas pelo modelo e as medidas reais. Ele é calculado utilizando a seguinte fórmula:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (4.3)$$

em que y_i são as medidas reais e $\hat{y}_i$ são as medidas previstas pelo modelo.

4.2.1 *Hardware e Software*

O desenvolvimento e treinamento dos modelos foram realizados utilizando um notebook Asus TUF *Gaming* F15 com as seguintes especificações de hardware:

- Processador: Intel Core i7-12700H
- Memória RAM: 8 GB
- Placa de Vídeo: NVIDIA GeForce RTX 3050 mobile com 4 GB de memória dedicada
- Armazenamento: SSD de 512 GB

O *software* utilizado para o desenvolvimento e treinamento dos modelos foi o *Visual Studio Code* (Microsoft, 2015), com a extensão do Python instalada. O sistema operacional foi o Linux Mint 22.3 "Zena" (Linux Mint Team, 2026). O treinamento das redes foi realizado na GPU, utilizando o TensorFlow com suporte para GPU, o que permitiu acelerar o processo de treinamento dos modelos.

4.2.2 Resultados e Análise

4.2.3 AOL

- U-Net *Multitask*: O modelo de U-Net *Multitask* para a AOL apresentou um desempenho satisfatório, com uma boa capacidade de segmentação da região correspondente à área do músculo. Os seguintes resultados foram obtidos para as métricas de avaliação, com o modelo sendo treinando quatro vezes para garantir a consistência dos resultados (validação cruzada):
 - Dice *Coefficient*: O modelo alcançou um Dice *Coefficient* médio de 93,85% no conjunto de validação, indicando uma boa sobreposição entre as máscaras de segmentação preditas e as máscaras de segmentação reais.
 - Acurácia: Como a região segmentada é relativamente grande em comparação com o fundo da imagem, a acurácia serviu como uma métrica complementar, o modelo apresentou uma acurácia média de 94,75%, indicando que a maioria dos pixels foram classificados corretamente como pertencentes à região segmentada ou ao fundo.
 - Número de Épocas: A média de convergência do modelo foi de 80 épocas das 100 épocas máximas definidas.
 - R^2 : O modelo apresentou um R^2 médio de 0,9367 entre as medidas preditas e as medidas reais da AOL.
 - Erro Absoluto Médio (MAE): O modelo apresentou um erro absoluto médio (MAE) de 2,15 cm² entre as medidas preditas e as medidas reais da AOL, indicando que, em média, as previsões do modelo estavam a uma distância de 2,15 cm² das medidas reais.
- U-Net *Multitask* com inferência Fuzzy: O modelo de U-Net *Multitask* com inferência Fuzzy para a AOL apresentou um desempenho superior em comparação com o modelo de U-Net *Multitask* sem fuzzy. A integração do sistema de inferência neuro-fuzzy adaptativo (ANFIS) permitiu que o modelo capturasse melhor as nuances e incertezas presentes nas imagens de ultrassom, resultando em uma segmentação mais precisa da região correspondente à área do músculo. As métricas de avaliação indicaram uma redução significativa no erro absoluto médio (MAE) em comparação com o modelo sem fuzzy, demonstrando a eficácia da integração do ANFIS na melhoria do desempenho do modelo para a tarefa de segmentação da AOL.

4.2.4 EGL

4.2.5 EGG

Referências Bibliográficas

- ABCCARACU. *História da raça caracu*. 2022. Acessado: 2026-04-03. Disponível em: <https://www.abccaracu.com.br/historia-da-raca-caracu>.
- ABIEC. *Beef Report 2025 — Perfil da Pecuária no Brasil*. 2021. Acessado: 2026-04-3. Disponível em: <https://abiec.com.br/publicacoes/beef-report-2025-perfil-da-pecuaria-no-brasil/>.
- BEAR, M. F.; CONNORS, B. W.; PARADISO, M. A. *Neurociências: desvendando o sistema nervoso*. 4. ed. Porto Alegre: Artmed, 2017. ISBN 9788582713091.
- BENGIO, Y. Practical recommendations for gradient-based training of deep architectures. *Cornell University*, Version 2, 2012.
- BOOK, D. L. *Capítulo 10: As Principais Arquiteturas de Redes Neurais*. 2025. Acessado: 2026-03-26. Disponível em: <https://www.deeplearningbook.com.br/>.
- BOOK, D. L. *Capítulo 19: Overfitting e Regularização: Parte 1*. 2025. Acessado: 2026-01-26. Disponível em: <https://www.deeplearningbook.com.br/overfitting-e-regularizacao-parte-1/>.
- BOOK, D. L. *Capítulo 22 – Regularização L1*. 2025. Acessado: 2026-03-26. Disponível em: <https://www.deeplearningbook.com.br/>.
- CANDIDO, G. *R Quadrado: coeficiente de determinação*. 2024. Acesso em: 23 abr. 2026. Disponível em: <https://medium.com/@lg0702/r-quadrado-3318259ff2e2>.
- CHOWDHURY, M. *Padding and Strides in CNN*. 2024. Acessado: 2026-04-14. Disponível em: <https://medium.com/@minhazc.engg/padding-and-strides-in-cnn-58dc56493887>.
- CREVIER, D. *AI: The Tumultuous History of the Search for Artificial Intelligence*. New York: Basic Books, 1993. ISBN 0-465-02997-3.
- Data Science Academy. *Comparativo Técnico – Label Encoding vs One-Hot Encoding em Machine Learning*. 2025. Acessado: 2026-04-13. Disponível em: <https://blog.dsacademy.com.br/comparativo-tecnico-label-encoding-vs-one-hot-encoding-em-machine-learning/>.
- ELHARROUSS, O.; MAHMOOD, Y.; BECHQITO, Y.; SERHANI, M. A.; BADIDI, E.; RIFFI, J.; TAIRI, H. Loss functions in deep learning: A comprehensive review. *arXiv preprint arXiv:2504.04242*, 2025. Disponível em: <https://arxiv.org/abs/2504.04242>.
- FELICIO, P. E. de. Classificação, tipificação e qualidade da carne bovina. *VI Congresso Brasileiro de Ciência e Tecnologia de Carnes*, São Pedro, SP, Brasil, p. 9, 2010.
- GARCAO, L. M. da C. Características socioeconômicas da pecuária brasileira: características socioeconômicas de los bovinos brasileños. *Revista Mirante*, Anápolis, GO, Brasil, jun. 2015. ISSN 1981-4089.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. [S.l.]: MIT Press, 2016. <http://www.deeplearningbook.org>.

Google LLC. *TensorFlow*. 2015. Acesso em: 11 abr. 2026. Disponível em: <https://www.tensorflow.org/>.

Grupo Agro Paraná. *Ultrassom de carcaça: precisão genética com tecnologia*. 2024. Acesso em: 19 abr. 2026. Disponível em: <https://grupoagroparana.com.br/ultrassom-de-carcaca-precisao-genetica-com-tecnologia/>.

GUARALDO, M. C. *Brasil é o quarto maior produtor de grãos e o maior exportador de carne bovina do mundo, diz estudo*. 2021. Acessado: 2026-04-3. Disponível em: <https://www.embrapa.br/busca-de-noticias/-/noticia/62619259/brasil-e-o-quarto-maior-produtor-de-graos-e-o-maior-exportador-de-carne-bovina-do-mundo-diz-estudo>

HARISHA, L. *Dice Coefficient! What is it?* 2023. Acessado: 2026-03-28. Disponível em: <https://lathashreeh.medium.com/dice-coefficient-what-is-it-ff090ec97bda>.

HECHT-NIELSEN, R. Theory of the backpropagation neural network. *Academic Press*, 1992.

HumanSignal. *Label Studio: Open Source Data Labeling Platform*. 2020. Acesso em: 11 abr. 2026. Disponível em: <https://labelstud.io/>.

JAFELICE, R. S. M.; BARROS, L. C.; BASSANEZI, R. C. *Teoria dos Conjuntos Fuzzy com Aplicações*. 3. ed. São Paulo -SP, Brasil: SBMAC, 2023. ISBN 978-65-86399-14-5.

Keras Team. *Keras: Deep Learning for Humans*. 2015. Acesso em: 11 abr. 2026. Disponível em: <https://keras.io/>.

KORSHUNOVA, K. P. A convolutional fuzzy neural network for image classification. In: *Proceedings of the IEEE Conference*. [S.l.]: IEEE, 2018.

LAMAS, F. M. *Artigo - A evolução da agricultura no Brasil*. 2023. Acessado: 2026-04-3. Disponível em: https://www.embrapa.br/web/portal/tema-integracao-lavoura-pecuaria-floresta-ilpf/busca-de-noticias/-/noticia/81665485/artigo---a-evolucao-da-agricultura-do-brasil?p_auth=HeYmQbdG.

LECUN, Y. *The MNIST Database of Handwritten Digits*. 1998. Acessado: 2026-02-22. Disponível em: <http://yann.lecun.com/exdb/mnist/>.

LETTVIN, J. Y.; MATURANA, H. R.; MCCULLOCH, W. S.; PITTS, W. H. What the frog's eye tells the frog's brain. *Proceedings of the Institute of Radio Engineers*, v. 47, n. 11, p. 1940–1951, 1959.

Linux Mint Team. *Linux Mint*. 2026. Acesso em: 23 abr. 2026. Disponível em: <https://linuxmint.com/>.

MCCULLOCH, W.; PITTS, W. A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 1943.

MELO, M. Jaderson de; GONÇALVES, D. N.; GOMES, M. de N. B.; FARIA, G.; SILVA, J. de A.; RAMOS, A. P. M.; OSCO, L. P.; FURUYA, M. T. G.; JUNIOR, J. M.; GONÇALVES, W. N. Automatic segmentation of cattle rib-eye area in ultrasound images using the UNet++ deep neural network. *Computers and Electronics in Agriculture*, Elsevier, v. 195, p. 106818, 2022.

- Microsoft. *Visual Studio Code*. 2015. Acesso em: 23 abr. 2026. Disponível em: <https://code.visualstudio.com/>.
- MÜLLER, V. C. Review of margaret boden, “mind as machine: A history of cognitive science”. *Minds and Machines*, Springer, v. 18, n. 1, p. 121–125, 2008.
- National Institutes of Health. *ImageJ: Image Processing and Analysis in Java*. 2026. Acessado em: 2026-04-04. Disponível em: <https://imagej.net/ij/>.
- O’SHEA, K.; NASH, R. An introduction to convolutional neural networks. *arXiv preprint arXiv:1511.08458*, 2015. Disponível em: <https://arxiv.org/abs/1511.08458>.
- PURWONO, P.; NATALIANI, Y.; PURNOMO, H. D.; TIMOTIUS, I. K. Dfu-fuzzyliteunet: A lightweight u-net with fuzzy sigmoid and lite transformer for diabetic foot ulcer segmentation. *Biomedical Signal Processing and Control*, Elsevier, v. 108, p. 107902, 2025.
- RONNEBERGER, O.; FISCHER, P.; BROX, T. U-net: Convolutional networks for biomedical image segmentation. In: *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*. Munich, Germany: Springer, 2015. p. 234–241.
- ROSENBLATT, F. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 1958.
- RUDER, S. An overview of gradient descent optimization algorithms. *Insight Centre for Data Analytics, NUI Galway Aylien Ltd., Dublin*, 2016.
- SENA, V.; JAFELICE, R. S. M.; SANTOS, C. A. R. dos. Reconstrução geométrica de uma função através de uma rede neural. In: *Anais da XXIV Semana da Matemática e XIV Semana da Estatística (SEMAT & SEMEST)*. Uberlândia, MG, Brasil: [s.n.], 2024. Disponível em: https://drive.google.com/file/d/1OmN2Gq6i3nHR7ZtEVitc5iCVH0ZFUY_8/view.
- SOVIERZOSKI, M. A. Convolução de sinais: definição, propriedades e ferramentas. *Revista Ilha Digital*, v. 2, p. 81–95, 2010. ISSN 2177-2649. Disponível em: <https://ilhadigital.florianopolis.ifsc.edu.br/index.php/ilhadigital>.
- STANFIELD, C. L. *Fisiologia Humana*. São Paulo: Pearson Education do Brasil, 2014. ISBN 978-85-430-1426-5.
- SUGUISAWA, L.; MATOS, B. da Conceição de; SUGUISAWA, J. M. Uso da ultrassonografia na avaliação de características de carcaça e de qualidade da carne. In: ROSA, A. do N.; MARTINS, E. N.; MENEZES, G. R. de O.; SILVA, L. O. C. da (Ed.). *Melhoramento Genético Aplicado em Gado de Corte*. Brasília, DF, Brasil: Embrapa, 2013. p. 98–107.
- SUTSKEVER, I. On the importance of initialization and momentum in deep learning. *International Conference on Machine Learning*, 2013.
- SZELISKI, R. *Computer Vision: Algorithms and Applications*. Cham, Suíça: Springer, 2010. ISBN 978-3030343712.
- TREVISAN, V. *Como funcionam as Redes Neurais Convolucionais (CNNs)*. 2021. Acessado: 2025-12-16. Disponível em: <https://medium.com/data-hackers/como-funcionam-as-redes-neurais-convolucionais-cnns-71978185c1>.
- VERMA, H.; IM, K.; GUPTA, A.; TANVEER, M. A novel framework using intuitionistic fuzzy logic with u-net and u-net++ architecture: A case study of mri brain image segmentation. *arXiv preprint arXiv:2603.18042*, 2026. Disponível em: <https://arxiv.org/abs/2603.18042>.

WERBOS, P. J. Backpropagation through time: What it does and how to do it. *PROCEEDINGS OF THE IEEE*, 1990.

WILSON, D. E. *Scanning into the Future: Real-Time Ultrasound in Beef Cattle*. 1998. ASB 1998-DEW-417. Disponível em: <https://admin.webplus.com.br/public/upload/downloads/090220121626346252000AHZV.pdf>.

ZADEH, L. A. Fuzzy sets. *Information and Control*, v. 8, n. 3, p. 338–353, 1965.

ZANCANER, R. *História da raça nelore*. 2026. Acessado: 2026-04-03. Disponível em: <https://zancaner.com.br/site/conteudo/1-historia-da-raca-nelore.html>.

ZHU, H.; ROHLING, R.; SALCUDEAN, S. Multi-task unet: Jointly boosting saliency prediction and disease classification on chest x-ray images. *arXiv preprint arXiv:2202.07118*, 2022. Disponível em: <https://arxiv.org/abs/2202.07118>.